

Master Degree in Aerospace Engineering
2025-2026

Master Thesis

“Spiking Neural Networks for Lift
Coefficient Reconstruction in Extreme
Vortex-Gust Airfoil Interactions”

Jaume Manresa Rigo

Tutor: Iacopo Tirelli
Leganés, 2026

ABSTRACT

Extreme vortex-gust interactions can produce large and rapidly varying aerodynamic loads, motivating estimation methods that combine accuracy with low computational cost for future onboard applications. This requirement is particularly relevant because conventional neural networks may demand substantial processing power, whereas the human brain is estimated to perform approximately 10^{15} – 10^{16} synaptic operations per second while consuming only 10–20 W, illustrating the potential of sparse and event-driven computation. Inspired by this efficiency, this thesis investigates whether a Spiking Neural Network (SNN) based architecture can reconstruct the instantaneous lift coefficient from time-resolved vorticity fields encoded as binary spike trains. The proposed framework uses a rate-based spike encoding technique with a convolutional SNN composed of Leaky Integrate-and-Fire neurons, trained using stateful truncated backpropagation through time. The methodology is evaluated using 155 numerical simulations of a NACA 0012 airfoil covering different angles of attack and vortex-gust conditions, while ablation studies assess the influence of input temporal resolution, spike encoding, training strategy, and latent space dimension. On the test set, the reference SNN achieves a mean relative L_2 error of 11.33% and a mean cosine similarity of 0.9919. Although a parameter-matched convolutional neural network (CNN) provides higher reconstruction accuracy, it requires approximately 17 times more estimated arithmetic inference energy, whereas the SNN remains competitive with an energy-matched CNN. These results indicate that spike-based processing can preserve sufficient aerodynamic information for accurate lift reconstruction while offering a more energy-efficient alternative for future embedded aerodynamic estimation.

Keywords: Spiking Neural Networks, neuromorphic computing, extreme aerodynamics, vortex-gust airfoil interactions, energy-efficient inference.

CONTENTS

1. INTRODUCTION.	1
1.1. Motivation and Scientific Context	1
1.2. Objective and Research Questions	3
2. BACKGROUND AND STATE OF THE ART	6
2.1. Extreme Vortex-Gust Interactions and Aerodynamic Load Response	6
2.2. Data-Driven Methods for Flow Reconstruction and Estimation	8
2.3. Low-Dimensional Representations and Load-Relevant Coordinates	9
2.4. Sparse and Event-Based Sensing for Aerodynamic Estimation	11
2.5. Principles of Spiking Neural Networks	13
2.5.1. Leaky Integrate-and-Fire (LIF) Neuron Model	14
2.5.2. Spike Encoding Techniques	17
2.5.3. Training Spiking Neural Networks	21
3. PROBLEM FORMULATION	27
3.1. Physical Statement of the Reconstruction Problem	27
3.2. Input Representation: the Non-Dimensional Vorticity Field	27
3.3. Supervised Reconstruction Problem	28
3.4. Spike-Based Reformulation	29
4. DATASET	31
4.1. Source Dataset and Physical Configuration	31
4.2. Parametric Space	32
4.3. Available Flow and Load Variables	35
5. SPIKE ENCODING AND DATA PRE-PROCESSING	36
5.1. From Vorticity Fields to Spike-Based Network Inputs	36
5.2. Dataset Temporal Interpolation	37
5.3. Robust Normalization and Clipping	39
5.4. Spike Encoding	41
5.4.1. Rate Encoding	41
5.4.2. Delta Encoding	45

5.4.3. Information Loss Introduced by Spike Encoding	49
6. METHODOLOGY	52
6.1. Dataset Split	52
6.2. Reference SNN Architecture	55
6.2.1. Input Tensor to the Network.	57
6.2.2. Convolutional Spiking Encoder.	58
6.2.3. Latent Representation	60
6.2.4. Lift Coefficient Readout Head	61
6.2.5. Temporal State Propagation	61
6.3. Training	62
6.3.1. Training Strategies	62
6.3.2. Optimization Details	64
6.4. Loss Function.	65
6.5. Evaluation Metrics	65
6.5.1. Accuracy Metrics	66
6.5.2. Energy-Efficiency Metrics.	68
6.6. CNN-Based Architectures.	71
6.6.1. Parameter-Matched CNN	72
6.6.2. Energy-Matched CNN	73
7. RESULTS AND DISCUSSION	75
7.1. Ablation Study	75
7.1.1. Influence of Temporal Resolution	75
7.1.2. Influence of Spike Encoding	77
7.1.3. Influence of Training Strategy	78
7.1.4. Influence of the Latent Space Dimension	80
7.2. Reference Model Reconstruction Performance	81
7.2.1. Overall Reconstruction Performance	81
7.2.2. Performance by Angle of Attack	83
7.2.3. Performance by Gust Intensity	85
7.3. Comparison with CNN-Based Architectures	89
7.3.1. Parameter-Matched CNN	89

7.3.2. Energy-Matched CNN	91
8. CONCLUSIONS, LIMITATIONS AND FUTURE WORK.	94
8.1. Conclusions.	94
8.2. Limitations and Future Work	96
BIBLIOGRAPHY.	99
A. REGULATORY FRAMEWORK	106
B. SOCIO-ECONOMIC IMPACT	108
B.1. Project Budget	108
B.2. Expected Socio-Economic Impact	109
C. CODE AVAILABILITY	112
D. DATA AVAILABILITY	113
E. AI DECLARATION OF USE	114

LIST OF FIGURES

1.1	Extreme aerodynamics on a low dimensional manifold	2
2.1	Visualization of extreme aerodynamics.	7
2.2	Spike and event camera outputs.	12
2.3	Representation of a spike train emitted by a neuron	14
2.4	LIF RC circuit	15
2.5	LIF neuron	17
2.6	Rate Encoding	18
2.7	Delta Encoding	20
2.8	Latency Encoding	21
2.9	SNN non-differentiability	22
2.10	Surrogate Gradient	24
2.11	BPTT Gradient Flow	25
4.1	Problem setup schematic	32
4.2	Vortex-gust interaction for $(\alpha, G, R, y) = (20^\circ, +3.8, 1.0, -0.5)$	34
4.3	Vortex-gust interaction for $(\alpha, G, R, y) = (60^\circ, -3.0, 1.0, +0.5)$	34
5.1	Overview of the pre-processing and spike encoding pipeline.	37
5.2	Distribution of the vorticity magnitude over the dataset.	40
5.3	Effect of robust normalization and clipping on a vorticity snapshot.	41
5.4	Effect of the exponent γ on rate encoding.	43
5.5	Effect of the gain factor g on rate encoding.	44
5.6	Rate encoding of a signed vorticity snapshot.	45
5.7	Delta encoding mechanism.	47
5.8	Effect of the delta threshold factor.	48
6.1	Random-Case Dataset Split.	54
6.2	Final train-test split in the parametric space	55
6.3	Reference SNN Architecture	57

6.4	Parameter-matched CNN architecture	72
6.5	Energy-matched CNN architecture	73
7.1	Influence of Temporal Resolution	76
7.2	Influence of Spike Encoding	77
7.3	Influence of Training Strategy	79
7.4	Influence of Latent Space Dimension	80
7.5	Overall reconstruction performance	82
7.6	C_l history for test case $\alpha = 50^\circ$, $G = 2.20$, $R = 0.50$, and $y = -0.30$	83
7.7	Relative L_2 error vs angle of attack	84
7.8	Relative L_2 error vs gust intensity	86
7.9	C_l history for test case $\alpha = 50^\circ$, $G = -3.0$, $R = 0.75$, and $y = -0.10$	87
7.10	C_l history for test case $\alpha = 30^\circ$, $G = -3.6$, $R = 0.50$, and $y = -0.30$	87
7.11	Accuracy metrics by gust sign	88
7.12	Parameter-matched CNN vs reference SNN	90
7.13	Energy-matched CNN vs reference SNN	92

LIST OF TABLES

4.1	Range of sampled gust ratios G for the 30 disturbed cases at each angle of attack.	33
6.1	Reference configuration	56
7.1	Influence of temporal resolution accuracy metrics	75
7.2	Influence of spike encoding accuracy metrics	77
7.3	Influence of training strategy accuracy metrics	79
7.4	Influence of the Latent-Space Dimension accuracy metrics	80
7.5	Overall reconstruction performance	82
7.6	Reconstruction performance by angle of attack	84
7.7	Reconstruction performance by gust intensity	85
7.8	Reconstruction and computational comparison between the reference SNN and the parameter-matched CNN	89
7.9	Reconstruction and computational comparison between the reference SNN and the energy-matched CNN	91
B.1	Estimated GPU training time for the experiments conducted in this thesis.	108
B.2	Budget	109

ACRONYMS

AC	Accumulate Operation
ANN	Artificial Neural Network
BPTT	Backpropagation Through Time
CMOS	Complementary Metal-Oxide-Semiconductor
CNN	Convolutional Neural Network
CUDA	Compute Unified Device Architecture
DMD	Dynamic Mode Decomposition
EASA	European Union Aviation Safety Agency
GAN	Generative Adversarial Network
GPU	Graphics Processing Unit
LIF	Leaky Integrate-and-Fire
MAC	Multiply-Accumulate Operation
MAE	Mean Absolute Error
MLP	Multilayer Perceptron
MSE	Mean Squared Error
NACA	National Advisory Committee for Aeronautics
PCA	Principal Component Analysis
PINN	Physics-Informed Neural Network
PIV	Particle Image Velocimetry
POD	Proper Orthogonal Decomposition
RC	Resistor-Capacitor
ReLU	Rectified Linear Unit
RMSE	Root Mean Squared Error
RNN	Recurrent Neural Network

SNN Spiking Neural Network

STDP Spike-Timing-Dependent Plasticity

TBPTT Truncated Backpropagation Through Time

VRAM Video Random Access Memory

1. INTRODUCTION

1.1. Motivation and Scientific Context

Future aerial vehicles are expected to operate with increasing levels of autonomy in environments that may contain severe atmospheric disturbances. This is particularly relevant for small unmanned aerial vehicles, whose lower flight velocities and reduced inertia make them particularly sensitive to gusts and vortical wakes. When a strong vortex interacts with a lifting surface, the response may involve flow separation, vortex deformation, vortex shedding, as well as large transients [1, 2]. Therefore, the estimation of these responses is important for the future development of active flight-control systems.

The challenge is not only aerodynamic. A time-resolved flow field contains a huge amount of spatial values at every instant. Processing this data continuously requires memory, computational resources and very low inference latency. Data-driven models have already been used in fluid mechanics for reconstruction, dimensionality reduction, state estimation and control [3]. However, their use onboard an aircraft is less straightforward. A model that performs well on a workstation or GPU might still be unsuitable for an embedded platform with limited power and memory.

This constraint motivates the search for compact and event-driven processing methods. Biological neural systems illustrate the efficiency that can be achieved: the human brain sustains on the order of 10^{15} - 10^{16} synaptic operations per second while consuming only around 10-20 W [4], whereas a conventional supercomputer performing comparable perceptual tasks requires many orders of magnitude more power. Spiking Neural Networks (SNNs) are one possible option [5]. Unlike conventional Artificial Neural Networks (ANNs), which propagate dense continuous activations, SNNs communicate through discrete spike events and preserve temporal information through the internal states of their neurons. On dedicated neuromorphic hardware, sparse spiking activity can potentially be processed using local memory and event-based operations rather than dense matrix calculations [6, 7, 8].

These properties have motivated the application of SNNs to event-based visual recognition and robotic perception, as well as their deployment on low-power embedded neuromorphic platforms [9, 10]. However, these potential advantages cannot be assumed to apply directly to aerodynamic data. Fluid-flow variables are continuous, signed, spatially distributed, and strongly dependent on their temporal evolution. Before a vorticity field can be processed by an SNN, it must be transformed into binary spike events. This conversion inevitably involves information loss. For this reason, the accuracy of the final reconstruction depends, among other factors, on whether the spike-encoding procedure is able to create an accurate representation of the continuous flow field.

This information-preservation problem provides the main motivation for the present thesis. The work does not attempt to develop a complete onboard sensing, prediction, and control system. Instead, it investigates a necessary intermediate step: whether a spike-based model architecture can extract sufficient load-relevant information from a high dimensional aerodynamic field to accurately reconstruct the sectional lift coefficient.

Extreme vortex-gust airfoil interactions provide a demanding test case for this question. Their corresponding flow fields present nonlinear behavior, while their lift histories may contain rapid variations and pronounced peaks. Nevertheless, previous studies indicate that the effective complexity may be much lower. Fukami and Taira [2] showed that vorticity fields from a broad range of extreme gust encounters could be compressed onto a three-dimensional latent manifold using a lift-augmented nonlinear autoencoder (see Fig. 1.1). Despite the severe compression, the representation retained the main vortical structures associated with the instantaneous lift coefficient. A later study at $Re = 5000$ reported a similar result for interactions that remained predominantly two-dimensional [11].

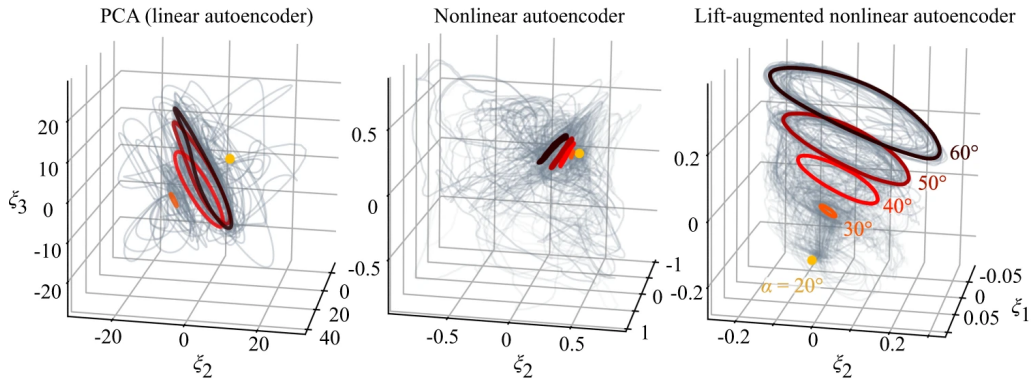


Figure 1.1. Three-dimensional latent representations of extreme vortex-gust flow data. From left to right: Principal Component Analysis (PCA), nonlinear convolutional autoencoder, and lift-augmented nonlinear convolutional autoencoder (LiftAug). Coloured trajectories correspond to undisturbed cases, while light-grey trajectories represent disturbed training encounters; LiftAug provides the clearest low-dimensional manifold. Figure extracted from [2].

These findings establish the scientific basis of the present study. They indicate that reconstructing the aerodynamic load may not require preserving every detail of the original flow field. Instead, the relevant task is to identify a compact representation containing the most relevant information needed for accurate C_l reconstruction. Conventional autoencoders have already shown that such load-relevant compression is possible for this aerodynamic problem.

The use of an SNN introduces an additional and largely unexplored difficulty. The network input is not the continuous flow field used by a conventional autoencoder, but a sparse binary representation obtained after spike encoding. Furthermore, the information is processed through neurons whose membrane potentials evolve over time, generating a

spike only when a firing threshold is crossed. Therefore, it is not evident that the low dimensional information identified using conventional neural networks will also hold after both input encoding and spike processing.

The research gap addressed by this thesis lies between two lines of investigation. The first, which is already well established, concerns dimensionality reduction of fluid flows. The second concerns the application of SNNs, which have shown promise in event-based computer vision, neuromorphic robotic control, and low-power embedded processing [9, 10, 12], but, to the best of the author's knowledge, remain comparatively underexplored in fluid mechanics. The present work connects these directions by examining whether a convolutional SNN-based architecture can recover load-relevant information from spike-encoded vorticity fields and whether doing so offers a meaningful computational advantage over conventional approaches.

This second aspect is important because the value of an SNN cannot be assessed from reconstruction accuracy alone. In practical embedded applications, the computational resources required to achieve that accuracy must also be considered. Conventional Convolutional Neural Networks (CNNs) [13] operate on dense continuous activations and rely predominantly on multiply-accumulate operations, whereas SNNs can exploit sparse spike activity and lower-cost spike-driven accumulation operations. The comparison between these processing paradigms therefore involves an inherent trade-off between reconstruction performance and estimated inference energy.

The motivation for using an SNN is therefore both scientific and engineering-oriented. From a scientific perspective, the thesis examines how much aerodynamic information survives the spike-encoding transformation and whether the remaining information can be organized into a compact representation suitable for lift reconstruction. From an engineering perspective, it evaluates whether sparse spiking computation provides a favourable balance between reconstruction accuracy and computational cost relative to dense CNN processing. Reaching an operational onboard system would still require physical sensors, short-term load prediction rather than instantaneous reconstruction, an architecture fully compatible with neuromorphic hardware, and integration with a control system. Previous work has shown that low-dimensional models of extreme vortex-gust interactions can support load-reduction strategies [14]. Before such a neuromorphic sensing and control system can be developed, however, it must first be established that spike-based inputs preserve sufficient information for reliable load reconstruction and that the resulting architecture offers a relevant efficiency benefit.

1.2. Objective and Research Questions

The main objective of this thesis is to evaluate whether a convolutional SNN-based architecture can accurately reconstruct the sectional lift coefficient C_l from time-resolved vorticity fields represented as spike trains. A complementary objective is to determine

whether the use of sparse spiking computation provides a favourable trade-off between reconstruction accuracy and estimated inference energy when compared with conventional CNN-based architectures.

The working hypothesis is that the aerodynamic load depends primarily on a limited subset of the information contained in the complete vorticity field. Although spike encoding inevitably discards part of the continuous input information, the resulting spike trains may still preserve the dominant vortical structures associated with the instantaneous lift response. Furthermore, the membrane states of the spiking neurons may retain information from preceding time steps, allowing the network to exploit temporal context that is not available from an isolated spike tensor. From a computational perspective, sparse spike propagation may also reduce the arithmetic cost of inference by replacing many dense multiply-accumulate operations with lower-cost spike-driven accumulation operations.

The investigation is organised around the following research questions:

1. Can a convolutional SNN-based model reconstruct the sectional lift coefficient C_l from spike-encoded vorticity fields for vortex-gust encounters that were not included during training?
2. How do the temporal resolution of the dataset and the selected spike-encoding representation affect the preservation of load-relevant information and the resulting reconstruction accuracy?
3. To what extent does preserving the neuronal membrane states across consecutive training windows improve the reconstruction of the lift history compared with a non-stateful training strategy?
4. What latent space dimension is required to compress the spike-encoded flow information without a significant loss of reconstruction accuracy?
5. How does the reconstruction performance of the reference SNN vary across the aerodynamic conditions represented in the test set, including angle of attack, gust intensity, and gust rotational direction?
6. How does the reference SNN compare with conventional CNN-based architectures in terms of reconstruction accuracy and estimated inference energy when the models are evaluated at approximately equal trainable capacity and at approximately equal energy budget?

To answer these questions, an end-to-end reconstruction pipeline is developed in which temporally resolved vorticity fields are pre-processed, converted into spike trains, and processed by a stateful convolutional SNN. The principal design choices are examined through ablation studies covering temporal interpolation, spike encoding, training strategy, and latent-space dimension. The selected reference configuration is then evaluated across the aerodynamic conditions contained in the test set.

The reconstruction and computational performance of the reference SNN are subsequently compared with two conventional CNN baselines. The parameter-matched CNN provides a comparison at approximately equal trainable capacity, while the energy-matched CNN evaluates both processing paradigms under approximately equal estimated inference energy budgets.

Together, these analyses determine whether spike-encoded aerodynamic fields retain sufficient information for reliable C_l reconstruction and whether the resulting spiking architecture offers a meaningful computational advantage relative to dense CNN processing. The results provide a basis for future investigations involving physical sensing, short-horizon load prediction, fully neuromorphic architectures, direct hardware measurements of energy consumption and inference latency, and eventual integration with active gust-load alleviation systems.

The remainder of the thesis develops the problem from this starting point. Chapter 2 reviews the aerodynamic and computational background required to position the work within the existing literature. Chapter 3 formulates the problem mathematically, while Ch. 4 describes the dataset used in the study. Chapter 5 presents the temporal interpolation and spike-encoding procedures. Chapter 6 describes the reference SNN architecture, the CNN-based comparison models, the training strategy, and the reconstruction and computational evaluation metrics. Chapter 7 reports and discusses the ablation studies, the performance of the reference model, and its comparison with the CNN architectures. Finally, Ch. 8 summarizes the conclusions, limitations and principal directions for future work.

2. BACKGROUND AND STATE OF THE ART

2.1. Extreme Vortex-Gust Interactions and Aerodynamic Load Response

A vortex-gust encounter takes place when a rotating region of air reaches a lifting surface. Unlike a uniform gust, which changes the airflow in roughly the same way across the whole surface, a vortex produces different velocity changes at different points. Its effect therefore depends on its strength, size, direction of rotation, and position relative to the airfoil. The angle of attack, Reynolds number and initial state of the surrounding flow also influence the response. Therefore, two vortices with similar maximum velocities can produce very different flow patterns and lift histories [1, 15, 16].

The strength of the gust is commonly described by the gust ratio

$$G = \frac{U_g}{U_\infty}, \quad (2.1)$$

where U_g is a characteristic gust velocity and U_∞ is the freestream velocity. When $|G| \ll 1$, the gust is weak compared to the incoming flow and usually causes only a small change in the aerodynamic state. As $|G|$ approaches one, the gust velocity becomes comparable to the freestream velocity and local airflow around the lifting surface may present rapid changes. For $|G| \gtrsim 1$, the interaction is often described as extreme aerodynamics because it can cause strong flow separation, the formation of new vortices, large wake changes, and severe variations in aerodynamic loading (see Fig. 2.1) [1, 2, 11].

For weak gusts and attached flow, the lift response can be approximated using classical linear theories. The Küssner model describes the response to a sharp-edged gust, while the Sears model describes the response to a harmonic gust [17, 18]. These theories remain useful as reference models. However, they assume that the disturbance is relatively small and that the flow remains attached to the airfoil. These assumptions are no longer valid when a strong gust produces large separation or strong vortices [1, 19].

More advanced models, such as the Leishman-Beddoes model, include nonlinear effects related to dynamic stall. These include delayed separation, the formation of vortices at the leading-edge and temporary load overshoots [20]. Even by considering these nonlinearities, such models are mainly designed for airfoils undergoing a prescribed motion or a known change in angle of attack. A vortex gust is more complex because the velocity disturbance changes across the airfoil and also evolves as it moves around it.

Experimental studies have confirmed that the lift response depends on more than gust strength. A study compared transverse and vortex gusts at low Reynolds number, showing that both types produced large changes in lift, but the response depended on the gust type, gust strength, and the relative position of the gust generator with respect to the airfoil

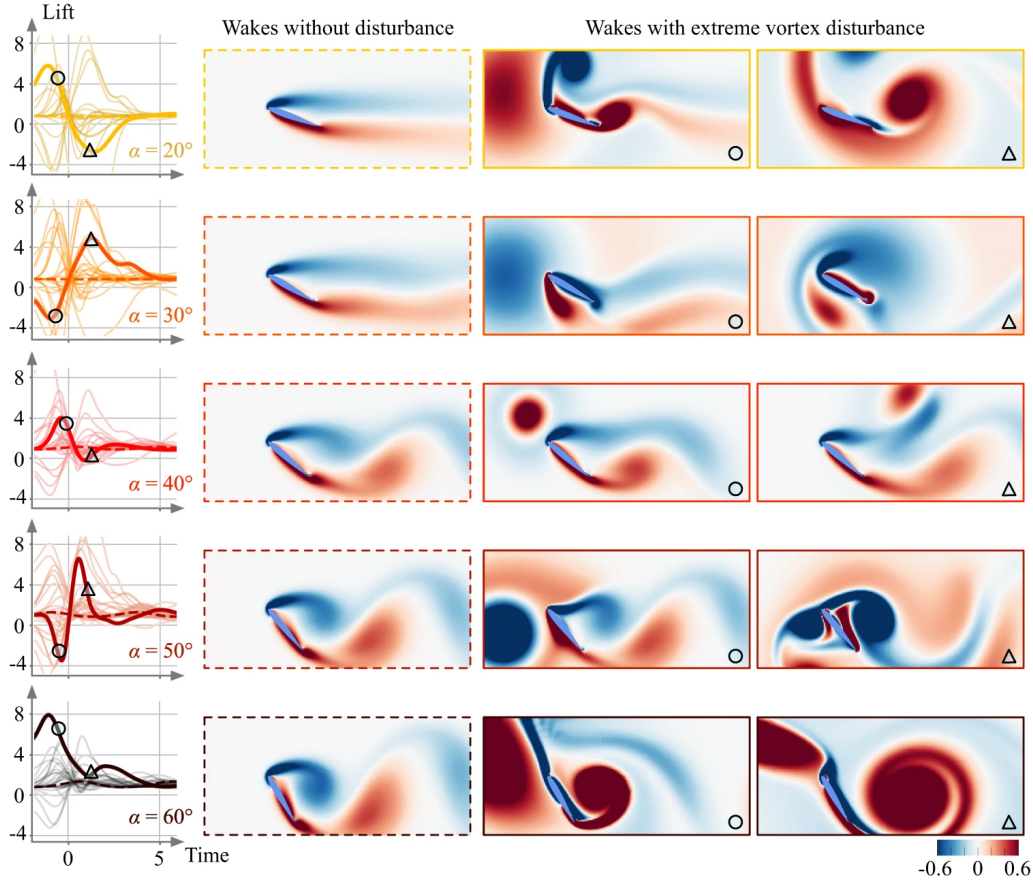


Figure 2.1. Visualizations of vorticity fields and corresponding lift responses for extreme aerodynamics. Figure extracted from [2].

[15]. Another study also showed that the positive lift peak caused by a vortex depends on its circulation and vertical position relative to the airfoil. The negative lift response that followed was more strongly correlated to the separated flow formed after the vortex had passed the wing [16].

These results show that the gust ratio alone is not enough to describe the aerodynamic response. In a recent study, Fukami et al. [11] examined numerically at $Re = 5000$ and at fixed angle of attack of 14° , the effects of gust ratio, gust size, vertical position, and rotational direction. Larger gusts reached the airfoil earlier and produced stronger lift variations. The position and direction of rotation also changed how the vortex interacted with the flow around the airfoil, sometimes causing positive and negative contributions to the lift to partly cancel. The principal conclusion was that the flow remained mainly two-dimensional for gust ratios up to approximately $|G| = 3$, while for stronger gusts smaller three-dimensional structures became more relevant.

2.2. Data-Driven Methods for Flow Reconstruction and Estimation

The increasing amount of available numerical and experimental flow data has led to a broad use of machine learning in fluid mechanics. Applications include turbulence modeling, dimensionality reduction, flow field reconstruction, temporal prediction, parameter estimation and control [3]. The appropriate model is selected based on the nature of the input data and on the desired output.

Multilayer Perceptrons (MLPs) can learn nonlinear mappings between a limited set of measurements and aerodynamic quantities [21]. CNNs are better suited for spatial data, such as velocity or vorticity fields, because they can identify local patterns across the domain. Recurrent Neural Networks (RNNs) are designed to process sequential data and are commonly applied when the future evolution must be inferred from a temporal history [22]. Generative Adversarial Networks (GANs), introduced by Goodfellow et al. [23], combine a generator and a discriminator and can be adapted to improve the reconstruction of spatial fields. Physics-Informed Neural Networks (PINNs), introduced by Raissi et al. [24], take a different approach by incorporating governing equations into the optimization problem, allowing physical constraints to be enforced during training.

Among these approaches, the concepts most relevant to the present work are convolutional feature extraction and temporal information processing. Both are retained, but within a spike-based computational framework. The network does not receive the vorticity field as a dense continuous input. It first operates on a binary sequence generated through spike encoding, while the membrane states of the spiking neurons carry information between successive time steps. The final readout remains continuous, since the target quantity is the instantaneous lift coefficient C_l . GANs and PINNs were not considered further because the objective is neither to generate complete flow fields nor to enforce the governing equations during training.

Previous flow-estimation studies show that neural networks can recover high dimensional flow quantities from incomplete measurements. Guastoni et al. [25] used CNNs to estimate wall-parallel velocity fields from wall pressure and wall shear stress measurements. Their models generally produced more accurate results than extended Proper Orthogonal Decomposition, especially when nonlinear relations between the measurements and the flow became important [26].

Similarly, a recent study [27] used a GAN to reconstruct turbulent velocity fields from low-resolution wall measurements. Although part of the spatial information had been removed from the input, the model was still able to recover the main flow features with useful accuracy. A follow up of this approach was then carried out for the reconstruction of complete three-dimensional velocity fields from wall measurements. Their results showed that flow features were reconstructed more accurately when they produced a clear effect on the measured quantities. Wall-attached motions were recovered more precisely than detached ones because they produced a stronger footprint at the wall [28].

These studies highlight a basic requirement of data-driven estimation, which is that the target quantity can only be recovered if the input contains enough correlated information. The same principle applies in this thesis. The vorticity field contains information about the incoming gust, the separated flow and the wake, all of which affect C_l . However, the network does not receive the continuous vorticity field directly, it first receives a spike-encoded version of it. The main question is therefore whether this binary representation still preserves enough load-related information for accurate reconstruction of C_l .

2.3. Low-Dimensional Representations and Load-Relevant Coordinates

A flow field may contain a substantial amount of redundant spatial and temporal information. Reduced-order methods use this fact to describe the main behavior of a flow with a much smaller number of coordinates. The objective is to reduce the amount of data while preserving the information needed for reconstruction, prediction or control.

Many reduced-order methods can be described using an encoder and a decoder. The encoder compresses the original flow field $\mathbf{q}$ into a smaller coordinate vector $\mathbf{z}$. The decoder then uses these coordinates to obtain an approximation $\hat{\mathbf{q}}$ of the original field:

$$\mathbf{z} = E(\mathbf{q}), \quad \hat{\mathbf{q}} = D(\mathbf{z}). \quad (2.2)$$

The encoder and decoder may be linear or nonlinear. A useful reduced representation should contain few coordinates while still preserving the information required for its intended task.

Proper Orthogonal Decomposition (POD), introduced in the context of turbulent flows by Lumley [29], is one of the most widely used linear methods for the dimensionality reduction of flow data. The encoder projects the input field onto an orthogonal basis, and the decoder reconstructs the field as a linear combination of the retained modes. The modes are ordered according to the amount of fluctuation energy they capture, and the field is approximated using only the most important ones. For a fixed number of modes, POD provides the best linear approximation in terms of the average squared reconstruction error [30].

The main limitation of POD is that it represents the data within a linear subspace. This may be inefficient when trying to reproduce low dimensional nonlinear evolution of the flow such as when a vortex moves across the domain, changes shape, or interacts with the airfoil in different ways. In these cases, many POD modes may be needed even when the main behavior of the system depends only on a few underlying coordinates. As POD orders its modes by energy, a flow feature with relatively low energy may therefore receive little importance even if it has a strong effect on the lift coefficient C_l .

Dynamic Mode Decomposition (DMD) [31] provides a different description of the flow. Rather than focusing mainly on captured energy, it identifies modes linked to character-

istic frequencies and rates of growth or decay. Related Koopman-based methods attempt to describe nonlinear systems through quantities whose evolution can be approximated using linear dynamics [32]. POD, DMD, and other modal methods provide different but complementary ways of analyzing flow data [33].

Although these methods are useful, a linear representation may become inefficient when the flow evolves nonlinearly over time or varies strongly across operating conditions. In such cases, many coordinates may be required to obtain an accurate reconstruction, while some nonlinear behavior may still not be captured adequately by a single linear subspace. Neural autoencoders provide an alternative by learning the dimensionality-reduction process directly from data. An autoencoder consists of an encoder and a decoder separated by a low-dimensional bottleneck. The encoder maps the original field q onto a smaller latent vector z , whereas the decoder uses this vector to produce a reconstruction $\hat{q}$. Since the same field is used as both the input and the training target, the network is optimized to make $\hat{q}$ as close as possible to q . Restricting the size of the bottleneck prevents the model from simply copying the input and forces z to retain the information required for reconstruction [34, 35].

A linear autoencoder trained with a squared reconstruction loss can recover the same optimal linear subspace as POD under suitable conditions, although the learned coordinates are not necessarily orthogonal or ordered by captured energy. The main advantage of a nonlinear autoencoder is that it is not restricted to this single linear subspace. It can therefore represent data distributed along a curved low-dimensional structure, commonly referred to as a manifold [36].

Convolutional autoencoders are well suited to flow fields because they process nearby spatial values together and use the same filters across the domain. This allows them to detect local flow patterns without requiring a different model parameter for every spatial position. Nonlinear convolutional autoencoders have been shown to compress cylinder-wake flow fields more accurately than POD when both methods use the same number of latent coordinates [35]. Hierarchical convolutional autoencoders have also been proposed to organise the learned latent coordinates and separate flow information across different levels of the representation [37].

However, accurate reconstruction of the complete flow field does not guarantee that the latent coordinates preserve the information needed to estimate a particular aerodynamic quantity. A model trained only to reduce the total vorticity reconstruction error may focus on large regions or strong vortices because they contribute more to that error. A smaller flow feature may receive less attention even if it has a large effect on C_l .

One way to address this problem is to include the target aerodynamic quantity in the training objective. A lift-augmented convolutional autoencoder was used to compress vorticity fields from extreme vortex-gust encounters into only three latent coordinates. The model preserved the main flow features related to lift, and the resulting low-dimensional representation organized both undisturbed and gust-affected cases [2].

A later study obtained a similar three-coordinate representation for vortex-gust encounters at $Re = 5000$ whose dynamics remained mainly two-dimensional. The autoencoder retained the main vortex cores associated with the lift response while removing part of the smaller-scale flow content [11]. This suggests that estimating aerodynamic loads may not require every spatial detail of the original flow field.

Low-dimensional representations have also been used for purposes beyond flow reconstruction. A reduced description of extreme vortex-gust interactions was used to design strategies for reducing transient lift variations [14]. This shows that a compact representation can support estimation and control when its coordinates preserve the main evolution of the aerodynamic system.

The present thesis follows the same general idea of learning a compact representation, however the proposed model does not reconstruct the complete vorticity field and therefore does not require a decoder. Instead, an encoder compresses the input into a latent representation, and a readout layer reconstructs the C_l directly. The model is trained according to the error between the reconstructed and reference lift coefficient. The latent representation is therefore judged by how well it supports load reconstruction, not by how accurately it reproduces the complete input field.

2.4. Sparse and Event-Based Sensing for Aerodynamic Estimation

Complete flow-field measurements are difficult to obtain in real aerodynamic systems. Particle Image Velocimetry (PIV) can provide detailed velocity-field measurements in laboratory experiments by tracking the displacement of seeded particles between successive images [38]. However, applying full-field PIV around an aircraft in operation is not practical, since it requires optical access, controlled particle seeding, specialized imaging equipment, and the acquisition and processing of very large amounts of data. A deployable aerodynamic estimation system must therefore rely on sparse or indirect measurements, such as surface pressure, structural strain, or information about an incoming gust.

The amount of data required depends on the main goal. Reconstructing the complete vorticity field may require many measurements, whereas reconstructing a single quantity, such as lift, may require much less. The sensors only need to capture flow changes that have a clear effect on the target quantity.

Previous studies of wall-bounded flows illustrate this idea. Velocity fields away from the wall have been reconstructed using only wall-pressure and wall-shear-stress measurements [25]. Other studies showed that the main flow features could still be estimated when the spatial resolution of the wall measurements was strongly reduced [27]. Three-dimensional reconstructions also showed that flow features were estimated more accurately when they produced a stronger effect on the available wall measurements [28].

A future gust-load sensing system could work in a similar way. Sensors placed upstream

of the airfoil could detect the position, strength, direction, and evolution of an incoming gust. Sensors placed on the airfoil could instead provide information about the current aerodynamic response. A trained model could then use these measurements to estimate the current lift coefficient or predict its future value.

The type and location of the sensors would depend on the final task. Instantaneous lift reconstruction could mainly rely on measurements of the current flow state. Short-term prediction would require information about the gust before it reaches the airfoil. A closed-loop control system would also require the estimate to be produced quickly enough for the controller to respond.

The present thesis does not implement this complete sensing system. It uses full-field vorticity data obtained from numerical simulations and reconstructs C_l at the same time instant. Sparse sensing, upstream measurements, and future-load prediction are left for future work. The first step is to determine whether a spike-based representation can preserve enough information when the complete numerical field is available.

Event-based sensing offers one way to reduce the amount of data that must be recorded and processed. A conventional camera records every pixel at fixed time intervals, even when most of the image has not changed. An event camera works differently. Each pixel produces a positive or negative event only when the local brightness changes by more than a defined threshold. Spike cameras use another event-based approach. Each pixel accumulates light intensity and emits a spike when a threshold is reached. The measured intensity can then be represented through the timing or frequency of the spikes (see Fig. 2.2) [39]. In both cases, the sensor focuses on changes or accumulated information instead of repeatedly transmitting a complete image.

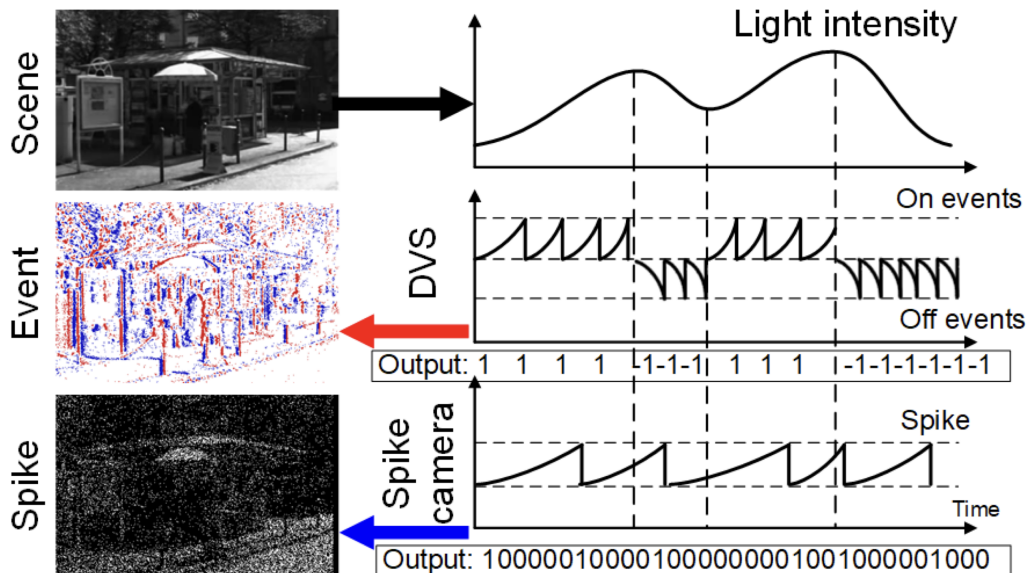


Figure 2.2. Reconstruction of a scene using an event and a spike camera. On the right, the sampling mechanism of each camera is visually represented. Figure extracted from [39].

These cameras measure light, not vorticity, and are therefore not direct aerodynamic sensors for the problem studied here. Their importance is in the way they process information. They show that a continuous physical signal can be converted into sparse events, reducing the need to process a dense measurement field at every time step. Similar sensing and processing systems have already been studied for autonomous vehicles and robotic applications [40].

The spike-encoding method used in this thesis follows the same general idea. The continuous vorticity field is converted into binary channels that represent its sign, approximate magnitude, and temporal changes. This reduces the precision of the original data and introduces information loss. At the same time, it produces an event-based input that may be processed more efficiently by a spiking architecture.

2.5. Principles of Spiking Neural Networks

SNNs are commonly described as the third generation of neural network models. Unlike conventional artificial neural networks, where neurons communicate through continuous-valued activations, SNNs transmit information through discrete events called spikes. These spikes occur at specific times and can therefore encode information not only through firing rates, but also through temporal patterns, latency, and synchronisation. This makes SNNs especially relevant for signals with an intrinsic temporal structure and for systems where sparse event-based computation is desirable [6, 7, 41].

A spike train emitted by neuron i can be represented mathematically as a sequence of event times t_i^k :

$$s_i(t) = \sum_k \delta(t - t_i^k), \quad (2.3)$$

where $\delta(t - t_i^k)$ denotes a Dirac delta located at the time of the k -th spike. This continuous-time notation represents the spike train as a sum of instantaneous events. In a discrete-time implementation, the output of neuron i at time step t is instead represented by the binary variable:

$$s_i[t] = \begin{cases} 1, & \text{spike} \\ 0, & \text{no spike.} \end{cases} \quad (2.4)$$

Figure 2.3 illustrates the difference between the continuous and discrete time representations. In the continuous formulation, each spike is located at its exact firing time and represented by a Dirac delta. In the discrete formulation used for numerical implementation, time is divided into fixed intervals and the neuron output is binary at each step; a value of one indicates that a spike occurred, while zero represents the absence of a spike. Therefore, the precise firing time is assigned to the corresponding discrete time bin.

This event-based binary representation is one of the mechanisms behind the computational efficiency of SNNs. Network updates can be driven by sparse spike events instead

of dense continuous activations being propagated through every layer at every time step [6, 42].

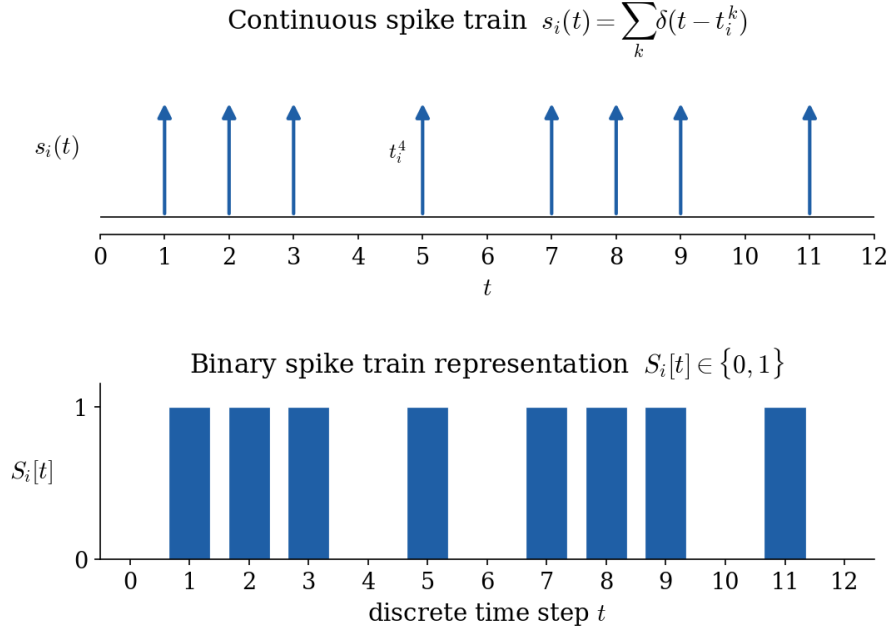


Figure 2.3. Representation of a spike train emitted by neuron i . (Top) Continuous-time formulation. (Bottom) Discrete-time formulation.

2.5.1. Leaky Integrate-and-Fire (LIF) Neuron Model

Several mathematical models can be used to represent a spiking neuron. Some, such as the Hodgkin-Huxley model, describe the electrical behavior of biological neurons in great detail as they model several ion currents and require multiple coupled equations to update the neuron state [43]. There exist simpler models, such as the Izhikevich and Adaptive Exponential Integrate-and-Fire models, that still reproduce a wide range of biological firing patterns [44, 45]. However, this level of biological detail comes at the practical cost of mathematical complexity and computational requirements. They require more state variables to be stored, more parameters to be selected and more operations to be performed for every time step.

In this thesis, the Leaky Integrate-and-Fire (LIF) neuron model is used because it keeps only the biological neuron properties needed for the particular task of interest. It accumulates incoming information (integrate), gradually loses part of its membrane potential simulating memory (leaky), emits a spike when a threshold is reached (fire), and then resets. The nice thing about this model is that these operations can be implemented with a simple mathematical formulation, making the model easier to train, less memory-hungry, and more suitable for larger networks and neuromorphic hardware [6, 7, 42].

A simple way to derive the LIF equation is to model the neuron membrane as a leaky capacitor (see Fig. 2.4). The membrane capacitance C stores charge, while the membrane

resistance R produces a leakage current that drives the membrane potential back toward its resting value U_{rest} . Applying current conservation gives:

$$C \frac{dU_i(t)}{dt} = -\frac{U_i(t) - U_{\text{rest}}}{R} + I_i(t), \quad (2.5)$$

where $U_i(t)$ is the membrane potential of neuron i and $I_i(t)$ is the input current. Multiplying by R and defining the membrane time constant $\tau = RC$ yields

$$\tau \frac{dU_i(t)}{dt} = -(U_i(t) - U_{\text{rest}}) + RI_i(t). \quad (2.6)$$

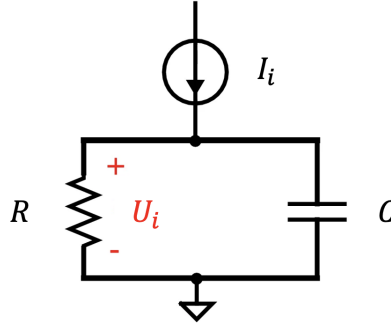


Figure 2.4. Leaky Integrate-and-Fire RC circuit model.

For many SNN implementations, the membrane potential is expressed relative to the resting potential, so U_{rest} can be taken as zero after a change of variable. The continuous-time dynamics then reduce to the following first-order ordinary differential equation:

$$\tau \frac{dU_i(t)}{dt} = -U_i(t) + RI_i(t). \quad (2.7)$$

One can now apply Euler method to discretise the continuous-time dynamics in Eq. 2.7. This gives the following update rule for the membrane potential¹:

$$U_i[t + 1] = U_i[t] + \frac{\Delta t}{\tau} (-U_i[t] + RI_i[t]). \quad (2.8)$$

Introducing the membrane decay factor

$$\beta = \left(1 - \frac{\Delta t}{\tau}\right), \quad (2.9)$$

the update can be rewritten as

$$U_i[t + 1] = \beta U_i[t] + (1 - \beta)RI_i[t]. \quad (2.10)$$

The problem with Eq. 2.10 is that there is no learnable parameter controlling how strongly the input current contributes to the membrane potential. It is desirable then to have a

¹The use of $[t]$ refers to discrete time, while (t) refers to the continuous time domain.

parameter multiplying the input current that can be optimized during training. Such parameter is the weight of the synaptic connection between the input and the neuron. Thus, expressing the input current term in vector form to account for all neurons, one can then redefine as

$$(1 - \beta)R\mathbf{i}[t] = \mathbf{W}\mathbf{x}[t], \quad (2.11)$$

where $\mathbf{W}$ is the synaptic weight matrix (the factor $(1 - \beta)R$ is absorbed by the weights) and $\mathbf{x}[t]$ is the input spike vector coming from the previous neuron layer at time step t . This gives the following update rule for the membrane potential (in vector form):

$$\mathbf{u}[t + 1] = \beta\mathbf{u}[t] + \mathbf{W}\mathbf{x}[t]. \quad (2.12)$$

Equation 2.12 captures two mechanisms of the LIF neuron:

- The term $\beta\mathbf{u}[t]$ represents the memory of the previous membrane potential, i.e. the *Leaky* property. The leakage factor β controls how much of the past information is retained, with higher β meaning longer memory and lower β meaning shorter memory.
- The term $\mathbf{W}\mathbf{x}[t]$ represents the integration of incoming synaptic current, i.e. the *Integrate* property, which contributes to the accumulation of membrane potential.

However, the neuron must have a way to propagate the information forward. This can be modeled by the spike generation mechanism, i.e. the *Fire* mechanism. The membrane potential is accumulated over time, and when it reaches a certain threshold U_{th} , the neuron emits a spike:

$$s_i[t] = \begin{cases} 1, & \text{if } U_i[t] \geq U_{\text{th}} \\ 0, & \text{otherwise.} \end{cases} \quad (2.13)$$

After the spike is emitted, the membrane potential needs a mechanism to reset its state. Without this reset, the potential could remain above the threshold and the neuron would continue firing at the following time steps even if no sufficient new input was received.

Different reset mechanisms can be used. A hard reset sets the membrane potential to a fixed value, while a soft reset subtracts the threshold from the current membrane potential. More complex models may also use adaptive reset rules. In this thesis, a subtractive reset is selected. This mechanism removes the amount of membrane potential associated with the emitted spike while preserving any value that remains above the threshold. The neuron therefore does not lose all the information accumulated before firing, which is useful when strong or closely spaced inputs are received [42, 46].

Now, combining the Leakage, Integration and Fire mechanisms gives the final model for the LIF neuron:

$$\mathbf{u}[t + 1] = \beta\mathbf{u}[t] + \mathbf{W}\mathbf{x}[t] - \mathbf{S}[t + 1]\mathbf{u}_{\text{th}}. \quad (2.14)$$

Note that $\mathbf{u}_{th}$ is a vector whose size corresponds to the number of neurons of the present LIF layer. Each layer contains its own threshold value. Moreover, $\mathbf{S}[t + 1]$ represents the diagonal matrix containing the spike output of each neuron in the present LIF layer.

Figure 2.5 visually represents the LIF neuron dynamics. One can see that the membrane potential $U_i[t]$ integrates the incoming synaptic current $w_i^t x_i[t]$ while decaying over time with a leakage factor β . When the membrane potential reaches the threshold U_{th} , the neuron emits a spike $s_i[t]$ and resets its membrane potential by subtracting the threshold.

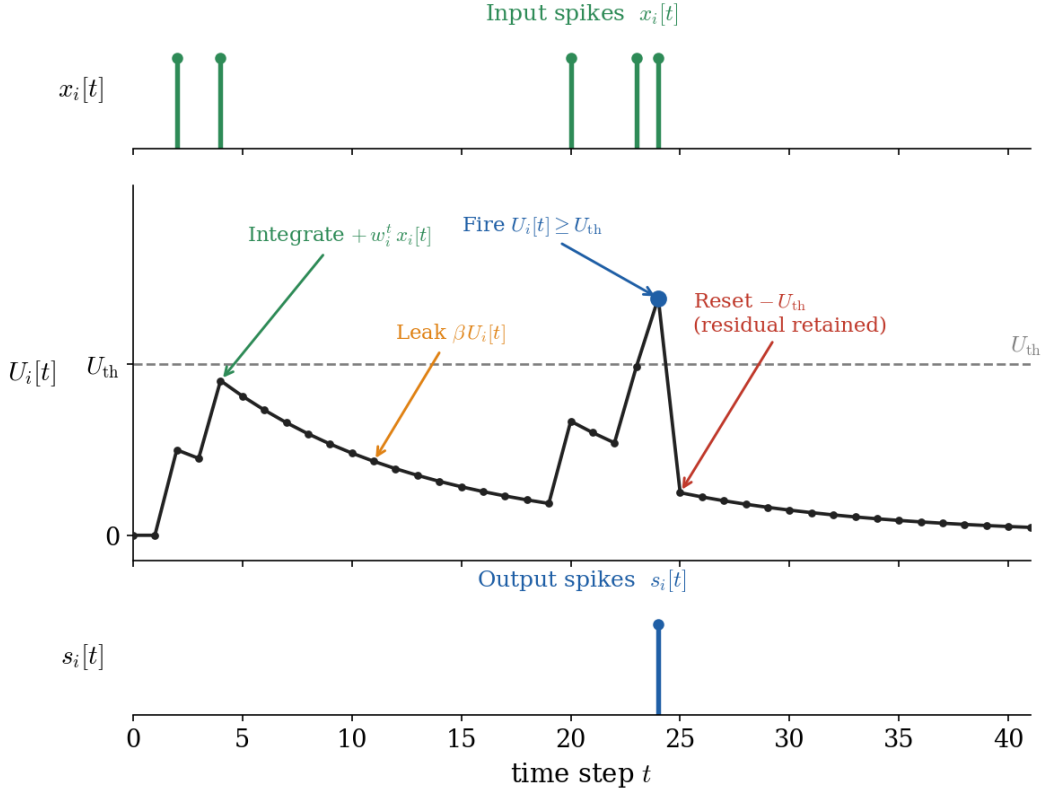


Figure 2.5. Leaky Integrate-and-Fire neuron model schematic of dynamics.

2.5.2. Spike Encoding Techniques

Spike encoding is the preprocessing step that converts a continuous-valued signal into a sequence of binary events that can be processed by spiking neurons [6, 7].

The choice of encoding strategy is important because it controls the balance between information preservation and spike sparsity. A dense spike representation may preserve the original input more accurately, but it reduces the computational advantage of event-driven processing. A very sparse representation may be more efficient, but it may also discard information that is relevant for the task. Therefore, spike encoding is not merely a preprocessing step, it defines the information made available to the SNN and, consequently, has a direct influence on the network's performance.

Some of the most common encoding strategies are the following:

- **Rate encoding.** In rate encoding, the magnitude of a continuous input value is represented by the number of spikes emitted over a time window. Given a normalized scalar input $\hat{x}[t]$, a spike at time t can be generated with probability

$$P(S[t] = 1) = p(\hat{x}[t]) \quad (2.15)$$

where $p(\hat{x}[t])$ is a bounded function such that $0 \leq p(\hat{x}[t]) \leq 1$. A larger input value produces a higher probability of spiking and therefore a higher average firing rate. Over a window of T time steps, the empirical firing rate is

$$r = \frac{1}{T} \sum_{t=1}^T S[t] \quad (2.16)$$

The main advantage of rate encoding is that it provides a simple and intuitive mapping between input magnitude and neural activity: larger values produce more spikes, while smaller values produce fewer spikes. This makes the encoded signal relatively robust, because the information is not carried by a single precise spike time but by the average activity over several time steps. The disadvantage follows from the same mechanism. To estimate the input magnitude reliably, the network needs to observe enough spikes over a sufficiently long window. Therefore, rate encoding can require many time steps or high firing activity, which increases latency and reduces the sparsity that makes SNNs computationally attractive.

Figure 2.6 illustrates rate encoding for three normalized input values, $\hat{x} = 0.15, 0.50$, and 0.85 . At each time step, a spike is generated independently with probability $\hat{x}$. Larger input values therefore produce denser spike trains. Over the $T = 40$ time steps shown, the empirical firing rate r is close to the corresponding input value, although small differences appear because the spikes are sampled randomly.

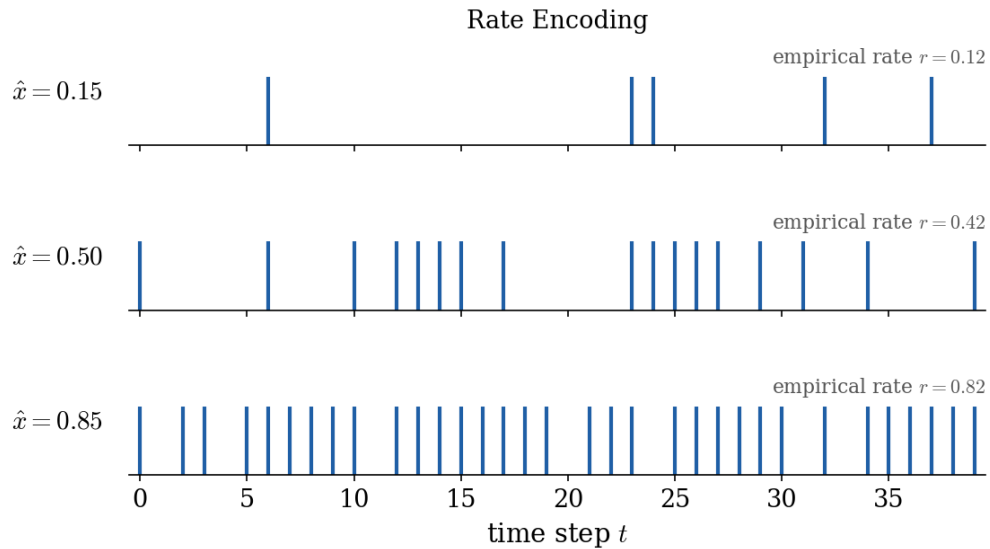


Figure 2.6. Rate encoding of normalized input values.

- **Delta encoding.** Delta encoding represents changes in the input signal rather than its absolute value. If $\hat{x}[t]$ is a continuous-valued signal sampled in time, the temporal increment can be written as

$$\Delta\hat{x}[t] = \hat{x}[t] - \hat{x}[t - 1] \quad (2.17)$$

A spike is generated when the magnitude of this change exceeds a prescribed threshold θ_Δ :

$$S_\Delta[t] = \begin{cases} 1, & \text{if } |\Delta\hat{x}[t]| \geq \theta_\Delta \\ 0, & \text{otherwise} \end{cases} \quad (2.18)$$

This encoding is especially useful for signals where temporal variation carries important information. Instead of repeatedly encoding values that remain nearly constant, delta encoding only emits spikes when the signal changes enough to be considered relevant. Its main advantage is therefore sparsity: slowly varying regions generate few or no spikes. Its main limitation is that information about the absolute level of the signal can be weakened or lost if delta encoding is used alone. The choice of threshold is also critical. A threshold that is too low produces many spikes and reduces sparsity, while a threshold that is too high may suppress relevant changes.

Figure 2.7 shows how delta encoding responds to changes in a continuous signal. The difference between consecutive values, $\Delta\hat{x}[t]$, is compared with the positive and negative thresholds, $+\theta_\Delta$ and $-\theta_\Delta$. Flat regions do not produce spikes because the signal does not change. Slow changes also remain below the threshold and are ignored. Spikes are generated only during rapid increases or decreases whose magnitude exceeds θ_Δ , so the encoded sequence mainly represents the moments when the signal changes strongly.

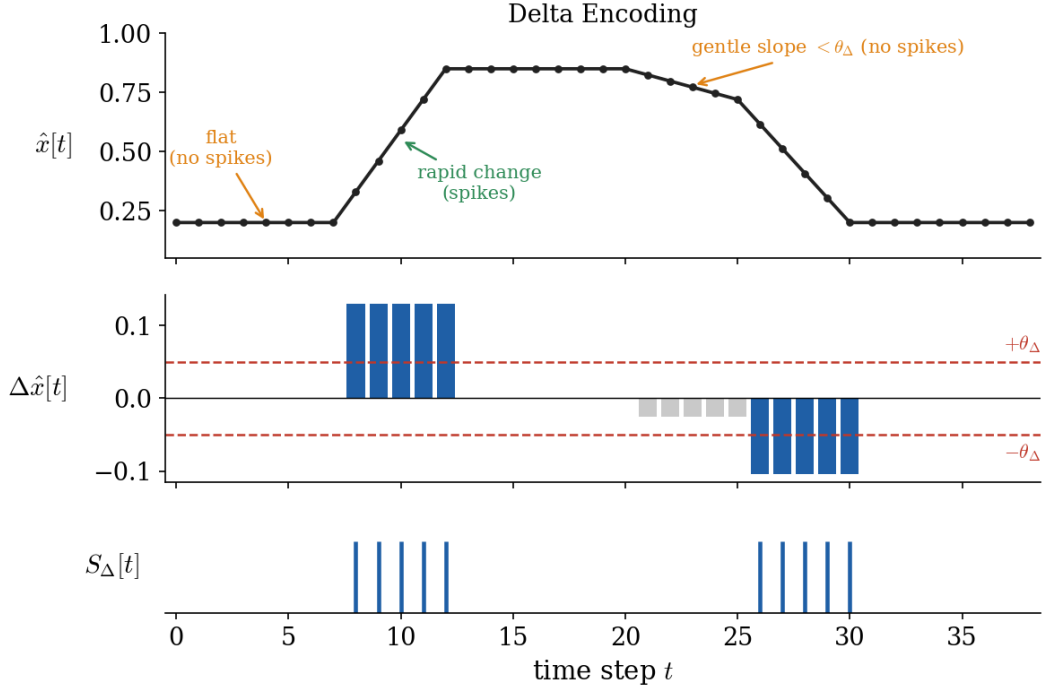


Figure 2.7. Delta encoding of a continuous signal.

- **Latency encoding.** In latency encoding, the value of a continuous input is represented by the time at which a spike is emitted. Larger input values are often assigned earlier spike times, while smaller values generate later spikes or no spike within the encoding window. A simple formulation is

$$t_{\text{spike}} = f(\hat{x}) \quad (2.19)$$

where $f(\cdot)$ maps input magnitude to spike time. The main advantage of latency encoding is that it can transmit information with very few spikes, sometimes only one spike per input unit. This can reduce the number of events and potentially lower latency. However, it is more sensitive to timing noise and requires the downstream network to interpret precise spike timing accurately. For this reason, it can be more difficult to train and tune than rate-based approaches [6, 46].

Figure 2.8 shows how latency encoding represents an input value through the timing of a single spike. Each neuron emits one spike within the encoding window. Larger input values produce earlier spikes, while smaller values produce later spikes. For example, the neuron with $\hat{x} = 0.90$ fires near the beginning of the window, whereas the neuron with $\hat{x} = 0.15$ fires much later. The input magnitude is therefore encoded by spike time rather than by the number of spikes.

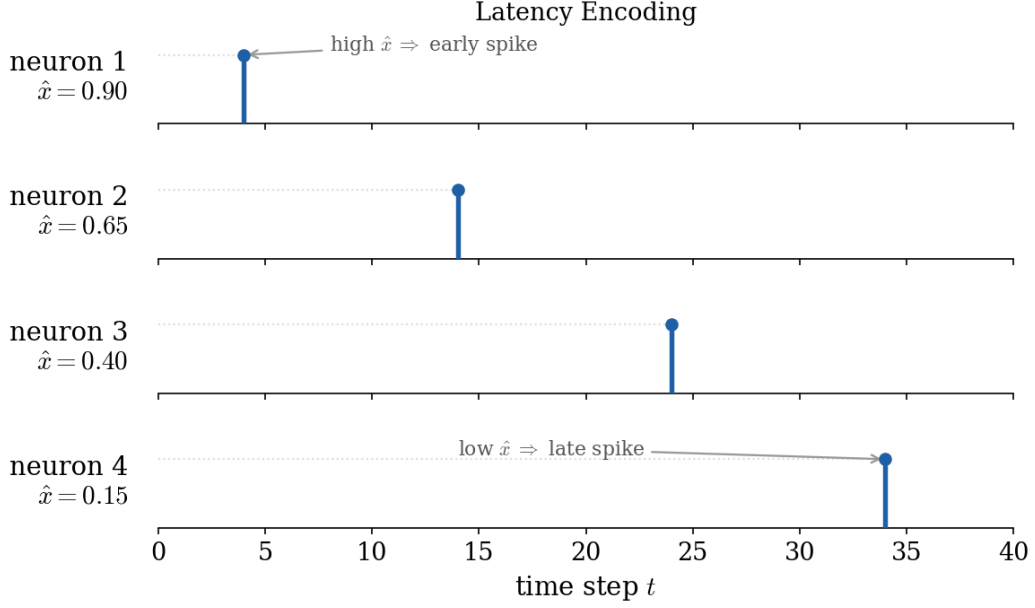


Figure 2.8. Latency encoding of four input values.

Many other encoding strategies exist, including population coding, phase coding, burst coding, time-to-first-spike variants, direct input encoding, and hybrid combinations of several methods [7, 46]. The present work focuses on the most common and practically useful families of spike encoding, with special emphasis on rate encoding and delta encoding.

2.5.3. Training Spiking Neural Networks

Training SNNs is more difficult than training conventional artificial neural networks for a variety of reasons, one of them being the non-differentiability of the spike-generation mechanism. In a conventional artificial neural network (ANN), each neuron applies a differentiable activation function to a weighted input, so the gradient of the loss with respect to the weights can be computed directly through backpropagation. In a SNN, the neuron emits a binary spike when its membrane potential crosses a threshold. This thresholding operation is usually written as

$$s_i[t] = \Theta(U_i[t] - U_{\text{th}}), \quad (2.20)$$

where $U_i[t]$ is the membrane potential of neuron i , U_{th} is the firing threshold, $s_i[t] \in \{0, 1\}$ is the spike output, and $\Theta(\cdot)$ is the Heaviside step function. The difficulty appears because $\Theta(\cdot)$ is discontinuous. Its derivative is zero almost everywhere and undefined at the threshold. In the ideal continuous formulation, the derivative of the Heaviside function is a Dirac delta distribution,

$$\frac{d\Theta(U)}{dU} = \delta(U), \quad (2.21)$$

which is not a regular function that can be used directly in standard gradient-based optimization. In practical discrete-time implementations, this means that

$$\frac{\partial s_i[t]}{\partial U_i[t]} = \begin{cases} 0, & \text{if } U_i[t] \neq U_{\text{th}}, \\ \infty, & \text{if } U_i[t] = U_{\text{th}}. \end{cases} \quad (2.22)$$

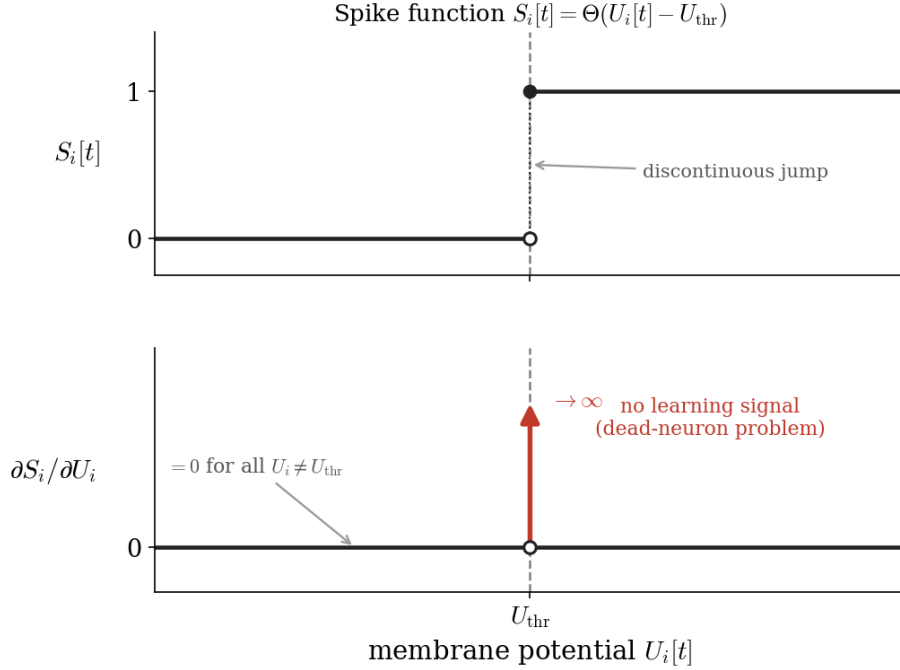


Figure 2.9. The Heaviside spike function (top) and its derivative with respect to $U_i[t]$ (bottom).

Figure 2.9 illustrates this difficulty. The upper panel shows the Heaviside spike function, whose output changes abruptly from zero to one when the membrane potential reaches the firing threshold. Its derivative is shown in the lower panel, it is zero away from the threshold and singular at the threshold crossing. This creates a problem during backpropagation (note that during the forward pass no problem is encountered). Consider a simple spiking neuron receiving an input vector $\mathbf{x}[t]$ through weights $\mathbf{W}$, so that the input current is

$$\mathbf{i}[t] = \mathbf{W}\mathbf{x}[t]. \quad (2.23)$$

The membrane potential depends on this current (Eq. 2.14), and the spike output depends on the membrane potential (Eq. 2.20). If the loss is denoted by $\mathcal{L}$, the gradient with respect to the weights contains the term

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}} = \frac{\partial \mathcal{L}}{\partial s_i[t]} \frac{\partial s_i[t]}{\partial U_i[t]} \frac{\partial U_i[t]}{\partial \mathbf{i}[t]} \frac{\partial \mathbf{i}[t]}{\partial \mathbf{W}}. \quad (2.24)$$

Considering each term:

- $\partial \mathcal{L} / \partial s_i[t]$ is the gradient of the loss with respect to the spike output, which is well defined and can be computed from the loss function.
- $\partial s_i[t] / \partial U_i[t]$ is the derivative of the spike output with respect to the membrane potential, which is problematic because of the non-differentiability of the spike function. This term is zero everywhere, except when $U_i[t] = U_{\text{th}}$, where it tends to infinity. Consequently, the gradient will always be null, or saturated if $U_i[t]$ is exactly equal to U_{th} , and so no learning can occur. This is known as the dead neuron problem.
- $\partial U_i[t] / \partial \mathbf{i}[t]$ is the sensitivity of the membrane potential to changes in the input current. In this case, given the present LIF dynamics, this term would have a value of 1, thus it is well defined.
- $\partial \mathbf{i}[t] / \partial \mathbf{W}$ is the gradient of the input current with respect to the weights, which simply equals $\mathbf{x}[t]$ since $\mathbf{i}[t]$ is a linear function of $\mathbf{W}$ and $\mathbf{x}[t]$.

A common way to mitigate this issue is surrogate-gradient learning. The idea is to keep the true binary spike function during the forward pass, so that the network still behaves as an SNN (Eq. 2.20), but to replace the derivative of the spike function during the backward pass by a smooth approximation, i.e. a surrogate function $\tilde{s}_i[t]$. Examples of such function include sigmoid-based, arctangent-based, triangular, rectangular, and fast-sigmoid surrogate derivatives. These functions assign a nonzero pseudo-gradient to membrane potentials near the firing threshold. In this way, neurons that are close to spiking can still receive a learning signal, even though the forward activation remains binary [46, 47]. Using an arctangent-based surrogate function, for example, the surrogate gradient can be defined as

$$\frac{\partial s_i[t]}{\partial U_i[t]} \approx \frac{\partial \tilde{s}_i[t]}{\partial U_i[t]} = \frac{1}{\pi} \frac{1}{1 + (\pi(U_i[t] - U_{\text{th}}))^2} \quad (2.25)$$

Figure 2.10 illustrates this process. In the forward pass, the neuron still uses the discontinuous binary spike function. In the backward pass, the arctangent-based surrogate derivative is used instead. Its value is largest at $U_i[t] = U_{\text{th}}$ and decreases as the membrane potential moves away from the threshold. The learning signal is therefore concentrated around the point where a small change in membrane potential could change whether the neuron fires.

Another important difference between SNNs and feedforward ANNs is that SNNs are dynamical systems. The membrane potential at time t depends not only on the current input, but also on the membrane state from previous time steps. Because $U_i[t + 1]$ depends on $U_i[t]$ (see Eq. 2.14), the network has an internal temporal state. Therefore, training requires gradients to be propagated not only across layers, but also across time. This is typically done using backpropagation through time (BPTT), where the recurrent computation is unrolled over a finite time window and the loss is differentiated through the unrolled graph.

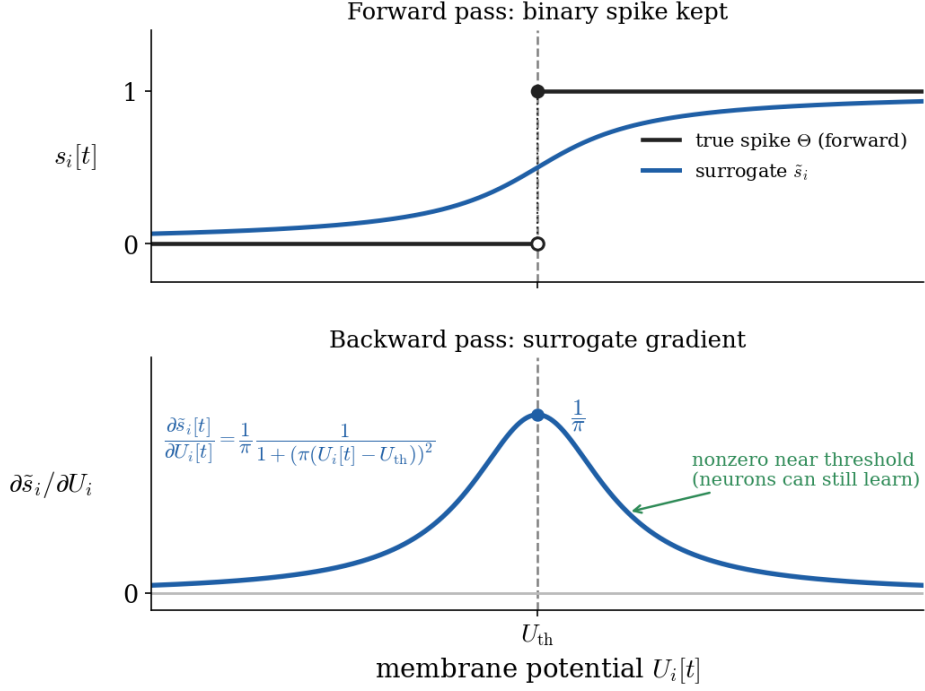


Figure 2.10. Surrogate gradient learning applied to the spike function.

If the total loss over a sequence of T time steps is

$$\mathcal{L} = \sum_{t=1}^T \mathcal{L}[t], \quad (2.26)$$

then the gradient with respect to a parameter θ can be written as

$$\frac{\partial \mathcal{L}}{\partial \theta} = \sum_{t=1}^T \frac{\partial \mathcal{L}[t]}{\partial \theta}. \quad (2.27)$$

Since the hidden state at each time depends recursively on previous states, each term contains temporal dependencies of the form

$$\frac{\partial \mathcal{L}[t]}{\partial \theta} = \sum_{\tau=1}^t \frac{\partial \mathcal{L}[t]}{\partial U[\tau]} \frac{\partial U[\tau]}{\partial \theta}. \quad (2.28)$$

Equivalently, the propagation of gradients across time involves products of temporal Jacobians,

$$\frac{\partial \mathcal{L}[t]}{\partial U[\tau]} = \frac{\partial \mathcal{L}[t]}{\partial U[t]} \prod_{k=\tau}^{t-1} \frac{\partial U[k+1]}{\partial U[k]}, \quad \tau < t. \quad (2.29)$$

This expression shows that training SNNs involves the same temporal-gradient difficulties found in recurrent neural networks. During BPTT, gradients are propagated backwards

through a sequence of membrane states. If the repeated temporal factors are consistently smaller than one, the gradient is gradually damped, and the earliest states receive only a weak learning signal. If these factors are consistently larger than one, the gradient can grow rapidly, leading to unstable parameter updates. In SNNs, this behavior is affected by several neuron and training parameters, including the leakage factor β , the firing threshold, the reset rule, the spike activity level, and the chosen surrogate derivative.

This process is illustrated in Fig. 2.11, where $|J|$ represents the magnitude of the temporal Jacobian applied repeatedly during backward propagation. When $|J| < 1$, successive multiplication reduces the gradient magnitude, so the learning signal reaching the earliest states becomes progressively weaker. This corresponds to the vanishing-gradient problem. Conversely, when $|J| > 1$, the gradient grows rapidly as it is propagated backwards, which may produce unstable parameter updates, i.e. the exploding-gradient problem. The case $|J| = 1$ represents the idealised situation in which the gradient magnitude remains unchanged across time.

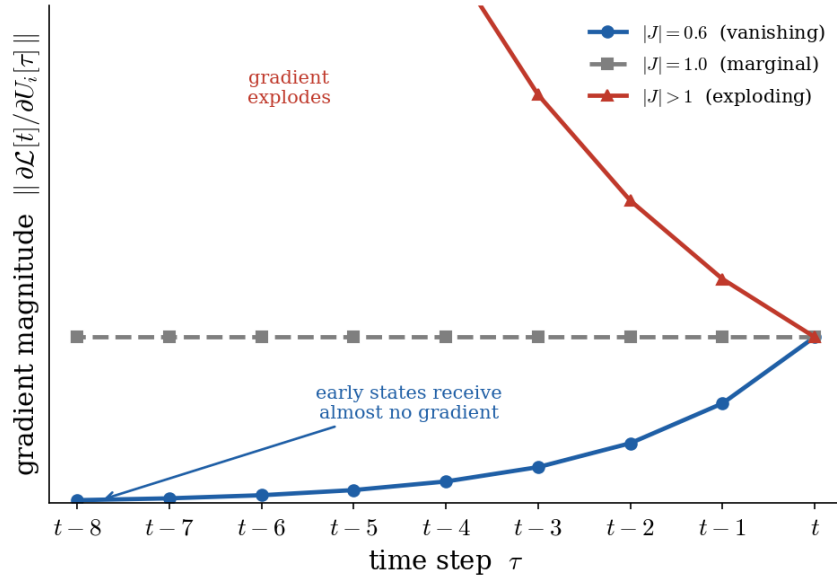


Figure 2.11. Effect of the temporal Jacobian magnitude on gradient propagation during BPTT.

BPTT also has a significant computational cost. To compute exact gradients through time, the training algorithm must store the membrane states, spike outputs, and intermediate activations for all layers and all time steps in the unrolled sequence. This increases memory usage substantially compared with a standard feedforward network. For long sequences, large spatial inputs, or deep convolutional SNNs, the required GPU memory can become a limiting factor. In practice, this often motivates truncated backpropagation through time (TBPTT), where the sequence is processed in shorter temporal windows and gradients are propagated only through the computational graph of the current window [48]. This reduces the memory and computational requirements of training, while the recurrent state may still be transferred between consecutive windows.

Therefore, training SNNs involves a trade-off. Surrogate gradients make end-to-end supervised learning possible despite the non-differentiability of spikes, and BPTT allows temporal dependencies to be learned through the membrane dynamics. However, this comes at the cost of approximate gradients, sensitivity to surrogate and neuron hyperparameters, possible vanishing or exploding gradients, and high memory requirements for long time sequences. These limitations are central when applying SNNs to time-dependent physical data, where the network must preserve information over time while remaining trainable and computationally feasible.

Other training methods have also been proposed. Spike-Timing-Dependent Plasticity (STDP) updates a synaptic weight according to the relative timing of the spikes emitted by the pre- and postsynaptic neurons [42]. Because the update depends on local neural activity, STDP is biologically motivated and can be suitable for online learning on neuromorphic hardware [8, 49]. However, it is more difficult to apply to deep supervised networks because the local updates do not directly minimise the error of the complete model [6, 7]. STDP has therefore been used mainly for unsupervised feature learning and pattern-recognition tasks, as illustrated by the digit-recognition network of Diehl and Cook [50]. For the continuous regression problem considered in this thesis, direct gradient-based training is more appropriate because it allows the lift reconstruction loss to be optimized end to end.

Another option is artificial neural network-to-spiking neural network (ANN-to-SNN) conversion. In this approach, a conventional ANN is first trained using standard backpropagation. Its weights are then transferred to a spiking network, and the firing thresholds are adjusted so that the spike rates approximate the original ANN activations [51]. This method can preserve much of the accuracy of the original model, but the approximation is obtained over a finite simulation window. Conversion errors and inference latency may therefore increase when many time steps are required, while some ANN components must be replaced or reformulated before deployment on neuromorphic hardware [41, 46]. Instead, the present thesis trains the SNN directly using surrogate gradients and TBPTT. This allows the membrane dynamics and the lift reconstruction objective to be optimized together, rather than inherited from a separately trained ANN.

3. PROBLEM FORMULATION

Chapter 2 provides the basis for the problem considered here. On the one hand, previous work has shown that the load-relevant information contained in an extreme vortex-gust interaction can be represented in a low-dimensional form, despite originating from a high-dimensional vorticity field. On the other hand, SNNs offer an event-driven way of processing temporal information, as discussed in Sec. 2.5. The purpose of this chapter is to bring these two ideas together by formulating the reconstruction of the instantaneous lift coefficient from a spike-encoded vorticity field.

3.1. Physical Statement of the Reconstruction Problem

The physical setting is the two-dimensional vortex-gust encounter described in Sec. 2.1: a coherent vortical disturbance is convected toward an airfoil and drives a transient, vortex-dominated load response. The aerodynamic quantity of interest is the instantaneous lift coefficient, $C_l(t)$. As argued in Ch. 2, $C_l(t)$ is a compact but physically rich observable that integrates the effect of the incoming vortex, the separated shear layers, and the wake into a single load history. The premise of this work is that the information determining this load is carried by the evolving vorticity field. The problem is therefore to recover the load from the flow at the corresponding physical time,

$$\text{vorticity field at time } t \longrightarrow C_l(t). \quad (3.1)$$

It is important to state that this is a reconstruction problem, not a forecasting one. The estimate at time t uses the flow information available at that same instant, together with information retained in the network state from earlier time steps. It does not predict the load from upstream measurements taken before the gust reaches the airfoil. That forecasting task is more demanding and is left for future work in Ch. 8. The present thesis addresses the necessary intermediate question of whether same-time flow information survives spike encoding with sufficient fidelity to recover $C_l(t)$.

3.2. Input Representation: the Non-Dimensional Vorticity Field

The observable used by the model is the non-dimensional vorticity field $\omega(x, y, t)$. Vorticity is chosen because it directly exposes the coherent rotational structures that govern the transient load response. This choice is consistent with the reduced-order evidence discussed in Sec. 2.3. After spatial discretization on the simulation grid, a single snapshot is represented as

$$\mathbf{\Omega}_t \in \mathbb{R}^{N_y \times N_x}, \quad (3.2)$$

where N_y and N_x denote the spatial dimensions of the discretised field. Each simulation case provides a time-ordered sequence of snapshots,

$$\{\mathbf{\Omega}_t\}_{t=1}^T, \quad (3.3)$$

together with the corresponding lift history. The grid resolution, number of cases, field of view, and physical parametrisation of the simulations are described in Ch. 4. The temporal interpolation and spike-encoding operations applied before training are described separately in Ch. 5.

3.3. Supervised Reconstruction Problem

The target variable is the scalar lift coefficient, represented as a time series aligned with the available flow snapshots:

$$C_l(t) \longrightarrow \{C_l^1, C_l^2, \dots, C_l^T\}. \quad (3.4)$$

The choice of C_l as the sole target follows the observable-aware perspective introduced in Sec. 2.3. Rather than reconstructing the complete flow field, the model is trained to recover the aerodynamic load generated by that flow. Fukami and Taira adopted a related approach by including the instantaneous lift coefficient in the training objective of their lift-augmented autoencoder, thereby encouraging the latent representation to retain load-relevant information [2].

The supervised learning problem considered here is therefore the reconstruction of C_l from the flow information available to the model. At snapshot index t , the reference value is denoted by C_l^t and the corresponding model prediction by $\hat{C}_l^t$. The trainable parameters are collected in θ and include the synaptic weights and biases of the encoder, together with the parameters of the final readout.

Because the model is an SNN with LIF dynamics, the estimate at time t does not depend only on the current vorticity snapshot. The network also maintains an internal state, denoted by $\mathbf{h}_t$, which contains the variables required to continue the temporal evolution of the spiking layers. This state is mainly determined by the membrane potentials and spike activity of the neurons and therefore retains a decaying trace of previous inputs.

The reconstruction can be expressed as

$$\hat{C}_l^t = f_\theta(\mathbf{\Omega}_t, \mathbf{h}_{t-1}), \quad (3.5)$$

where f_θ denotes the complete model, $\mathbf{\Omega}_t \in \mathbb{R}^{N_y \times N_x}$ is the discretised vorticity field at snapshot t , and $\mathbf{h}_{t-1}$ is the internal state before that snapshot is processed. This formulation does not imply that the network has direct access to the complete sequence $\mathbf{\Omega}_{1:t}$. Earlier flow information can affect the prediction only if it remains encoded in $\mathbf{h}_{t-1}$.

As the sequence is processed, the state evolves recursively according to

$$\mathbf{h}_t = \Phi_\theta(\mathbf{h}_{t-1}, \mathbf{\Omega}_t), \quad (3.6)$$

where Φ_θ represents the update rule induced by the spiking layers. The effective temporal memory is therefore determined by the learned parameters and by the LIF dynamics, including the decay factor β , the reset mechanism, the firing activity, and the temporal structure of the input.

This temporal formulation is suitable for the aerodynamic problem considered here. Although the instantaneous lift is strongly related to the current flow configuration, it is also influenced by the recent evolution of the vortical structures around the airfoil. Vortex formation, convection, flow separation, and wake development all introduce temporal dependence into the load response. The internal SNN state provides a mechanism through which this recent history can contribute to the reconstruction.

The model is also required to form a compact intermediate description of the aerodynamic state. This latent representation is denoted by $\mathbf{z}_t$. Motivated by the low-dimensional manifold evidence reviewed in Ch. 2, the reconstruction is separated into an encoding step and a readout step. First, the encoder maps the current vorticity field and the retained temporal state to a latent vector:

$$\mathbf{z}_t = E_\theta(\mathbf{\Omega}_t, \mathbf{h}_{t-1}), \quad \mathbf{z}_t \in \mathbb{R}^{N_z}. \quad (3.7)$$

Here, N_z is the latent-space dimension. Since each vorticity snapshot contains $N_y N_x$ spatial degrees of freedom, the condition

$$N_z \ll N_y N_x \quad (3.8)$$

indicates that $\mathbf{z}_t$ is a compact representation of the original field.

The readout then converts this latent vector into the reconstructed lift coefficient:

$$\hat{C}_l^t = g_\theta(\mathbf{z}_t). \quad (3.9)$$

This decomposition makes it possible to examine whether the learned latent representation behaves as a load-relevant compression of the aerodynamic state. The specific implementations of the encoder E_θ and the readout g_θ are presented in Ch. 6.

Training consists of adjusting θ so that the reconstructed lift histories match the corresponding reference histories over the training encounters. The optimization problem is written as

$$\theta^* = \arg \min_{\theta} \mathcal{L}(\hat{C}_l, C_l), \quad (3.10)$$

where $\mathcal{L}$ measures the discrepancy between the reconstructed and reference lift histories, and θ^* denotes the resulting optimized parameter set. The loss function and training procedure are defined in Ch. 6.

3.4. Spike-Based Reformulation

The formulation above was written in terms of the continuous vorticity field in order to define the reconstruction problem at the physical level. In the problem solved in this

thesis, however, the SNN does not receive the continuous snapshot $\mathbf{\Omega}_t$ directly. Before being passed to the network, the vorticity data are transformed into binary spike events by an encoding procedure.

Before training begins, all pre-processed vorticity sequences are converted into their corresponding binary spike representations using the encoding procedure described in Ch. 5. For one simulation case, this transformation is written as

$$\mathbf{S} = \mathcal{E}(\mathbf{\Omega}) = \{\mathbf{S}_1, \mathbf{S}_2, \dots, \mathbf{S}_T\}, \quad (3.11)$$

where $\mathbf{\Omega}$ is the sequence of continuous vorticity fields and $\mathbf{S}_t$ is the spike tensor associated with snapshot t . Once this preprocessing step has been completed, the SNN is trained using the spike-encoded dataset rather than the original continuous fields.

At each time step, the network receives the current spike tensor together with the internal state retained from the preceding steps. The reconstruction and updated state can be written together as

$$(\hat{C}_l^t, \mathbf{h}_t) = F_\theta(\mathbf{S}_t, \mathbf{h}_{t-1}), \quad (3.12)$$

where F_θ represents the complete SNN, $\hat{C}_l^t$ is the reconstructed lift coefficient, and $\mathbf{h}_t$ contains the updated neuronal states.

This is the central formulation of the thesis. The reconstruction at snapshot index t depends on the current spike-encoded representation of the flow and on the information from previous steps that remains stored in the internal state of the network. Because LIF neurons are leaky, this retained information is progressively attenuated rather than preserved indefinitely.

Since $\mathcal{E}$ is a lossy transformation, the spike representation generally discards part of the information contained in the original vorticity field. The scientific question is therefore whether the encoded sequence retains enough load-relevant information to reconstruct $C_l(t)$ with useful accuracy. The design of $\mathcal{E}$ determines what information is made available to the SNN. This includes the encoding schemes considered, the treatment of positive and negative vorticity, the use of event or change-based channels, the choice of thresholds, and the temporal mapping between flow snapshots and SNN processing steps. These details are discussed in Ch. 5.

4. DATASET

Chapter 3 formulated the reconstruction problem as a mapping from a vorticity field to the sectional lift coefficient, $\mathbf{\Omega}_t \rightarrow C_l^t$. This chapter specifies the data used to validate that mapping: their origin, the physical configuration of the simulations, the parameters that distinguish one case from another, and the variables used later by the model.

4.1. Source Dataset and Physical Configuration

The flow data used in this thesis are taken from the publicly available dataset of extreme vortex-gust airfoil interactions introduced by Fukami and Taira [2]². The data were obtained from two-dimensional direct numerical simulations of incompressible flow around a NACA 0012 airfoil at a chord-based Reynolds number $Re = U_\infty c / \nu = 100$, where U_∞ is the freestream velocity, c is the chord length, and ν is the kinematic viscosity. The airfoil is placed with its leading edge at the origin, at an angle of attack

$$\alpha \in \{20^\circ, 30^\circ, 40^\circ, 50^\circ, 60^\circ\}. \quad (4.1)$$

For each angle of attack, the dataset contains one undisturbed baseline simulation and 30 disturbed vortex-gust cases, for a total of $5 \times 31 = 155$ simulations: 150 gust encounters and 5 undisturbed references. In the baseline cases, the flow is steady at the lowest incidence and develops periodic vortex shedding at higher incidence, so the undisturbed wakes already span a range of unsteady behavior before any gust is introduced. In the disturbed cases, the gust is represented by a coherent Taylor vortex introduced upstream of the airfoil at $x_0/c = -2$ and convected towards it by the freestream. Its interaction with the airfoil produces a transient response involving the incoming disturbance, the separated shear layers, and the wake.

The physical configuration is summarized in Fig. 4.1, which shows the NACA 0012 airfoil, the freestream direction, the coordinate system, and the initial upstream position of the Taylor vortex.

Each simulation case provides a time-resolved vorticity field $\omega(x, y, t)$ and the corresponding sectional lift-coefficient history $C_l(t)$, with

$$C_l = \frac{F_l}{\frac{1}{2}\rho U_\infty^2 c}, \quad t^* \equiv \frac{U_\infty t}{c}, \quad (4.2)$$

where F_l is the sectional lift force, ρ is the fluid density, and t^* is the convective time. These two quantities constitute the input-target pairs required by the supervised reconstruction problem defined in Ch. 3: the spike-encoded vorticity field is the flow observable available to the model, while the lift coefficient is the aerodynamic quantity to be reconstructed.

²Annex D provides a direct link to the original dataset.

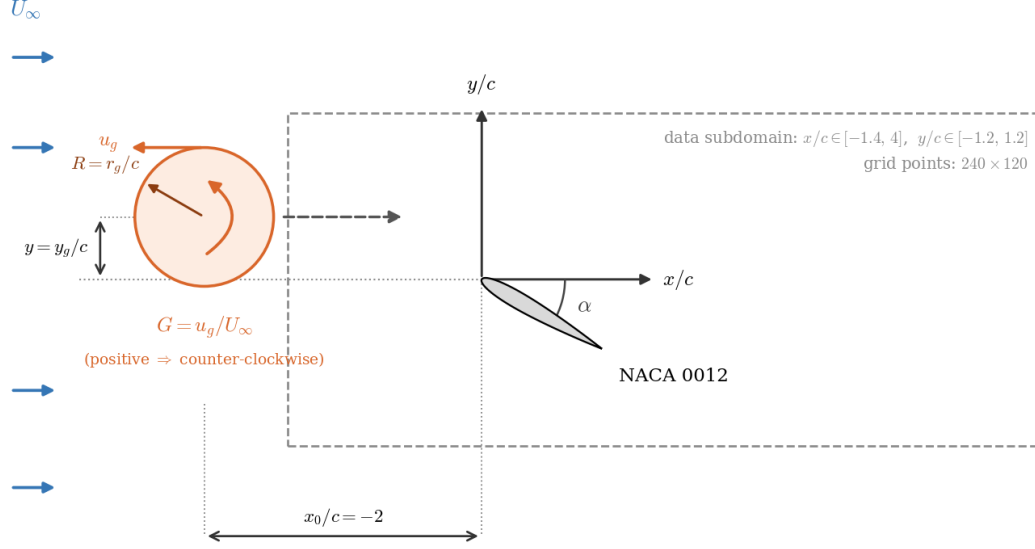


Figure 4.1. Schematic of the vortex-gust airfoil simulation configuration.

4.2. Parametric Space

Each disturbed case is defined by the airfoil angle of attack and the properties of the incoming vortex. Thus, it is specified by the parameter combination

$$(\alpha, G, R, y), \quad (4.3)$$

where

$$G = \frac{u_g}{U_\infty}, \quad R = \frac{r_g}{c}, \quad y = \frac{y_g}{c}. \quad (4.4)$$

Here, G is the gust ratio, whose sign indicates the rotational direction of the vortex. The parameter R is the vortex-core radius normalized by the airfoil chord and therefore represents the size of the disturbance relative to the airfoil. Finally, y is the vertical position of the vortex centre normalized by the chord, which determines whether the vortex initially passes above, through, or below the airfoil. Positive and negative gust ratios interact differently with the airfoil and its separated wake, meaning that the sign of G is a physically relevant parameter rather than a simple convention. Each disturbed simulation in the dataset is identified by its corresponding (G, R, y) triplet.

The 30 disturbed cases available at each angle of attack do not form a complete factorial grid over (G, R, y) . Instead, every case corresponds to one sampled combination of the three parameters. The sampled values of vortex size and vertical position are common to all angles of attack:

$$R \in \{0.25, 0.50, 0.75, 1.00\}, \quad y \in \{-0.5, -0.3, -0.1, 0, 0.1, 0.3, 0.5\}. \quad (4.5)$$

The range of sampled gust ratios varies slightly with the angle of attack, as summarized in Table 4.1. In every case, the sampled values reach $|G| \gtrsim 3$, well beyond the small-disturbance regime. Consequently, all five angles of attack include encounters within the

extreme-aerodynamics regime discussed in Ch. 2. Although only 30 disturbed simulations are available for each angle of attack, they cover several sources of physical variation: the baseline separated-flow state through α , the gust strength and rotational direction through G , the disturbance scale through R , and the encounter trajectory through y . Therefore, the resulting parameter space is large relative to the number of available simulations.

α	Sampled G range
20°	[−3.2, 3.8]
30°	[−4.0, 3.8]
40°	[−3.8, 4.0]
50°	[−4.0, 3.4]
60°	[−3.6, 3.8]

Table 4.1. Range of sampled gust ratios G for the 30 disturbed cases at each angle of attack.

Two representative encounters are presented in Figs. 4.2 and 4.3. In both figures, the upper row shows the normalized continuous vorticity field at three stages of the interaction, while the corresponding instants are marked on the lift coefficient history in the lower panel.

Figure 4.2 corresponds to a strong positive gust initially located below the chord line and interacting with the airfoil at $\alpha = 20^\circ$. As the vortex approaches and passes the airfoil, it substantially modifies the surrounding flow and produces a pronounced transient variation in C_l . Figure 4.3 presents a negative gust initially located above the chord line at $\alpha = 60^\circ$. In this case, the incoming vortex interacts with an already separated and unsteady baseline flow, producing a different evolution of the vortical structures and of the lift response. Together, the two examples illustrate how variations in angle of attack, gust sign, and vertical position can produce markedly different aerodynamic interactions within the same dataset.

$$\alpha = 20^\circ, \quad G = +3.80, \quad R = 1.00, \quad y = -0.50$$

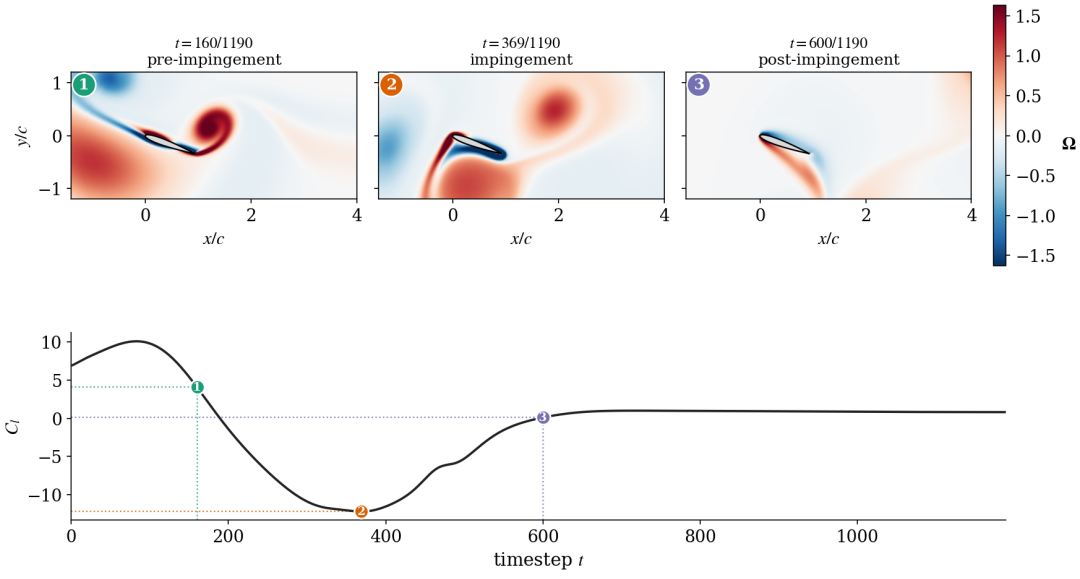


Figure 4.2. Evolution of the normalized vorticity field and lift coefficient for $(\alpha, G, R, y) = (20^\circ, +3.8, 1.0, -0.5)$. The upper row shows three vorticity snapshots, while the markers in the lower panel indicate their positions on the C_l history.

$$\alpha = 60^\circ, \quad G = -3.00, \quad R = 1.00, \quad y = +0.50$$

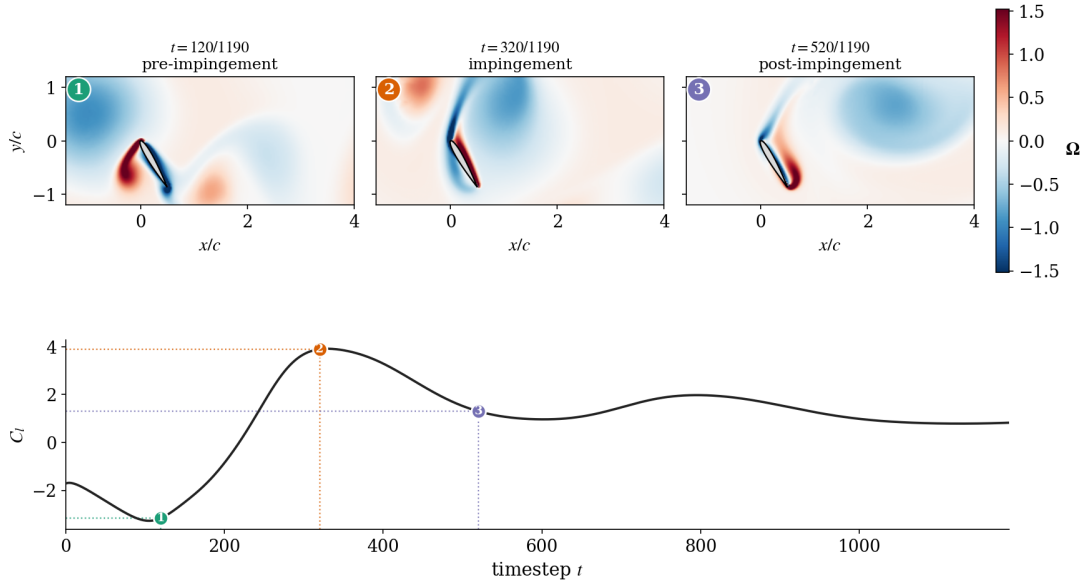


Figure 4.3. Evolution of the normalized vorticity field and lift coefficient for $(\alpha, G, R, y) = (60^\circ, -3.0, 1.0, +0.5)$. The upper row shows three vorticity snapshots, while the markers in the lower panel indicate their positions on the C_l history.

4.3. Available Flow and Load Variables

The vorticity fields are provided on a uniform Cartesian grid covering the subdomain $(x, y)/c \in [-1.4, 4] \times [-1.2, 1.2]$, where the vortex-airfoil interaction primarily takes place. At snapshot index t , the vorticity field is represented as a two-dimensional array

$$\mathbf{\Omega}_t \in \mathbb{R}^{N_y \times N_x}, \quad (N_y, N_x) = (120, 240), \quad (4.6)$$

where N_y and N_x are the numbers of grid points in the vertical and streamwise directions, respectively. The corresponding aerodynamic target is the scalar sectional lift coefficient at the same snapshot index, denoted by C_l^t .

Then, each simulation case consists of a time-ordered sequence of paired vorticity fields and lift coefficients:

$$\{\mathbf{\Omega}_t, C_l^t\}_{t=1}^T, \quad T = 1190. \quad (4.7)$$

In practice, each case is stored as a vorticity array of size $N_y \times N_x \times T$, together with a lift coefficient time series aligned with the same snapshot indices. The sequence covers the approach of the vortex before impingement, the interaction with the airfoil, and the subsequent relaxation of the wake.

The source simulations generated 1200 snapshots over 10.2 non-dimensional convective time units, corresponding to a sampling interval of

$$\Delta t^* = \frac{10.2}{1200} = 8.5 \times 10^{-3} \quad (4.8)$$

between consecutive snapshots. The publicly distributed arrays contain a slightly shorter portion of each simulation, consisting of $T = 1190$ consecutive snapshots sampled at the same interval. Each stored case therefore spans approximately 10.1 convective time units. This small difference in sequence length does not alter the temporal resolution of the data.

Using the definition of convective time given in Eq. 4.2, the equivalent dimensional sampling period is

$$\Delta t = \frac{c \Delta t^*}{U_\infty}, \quad (4.9)$$

which depends on the airfoil chord length and the free stream velocity.

5. SPIKE ENCODING AND DATA PRE-PROCESSING

The previous chapters formulated the aerodynamic load reconstruction problem as the mapping from a time-resolved flow sequence to the corresponding lift coefficient history. In particular, Ch. 3 introduced the spike-based formulation through an encoding operator $\mathcal{E}$, which transforms the continuous vorticity sequence into a sequence of spike tensors. This chapter defines that transformation in detail.

5.1. From Vorticity Fields to Spike-Based Network Inputs

Let the aerodynamic flow input be represented by a sequence of two-dimensional vorticity fields,

$$\mathbf{\Omega}_{1:T} = \{\mathbf{\Omega}_1, \mathbf{\Omega}_2, \dots, \mathbf{\Omega}_T\}, \quad (5.1)$$

where each snapshot is defined on the spatial grid introduced in Ch. 4,

$$\mathbf{\Omega}_t \in \mathbb{R}^{N_y \times N_x}, \quad (N_y, N_x) = (120, 240). \quad (5.2)$$

Here, $\mathbf{\Omega}_t(x, y)$ denotes the vorticity value at spatial position (x, y) and physical time index t . The goal of the encoding pipeline is to transform this continuous-valued sequence into a binary spike sequence,

$$\mathbf{S}_{1:T'} = \mathcal{E}(\mathbf{\Omega}_{1:T}), \quad (5.3)$$

where $\mathbf{S}_t$ is the spike tensor supplied to the spiking neural network at SNN simulation step t . The length of the encoded sequence is denoted by T' . In the absence of temporal interpolation, $T' = T$, and each physical vorticity snapshot corresponds to one SNN input step. When temporal interpolation is applied, intermediate vorticity fields (as well as intermediate lift coefficient values) are generated between the original snapshots, so that $T' > T$.

At each SNN time step, the network receives one spike tensor,

$$\mathbf{S}_t \in \{0, 1\}^{C \times N_y \times N_x}, \quad (5.4)$$

where C is the number of spike channels. The value of C depends on the chosen encoding configuration. For example, a rate-only input uses two channels corresponding to positive and negative vorticity polarity, while a hybrid rate-delta input uses four channels.

The complete pre-processing and encoding procedure is summarized in Fig. 5.1. The original vorticity sequence and the corresponding lift history first pass through the optional temporal-interpolation stage, which increases the temporal resolution while preserving the alignment between the input fields and target values (Sec. 5.2). The interpolated vorticity fields are then normalized and clipped to limit the influence of extreme values (Sec. 5.3). Finally, the continuous fields are converted into binary spike channels using the

selected encoding method (Sec. 5.4). The resulting spike tensor at each time step remains paired with the lift coefficient at the same instant, allowing the network prediction to be compared directly with the corresponding aerodynamic target during supervised training.

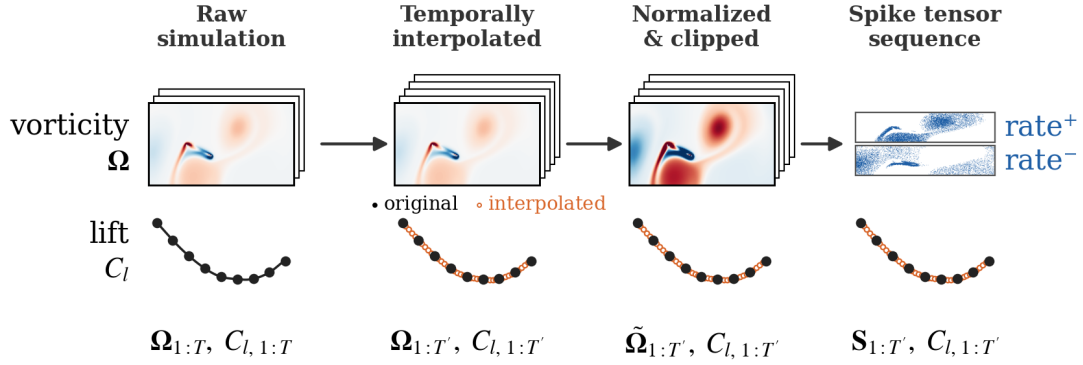


Figure 5.1. Overview of the pre-processing and spike encoding pipeline.

5.2. Dataset Temporal Interpolation

The original aerodynamic dataset contains vorticity fields and lift coefficients sampled at uniformly spaced time instants. Before robust normalization and spike encoding, the temporal grid can be refined through interpolation. The purpose of this step is not to increase the physical duration of the simulations, but to provide a finer representation of the flow evolution between the available snapshots.

This offline conversion is related to the concept of virtual eventization described by Ianiro et al. [52]. In virtual eventization, conventionally sampled data are converted into events after acquisition by applying a chosen event-generation rule. The present thesis follows the same general approach, once the simulations have been completed, the stored vorticity fields are processed and converted into spike sequences. Because the event times are then restricted by the temporal resolution of the stored snapshots, the sequence is first interpolated onto a finer temporal grid.

In an ideal case, the vorticity field would be available as a continuous function of time, and spikes would be generated at the instants where the encoded flow variable crosses the corresponding rate or delta-encoding condition. In practice, the flow is only available at discrete time indices. This introduces a temporal quantization effect: if an event is detected at time step $t = 5$, it is not possible to know from the sampled sequence whether the corresponding change would have occurred exactly at $t = 5$ or at an intermediate instant, such as $t = 4.6$, in a continuous-time representation. The spike can only be assigned to one of the available discrete input steps.

This limitation affects both rate and delta encoding. When the time interval between consecutive snapshots is large relative to the evolution of the flow, relevant changes can only be represented at a small number of discrete instants. In rate encoding, intermediate

vorticity magnitudes are omitted, so the spike probabilities may change too abruptly from one step to the next. In delta encoding, gradual variations may instead be concentrated into a single frame-to-frame difference. In both cases, the resulting spike sequence provides a coarser representation of the temporal evolution. Since the SNN processes these spikes sequentially and carries information through its membrane states, errors in the timing or distribution of spikes can affect the subsequent network response.

Temporal interpolation is used to reduce this discretization effect by inserting intermediate vorticity fields between the original snapshots,

$$\mathbf{\Omega}_{1:T} \longrightarrow \mathbf{\Omega}_{1:T'}. \quad (5.5)$$

When interpolation is used, $T' > T$, and the SNN receives a longer sequence with smaller temporal changes between consecutive input steps. This gives the spike encoder a finer temporal grid on which to place rate and delta events. In this sense, interpolation does not recover the exact continuous-time flow, nor does it add new simulated physics, but it provides a smoother approximation of the temporal evolution and reduces the dependence of spike timing on the original sampling interval.

This motivation is closely related to the operation of neuromorphic vision sensors. In real event-based or spike-based cameras, the very high temporal sampling rate allows events to be generated with much finer timing resolution than conventional frame-based data. For instance, spike cameras reported in neuromorphic vision literature can operate at sampling rates on the order of 20,000-40,000 Hz [39]. The present dataset is not acquired from such a sensor; it is a sequence of simulated flow snapshots. Temporal interpolation is therefore used as a preprocessing step to partially compensate for the limited temporal resolution before generating spike trains from the vorticity field.

Let the original vorticity sequence be $\mathbf{\Omega}_{1:T}$. Temporal interpolation is performed using a cubic spline interpolation technique. Therefore, after applying this procedure, the sequence becomes

$$\mathbf{\Omega}_{1:T'} = \{\mathbf{\Omega}_1, \mathbf{\Omega}_2, \dots, \mathbf{\Omega}_{T'}\}, \quad (5.6)$$

where the new sequence contains both original and intermediate fields. In the implementation considered in this thesis, interpolation is applied in time at each spatial grid location, producing a temporally refined vorticity sequence on the same spatial grid (N_y, N_x).

The lift sequence must be aligned with the interpolated vorticity sequence. If the vorticity input is increased from T to T' time steps, the target lift coefficient sequence must also be represented at T' time steps,

$$C_{l,1:T} \longrightarrow C_{l,1:T'}. \quad (5.7)$$

This alignment is required because the supervised problem compares the network prediction at each SNN step with the corresponding target value.

5.3. Robust Normalization and Clipping

The raw vorticity fields contain continuous values with a broad dynamic range, including localized regions of very high magnitude around vortical structures. Before converting these fields into spikes, the vorticity values are normalized to a bounded interval. This normalization is required for both rate and delta encoding.

In rate encoding, the normalized vorticity magnitude is used to define a spike-generation probability; therefore, the input must be scaled to a range where probabilities can be meaningfully computed and compared across cases. Without normalization, a small number of extreme vorticity values could dominate the mapping and force many spike probabilities into saturation, reducing the ability of the encoder to distinguish between moderate and strong structures. In delta encoding, normalization is equally important because temporal events are generated from differences between consecutive fields. If the absolute scale of the vorticity were not controlled, the same threshold could correspond to very different physical levels of temporal variation across cases.

A simple option would be to normalize each vorticity field by its maximum absolute value. However, this would make the scaling highly sensitive to the largest local vorticity magnitudes. In the present dataset, these high values are not merely isolated numerical outliers. They are a physically relevant and recurring feature of extreme vortex-gust interactions, particularly near strong vortex cores, separated shear layers, and the airfoil leading edge. Nevertheless, they occupy only a small fraction of the complete spatiotemporal dataset. If the global maximum were used as the normalization scale, these localized peaks would determine the full range and compress most of the remaining vorticity values into a narrow interval.

For this reason, a percentile-based normalization is applied. Its purpose is not to remove the extreme aerodynamic structures, but to prevent the largest local values from dominating the spike-generation process while preserving the relative magnitude of most of the field.

Let $P_q(|\mathbf{\Omega}|)$ denote the q -th percentile of the absolute vorticity magnitude. In this thesis, this percentile is computed as a single scaling value for the vorticity fields processed by the encoding pipeline. It is not recomputed separately for every time snapshot. This is important because recomputing the scale at each time step would artificially normalize every snapshot to a similar range, even when the true vorticity intensity changes over time. Using a single scaling value therefore preserves the relative variation between snapshots while still reducing the effect of extreme vorticity values. The percentile scale was computed once using the complete set of simulation cases and was then kept fixed when normalizing both the training and test cases.

The normalized and clipped vorticity field is defined as

$$\tilde{\mathbf{\Omega}}_t(x, y) = \text{clip} \left(\frac{\mathbf{\Omega}_t(x, y)}{P_q(|\mathbf{\Omega}|)}, -1, 1 \right). \quad (5.8)$$

The clipping operation enforces

$$\tilde{\Omega}_t(x, y) \in [-1, 1]. \quad (5.9)$$

This bounded representation preserves the sign and spatial location of the strongest vorticity regions, although values above the selected percentile lose their exact relative amplitudes because they are mapped to either -1 or 1 . Then, clipping introduces a controlled loss of information in the upper tail of the distribution, but prevents a small fraction of very large values from reducing the sensitivity of the encoder to the rest of the flow field.

The percentile q determines the balance between preserving high-magnitude vorticity and maintaining sufficient contrast across the full field. This is illustrated in Fig. 5.2, which compares the maximum absolute value with the P_{95} and P_{99} scales. Normalization by the maximum is dominated by the strongest localized vorticity values and maps most of the field close to zero. P_{95} is more aggressive and clips approximately 5% of the values, whereas P_{99} clips only approximately 1%. Since high vorticity is a central feature of the extreme aerodynamic conditions considered here, P_{99} provides a more conservative compromise between limiting the influence of the highest peaks while leaving the great majority of the physically relevant amplitude variation unsaturated.

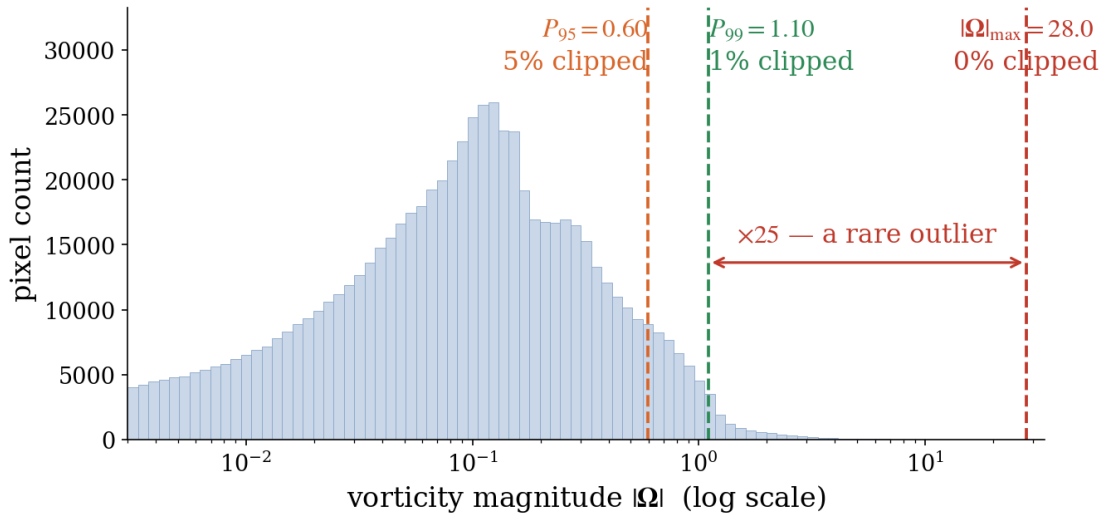


Figure 5.2. Distribution of the absolute vorticity magnitude $|\Omega|$ with the candidate normalization scales P_{95} , P_{99} , and $|\Omega|_{\max}$.

The effect of these choices on an actual vorticity snapshot is shown in Fig. 5.3. When the field is normalized by $|\Omega|_{\max}$, the main vortical structures become difficult to distinguish because the scale is determined by the largest localized vorticity value. With P_{95} , the structures are visible, but a larger fraction of the field is saturated, as indicated by the gold regions clipped to ± 1 . The P_{99} normalization retains the leading-edge vortex, shear layer, and wake structures while clipping only a small fraction of pixels. This makes it a suitable scaling choice for the spike encoder as the input is bounded, robust to large values, and still retains enough contrast for both rate and delta spike generation.

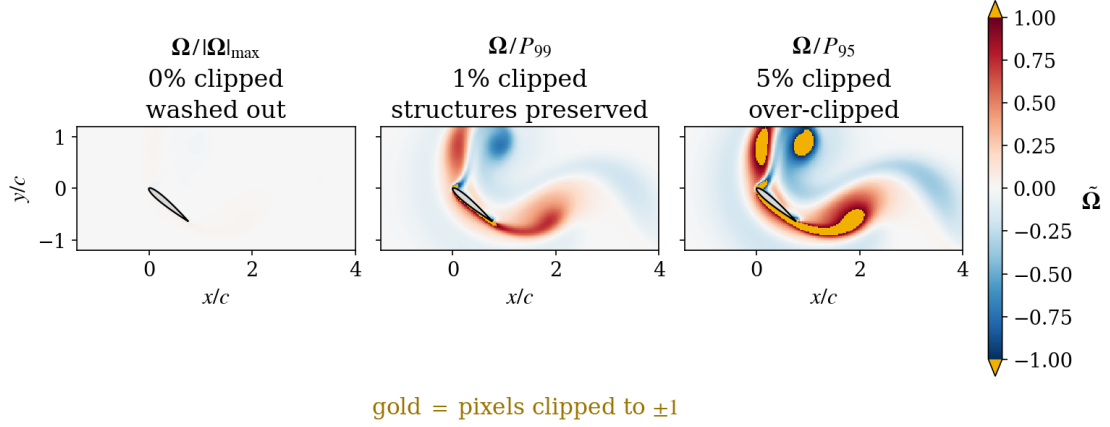


Figure 5.3. Normalized vorticity field obtained using $|\Omega|_{\max}$, P_{99} , and P_{95} as scaling values. Gold marks pixels clipped to ± 1 .

5.4. Spike Encoding

The normalized vorticity fields must be converted into binary spike tensors. Two different techniques are studied in this thesis: rate encoding and delta encoding. Rate encoding represents the local magnitude and sign of the vorticity through spike activity, while delta encoding responds to temporal changes between consecutive fields. The following subsections describe both methods separately, including the polarity channels and parameters used to generate the final spike representation.

5.4.1. Rate Encoding

Rate encoding was introduced in Sec. 2.5.2 as a general method for converting continuous values into spike trains. In the present problem, the method must be specialized to signed, two-dimensional vorticity fields. The objective is to generate spike events whose probability depends on the local magnitude of the normalized vorticity, while preserving the sign of the vortical structure through separate polarity channels.

A direct rate encoder could be obtained by setting the spike probability equal to the normalized vorticity magnitude, $p_t(x, y) = |\tilde{\Omega}_t(x, y)|$. This corresponds to a purely linear mapping between vorticity magnitude and firing probability. However, such a definition would make the spike activity almost entirely determined by the distribution of the normalized vorticity field. In other words, it would provide little control over the firing rate of the encoded representation. For this reason, two additional parameters are introduced: the nonlinear exponent γ and the gain factor g . These parameters allow the encoder to modify the density of spikes without changing the underlying normalized vorticity field.

Rate Encoding Formulation for Signed Vorticity Fields

For each normalized vorticity value $\tilde{\Omega}_t(x, y)$, the spike probability is defined as

$$p_t(x, y) = \min\left(1, g|\tilde{\Omega}_t(x, y)|^\gamma\right), \quad (5.10)$$

where g is a gain factor and γ is a nonlinear exponent. Since $\tilde{\Omega}_t(x, y) \in [-1, 1]$, the absolute value $|\tilde{\Omega}_t(x, y)|$ provides a normalized measure of local vorticity magnitude. The function is clipped at one to ensure that $p_t(x, y)$ remains a valid probability.

The case $g = 1$ and $\gamma = 1$ recovers the direct linear mapping $p_t(x, y) = |\tilde{\Omega}_t(x, y)|$. Therefore, the use of g and γ should be understood as a controlled generalization of the simplest rate encoder. The exponent γ reshapes the sensitivity of the encoder to weak and strong vorticity values, while the gain g scales the overall probability level before saturation. Together, they provide a direct way to regulate the firing rate of the spike representation.

Because the vorticity field is signed, positive and negative vortical structures are not encoded in the same channel. Instead, rate spikes are separated into positive and negative polarity channels,

$$S_t^{r,+}(x, y), \quad S_t^{r,-}(x, y). \quad (5.11)$$

The positive rate channel can generate spikes only where the normalized vorticity is positive, while the negative rate channel can generate spikes only where the normalized vorticity is negative.

This polarity split is important because positive and negative vorticity correspond to opposite rotational directions. Collapsing both signs into a single magnitude channel would remove this information and would make physically distinct vortical structures indistinguishable to the network.

The density of the resulting spike representation is characterized using the following firing-rate quantities:

- **Positive spike firing rate:** fraction of positive rate spikes over the full spatio-temporal sequence,

$$\rho^{r,+} = \frac{1}{T'N_xN_y} \sum_{t=1}^{T'} \sum_{x=1}^{N_x} \sum_{y=1}^{N_y} S_t^{r,+}(x, y). \quad (5.12)$$

- **Negative spike firing rate:** fraction of negative rate spikes over the full spatio-temporal sequence,

$$\rho^{r,-} = \frac{1}{T'N_xN_y} \sum_{t=1}^{T'} \sum_{x=1}^{N_x} \sum_{y=1}^{N_y} S_t^{r,-}(x, y). \quad (5.13)$$

- **Channel mean firing rate:** average firing rate across the two polarity channels,

$$\rho_{\text{mean}}^r = \frac{\rho^{r,+} + \rho^{r,-}}{2}. \quad (5.14)$$

- **Combined firing rate:** total fraction of active spatial locations across the positive and negative rate channels,

$$\rho_{\text{comb}}^r = \rho^{r,+} + \rho^{r,-}. \quad (5.15)$$

The combined firing rate is especially useful because the positive and negative rate channels are mutually exclusive at each spatial location: a pixel can belong to the positive or negative polarity channel depending on the sign of the vorticity, but not both. Therefore, ρ_{comb}^r directly indicates how much of the spatio-temporal vorticity field is active after rate encoding. These quantities describe the density of the representation, not the reconstruction performance.

Effect of the Gamma Exponent

The exponent γ controls the nonlinear relationship between normalized vorticity magnitude and spike probability. To isolate its effect, consider the simplified transfer function

$$p = |\tilde{\Omega}|^\gamma, \quad (5.16)$$

where the gain is fixed to $g = 1$. This mapping is shown in Fig. 5.4. The dashed line corresponds to the linear case $\gamma = 1$, for which the spike probability is equal to the normalized vorticity magnitude. When $\gamma < 1$, the curve lies above the diagonal for $0 < |\tilde{\Omega}| < 1$, so weak and moderate vorticity values are assigned higher spike probabilities than in the linear case. This increases the density of the spike representation and makes weaker vortical structures more visible to the encoder. When $\gamma > 1$, the curve lies below the diagonal, so spike generation becomes more concentrated in regions of strong vorticity and the representation becomes sparser.

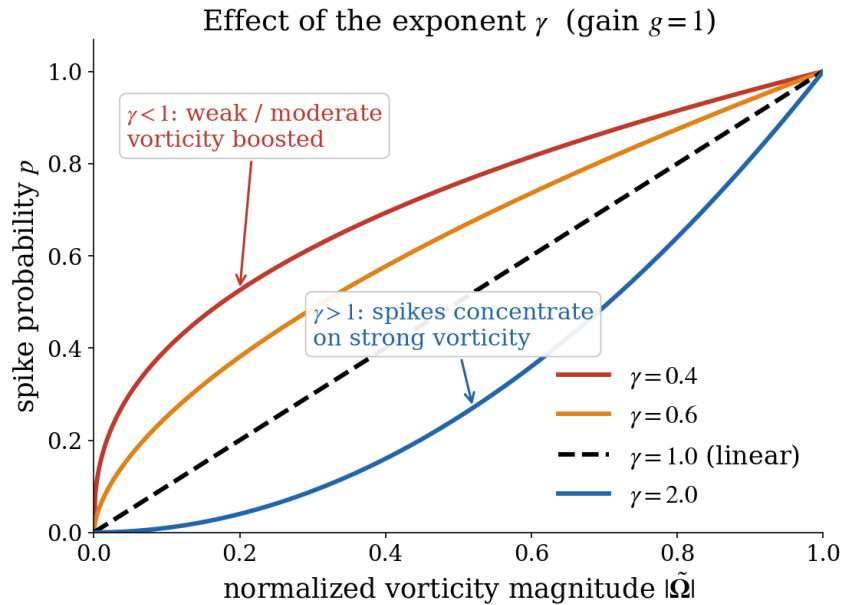


Figure 5.4. Effect of γ on the rate-encoding transfer function for fixed gain $g = 1$.

Therefore, γ does not simply change the number of spikes; it changes which parts of the vorticity field are likely to generate spikes. Lower values of γ increase the relative contribution of weak and moderate structures, while higher values suppress those regions and concentrate activity around the strongest vortical structures.

Effect of the Gain Factor

While γ changes the shape of the probability curve, the gain factor g shifts the overall level of spike activity. For clarity, Fig. 5.5 shows the effect of g when the exponent is fixed at $\gamma = 1$, so that the transfer function becomes

$$p = \min(1, g|\tilde{\Omega}|). \quad (5.17)$$

Increasing g raises the spike probability for all nonzero vorticity magnitudes and therefore produces a denser spike representation. However, for $g > 1$, the probability saturates at $p = 1$ for all values satisfying $|\tilde{\Omega}| \geq 1/g$. Once this saturation occurs, different vorticity magnitudes above that threshold become indistinguishable to the rate encoder.

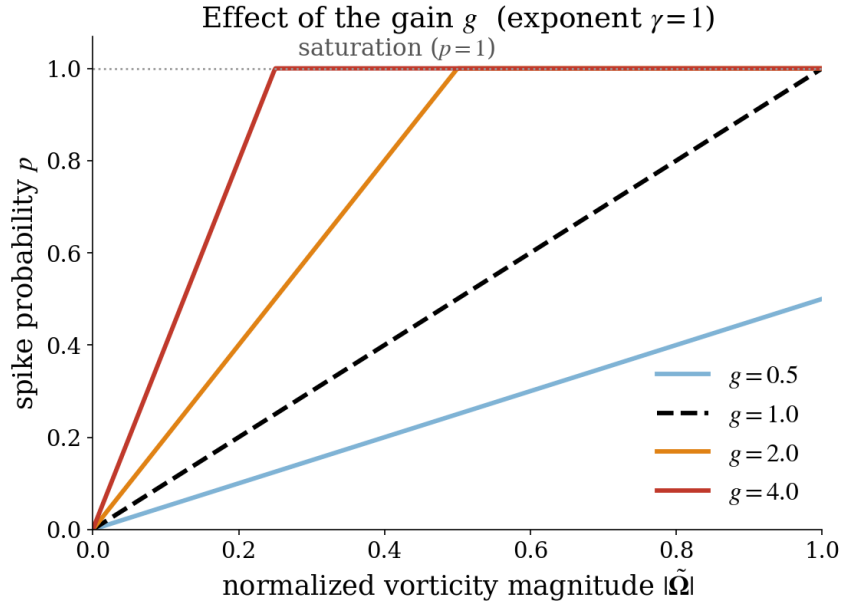


Figure 5.5. Effect of g on the rate-encoding transfer function for fixed exponent $\gamma = 1$.

The gain therefore controls a trade-off between sparsity and amplitude discrimination. Higher gain increases the number of active pixels and may make weaker structures more likely to fire, but it also increases the computational activity presented to the SNN and can saturate high-magnitude regions. Lower gain preserves sparsity more strongly, but may underrepresent weak or moderate vortical structures. The effect of g is characterized using the same firing-rate definitions introduced above: $\rho^{r,+}$, $\rho^{r,-}$, ρ_{mean}^r , and ρ_{comb}^r .

The combined visual effect of γ and g on a single vorticity snapshot is shown in Fig. 5.6. The normalized vorticity field defines the probability map $p_t(x, y)$, which is then sampled into positive spikes $S_t^{r,+}$ and negative spikes $S_t^{r,-}$. In the top row, the gain is fixed at $g = 1$ and the exponent is varied. Increasing γ progressively suppresses weak structures, reducing the combined firing rate from a dense representation to a much sparser one. In the bottom row, γ is fixed and the gain is varied. Increasing g raises the combined firing rate and activates a larger fraction of the spatial field, but also increases the saturation of high-magnitude regions.

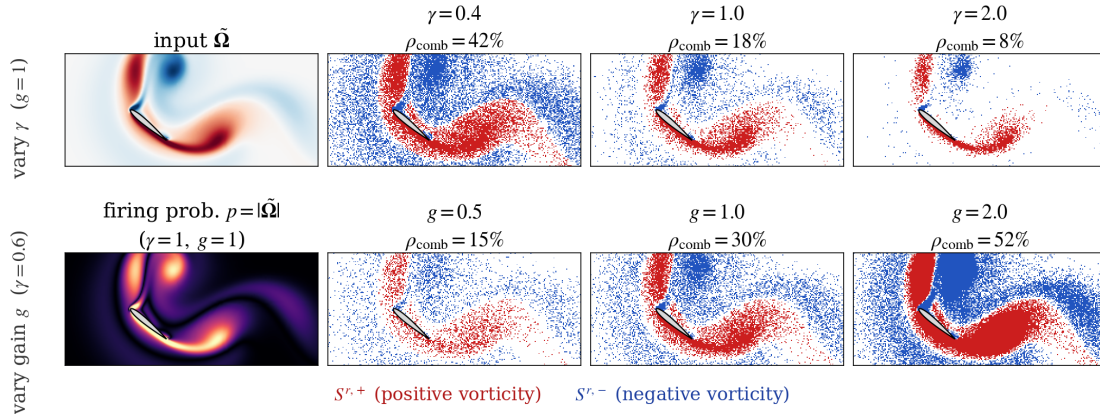


Figure 5.6. Rate encoding of a signed normalized vorticity field into positive and negative polarity spike channels.

This example illustrates why the firing-rate quantities are useful. The channel mean firing rate describes the average activity per polarity channel, while the combined firing rate ρ_{comb}^r indicates how much of the spatial field is active after rate encoding. Thus, γ and g provide two complementary ways of controlling the rate-encoded representation: γ redistributes sensitivity between weak and strong vorticity regions, while g controls the overall firing level and the onset of saturation.

5.4.2. Delta Encoding

Rate encoding represents the instantaneous magnitude of the normalized vorticity field. Delta encoding instead represents temporal changes between consecutive fields. This is relevant for aerodynamic load reconstruction because the lift response is influenced not only by the instantaneous vortical structures, but also by how those structures move, strengthen, weaken, and interact with the airfoil over time.

The general idea of delta encoding was introduced in Sec. 2.5.2. In this chapter, it is specialized to the normalized vorticity sequence $\tilde{\Omega}_{1:T'}$.

Positive and Negative Event Generation

Delta encoding begins by computing the frame-to-frame variation of the normalized vorticity field,

$$\Delta\tilde{\Omega}_t = \tilde{\Omega}_t - \tilde{\Omega}_{t-1}, \quad t = 2, \dots, T'. \quad (5.18)$$

At each spatial location, $\Delta\tilde{\Omega}_t(x, y)$ measures how much the normalized vorticity changes between two consecutive input steps. Positive values indicate an increase in normalized vorticity, while negative values indicate a decrease. Unlike rate encoding, which depends on the instantaneous magnitude $|\tilde{\Omega}_t(x, y)|$, delta encoding depends on local temporal variation. This makes the representation sensitive to the motion and evolution of vortical structures.

Delta events are generated by comparing the temporal difference field with a threshold θ_Δ . As with rate encoding, polarity is preserved by using separate positive and negative channels,

$$S_t^{\Delta,+}(x, y), \quad S_t^{\Delta,-}(x, y). \quad (5.19)$$

A positive delta event is emitted when the normalized vorticity increases by more than the threshold,

$$S_t^{\Delta,+}(x, y) = \begin{cases} 1, & \Delta\tilde{\Omega}_t(x, y) > \theta_\Delta, \\ 0, & \text{otherwise.} \end{cases} \quad (5.20)$$

A negative delta event is emitted when the normalized vorticity decreases by more than the same threshold,

$$S_t^{\Delta,-}(x, y) = \begin{cases} 1, & \Delta\tilde{\Omega}_t(x, y) < -\theta_\Delta, \\ 0, & \text{otherwise.} \end{cases} \quad (5.21)$$

The values +1 and -1 are then routed to the two binary channels $S_t^{\Delta,+}$ and $S_t^{\Delta,-}$, respectively. This separation avoids collapsing opposite temporal variations into the same channel. Positive and negative events may represent different local flow evolutions, such as local vortex strengthening, weakening, advection, or the passage of a shear-layer structure through a grid point.

Figure 5.7 illustrates this operation for two consecutive normalized vorticity fields. The fields $\tilde{\Omega}_{t-1}$ and $\tilde{\Omega}_t$ are visually similar because the frame-to-frame change is small, but their difference $\Delta\tilde{\Omega}_t$ highlights the regions where the flow is actually evolving. After thresholding at $\pm\theta_\Delta$, only the changes whose magnitude is large enough are kept as events. As a result, the delta representation is concentrated near the moving edges of vortical structures, while regions that remain nearly unchanged are silent.

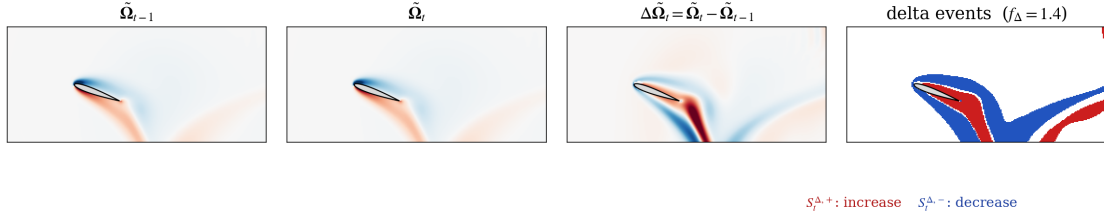


Figure 5.7. Delta encoding of two consecutive normalized vorticity fields into positive and negative event channels.

Threshold Factor

The delta threshold θ_Δ determines how large a temporal change must be before a spike is emitted. Rather than choosing this threshold directly as an arbitrary fixed value, it is obtained from the characteristic magnitude of the frame-to-frame changes in the normalized vorticity sequence.

For each simulation case, the absolute temporal change is computed from the normalized fields,

$$|\Delta\tilde{\Omega}_t(x, y)| = |\tilde{\Omega}_t(x, y) - \tilde{\Omega}_{t-1}(x, y)|. \quad (5.22)$$

A representative scale for temporal variation is then obtained by averaging this quantity over the cases, timesteps, and spatial grid points considered by the encoding pipeline,

$$\overline{|\Delta\tilde{\Omega}|} = \frac{1}{|C|} \sum_{c \in C} \frac{1}{(T'_c - 1)N_x N_y} \sum_{t=2}^{T'_c} \sum_{x=1}^{N_x} \sum_{y=1}^{N_y} |\tilde{\Omega}_t^{(c)}(x, y) - \tilde{\Omega}_{t-1}^{(c)}(x, y)|, \quad (5.23)$$

where C denotes the collection of simulation cases used to estimate the encoding statistic, and T'_c is the number of timesteps in case c after temporal interpolation, if interpolation is applied. In this calculation, C contains only the simulation cases assigned to the training partition. The resulting temporal change statistic is then kept fixed and applied when encoding the test cases.

The threshold is then defined by multiplying this characteristic scale by a dimensionless threshold factor f_Δ ,

$$\theta_\Delta = f_\Delta \overline{|\Delta\tilde{\Omega}|}. \quad (5.24)$$

The factor f_Δ is the main hyperparameter of the delta encoder. It controls how selective the event-generation rule is. If $f_\Delta > 1$, a pixel must undergo a change larger than the average frame-to-frame variation before emitting an event, which produces a sparser representation. If $f_\Delta < 1$, smaller temporal changes are sufficient to generate events, producing a denser representation.

This effect is shown in Fig. 5.8, which plots the distribution of frame-to-frame change magnitudes $|\Delta\tilde{\Omega}_t|$ over the sequence. A large fraction of pixels is static, meaning that the local normalized vorticity does not change between consecutive frames. The remaining pixels form a distribution of nonzero temporal changes. For a given value of f_Δ , only

pixels whose change magnitude lies to the right of the corresponding threshold line generate delta events. Therefore, increasing f_Δ moves the threshold to the right and removes weaker temporal changes from the spike representation. Decreasing f_Δ moves the threshold to the left and admits more events.

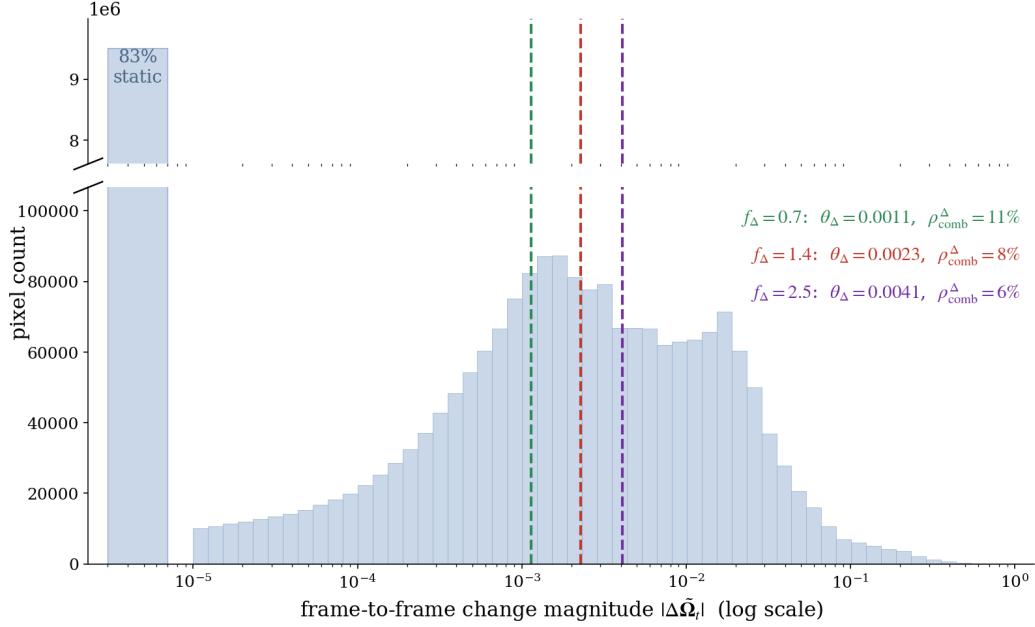


Figure 5.8. Effect of the threshold factor f_Δ on the delta-event threshold and resulting event density.

The figure also shows the corresponding combined delta firing rate $\rho_{\text{comb}}^\Delta$ for different threshold factors. This quantity has the same interpretation as in the rate-encoding section: it measures how much of the spatio-temporal field is active after encoding, now considering the positive and negative delta channels. Raising f_Δ reduces $\rho_{\text{comb}}^\Delta$, because fewer pixels exceed the event threshold. Lowering f_Δ increases $\rho_{\text{comb}}^\Delta$, because weaker temporal changes are also converted into spikes.

This creates a trade-off. A low threshold factor retains more temporal detail but may also encode small fluctuations that are not necessarily informative. A high threshold factor suppresses weak temporal changes and produces a more selective event representation, but may discard variations that contribute to the evolution of the lift coefficient. Thus, f_Δ plays a role analogous to γ and g in rate encoding: it does not modify the underlying normalized vorticity sequence, but it controls how much of that sequence is converted into spikes.

For the experiments in this thesis, the threshold factor was set to $f_\Delta = 1.4$. This value was found to provide a suitable compromise between sparsity and retention of temporal input detail. As shown in Fig. 5.8, it produces a more selective representation than lower threshold factors while avoiding the excessive suppression of temporal events produced by larger values. The choice $f_\Delta = 1.4$ therefore keeps the delta channels sufficiently sparse while still preserving the main evolving vortical structures that contribute to the

aerodynamic load response.

This definition also makes the threshold dependent on the temporal resolution of the encoded sequence. If temporal interpolation is applied, consecutive fields are closer in time and the frame-to-frame changes are generally smaller. In that case, $|\Delta\tilde{\Omega}|$ decreases accordingly, and so does θ_Δ . The threshold factor f_Δ therefore keeps the same interpretation across temporal resolutions: it specifies the event threshold relative to the average normalized change between consecutive frames.

First-Frame Treatment

Delta encoding requires a previous frame. Therefore, the first input step has no natural temporal difference field. In this thesis, the first-frame convention is defined explicitly by setting the delta channels at the first step to zero,

$$S_1^{\Delta,+} = 0, \quad S_1^{\Delta,-} = 0. \quad (5.25)$$

The first frame may still contain rate spikes, because rate encoding depends only on the instantaneous normalized vorticity field $\tilde{\Omega}_1$. However, it does not contain delta events under the zero-first convention.

This detail is important because the first few SNN states may be affected both by the absence of previous temporal information and by the initial state of the recurrent spiking dynamics. Later discussions of warm-up or initial transient behavior should therefore be interpreted consistently with this definition.

5.4.3. Information Loss Introduced by Spike Encoding

The spike-encoding pipeline converts continuous vorticity fields into sparse binary tensors. This transformation is useful because it produces inputs compatible with spiking neural networks and neuromorphic processing. However, it also introduces an information bottleneck. The encoded representation cannot preserve every detail of the original continuous field. The main sources of information loss are summarized below.

1. **Continuous-to-binary conversion.** A vorticity value contains continuous amplitude information, whereas a spike only indicates whether an event occurred in a given channel, position, and timestep. Rate encoding mitigates this loss statistically by making the spike probability depend on the vorticity magnitude, but a single binary realization does not contain the exact original value.
2. **Stochastic sampling.** In rate encoding, the probability map is determined by the normalized vorticity field, but the sampled spike tensor is stochastic unless a specific realization is fixed. Two spike tensors generated from the same field may differ, especially in regions where the spike probability is neither close to zero nor close to one.

3. **Clipping of extreme vorticity values.** Values whose magnitude exceeds the clipping range are mapped to the same normalized value. This prevents rare extreme values from dominating the spike-generation process, but it also makes all clipped amplitudes indistinguishable.
4. **Nonlinear reshaping through the exponent γ .** The exponent γ changes the relative emphasis of weak, moderate, and strong vorticity regions. Depending on its value, weaker structures may be amplified or suppressed relative to stronger structures. This is not a numerical detail only; it changes which parts of the vorticity field are more likely to generate spikes.
5. **Gain saturation.** When the gain factor causes spike probabilities to reach $p = 1$, further increases in vorticity magnitude do not change the spike probability. Saturated regions therefore lose amplitude discrimination.
6. **Delta thresholding.** Delta encoding emits events only when the temporal change exceeds the threshold θ_Δ . Temporal variations whose magnitude remains below this threshold do not produce events. This makes the delta representation sparse, but it also discards small temporal changes.
7. **Loss of absolute level in delta encoding.** Delta channels encode temporal changes $\Delta\tilde{\Omega}_t$, not the absolute vorticity level. Therefore, the baseline component of the vorticity field is not represented by delta events alone.
8. **Delta event-magnitude quantization.** Each supra-threshold temporal change is encoded as a binary event in either the positive or negative delta channel. The sign of the change is retained, but its magnitude above the threshold is not. A small supra-threshold change and a much larger change both generate the same unit event.
9. **First-frame convention.** Delta encoding requires a previous frame. Therefore, if the first delta channels are set to zero, the first spike tensor contains no temporal-change information in the delta channels. This is a consequence of the absence of a previous input step.
10. **Temporal discretization and spike-timing quantization.** The original vorticity field is available only at discrete time instants. Therefore, the spike encoder can only assign events to the available timesteps. If a temporal change would have crossed the event-generation condition at an intermediate instant in a continuous-time representation, the discrete sequence cannot locate that event exactly. The spike may be shifted to the nearest available input step, or several smaller changes may be grouped into one larger frame-to-frame difference. Since the SNN processes spikes sequentially and its membrane state depends on previous inputs, such timing errors can propagate through the spike train and affect the subsequent internal state of the network.

11. **Interpolation approximation.** Temporal interpolation is used to reduce the previous discretization effect by inserting intermediate fields between the original snapshots. This gives the spike encoder a finer temporal grid on which to place rate and delta events. However, interpolation does not recover the exact continuous-time flow and does not add new simulated physics. The intermediate fields are approximations of the underlying evolution, so interpolation can smooth or reshape temporal variations that were not directly available in the original dataset.

These mechanisms show that spike encoding is not a lossless transformation. It trades continuous field information for a sparse, event-based representation suitable for SNN processing. The objective is therefore not to preserve the vorticity field exactly, but to retain enough load-relevant aerodynamic information for reconstructing the lift coefficient history while maintaining the sparsity that motivates the neuromorphic approach.

6. METHODOLOGY

This chapter describes the methodology used to infer the lift coefficient from spike-encoded vorticity fields and to evaluate both the reconstruction performance and computational efficiency of the proposed approach. It defines the complete learning pipeline, starting from the spike tensors produced during pre-processing and ending with the reconstructed lift coefficient history. The chapter specifies the case-level data splits, the selected SNN architecture, the temporal training strategy, the loss function, and the metrics used to assess reconstruction accuracy. It also introduces the energy-efficiency metrics and the CNN-based reference architectures used for parameter-matched and energy-matched comparisons with the SNN.

6.1. Dataset Split

All dataset splits are performed at the simulation-case level. A simulation case corresponds to one complete aerodynamic encounter, including its full time-resolved vorticity sequence and the associated lift coefficient signal. Frames from the same case are never divided between training, validation, and testing.

This convention is necessary because adjacent frames within a single simulation are strongly correlated. If a frame-level split were used, the model could see a portion of one encounter during training and then be evaluated on neighboring frames from the same encounter. Such a split would overestimate generalization performance, because the test set would contain temporally adjacent information from cases already observed during training. Case-level splitting avoids this leakage and provides a more meaningful assessment of whether the model can generalize to unseen gust-airfoil interactions.

Three case-level split strategies were considered, based on model generalization difficulty. The first was an *angle-of-attack split*, in which the model was trained on one set of angles of attack, for example 20° , 40° and 60° , validated on another angle, for example 50° , and tested on another, for example 30° . This strategy tests whether the model can generalize to a different angle of attack under similar gust conditions. However, this is quite challenging because the vortex-airfoil dynamics can change substantially with angle of attack.

The second strategy was a *gust-intensity split*. In this case, simulations were classified according to the absolute gust ratio $|G|$, and the strongest gust cases were assigned to the test set. This split evaluates whether a model trained on milder gust interactions can extrapolate to more severe vortex-airfoil encounters. This turned out to be a very difficult task, due to the different aerodynamic characteristics associated with varying gust intensities.

Both the angle-of-attack and gust-intensity splits were useful as stress tests, but they were

not selected as the validation strategy. The reason is the limited size of the available dataset relative to the size of the parametric space. The simulations vary in angle of attack, gust ratio, gust sign, vortex size, and vertical vortex position. Holding out entire regions of this space creates an extrapolation problem that is very demanding for the amount of available training data. Under these splits, poor performance may reflect insufficient coverage of the aerodynamic parameter space rather than a failure of the spike-based representation or the SNN architecture itself.

For this reason, the selected reference strategy is the *random-case split*. This strategy performs a stratified random holdout at the simulation-case level. Cases are first grouped by two attributes: the angle of attack and the gust-intensity band defined by the absolute gust amplitude $|G|$: mild cases with $0 \leq |G| \leq 1.0$, medium cases with $1.0 < |G| \leq 2.5$, and strong cases with $|G| > 2.5$. Within each angle-intensity group, a fixed fraction of cases is drawn at random to form the training, validation, and test sets. The no-gust baseline case of each angle of attack is always kept in the training set.

Because the holdout is applied inside each group, the validation and test sets contain representative samples of the available angles of attack and gust intensities, rather than being concentrated in one region of the parameter space. The random draw is controlled by a fixed seed, so the resulting partition is reproducible across runs. This split reduces the generalization burden of the model, then, instead of asking the network to extrapolate to unseen angles of attack or more extreme gust ratios, it evaluates whether the model can interpolate within the sampled distribution of vortex-gust interactions. Figure 6.1 illustrates the stratified random-case split used during model development.

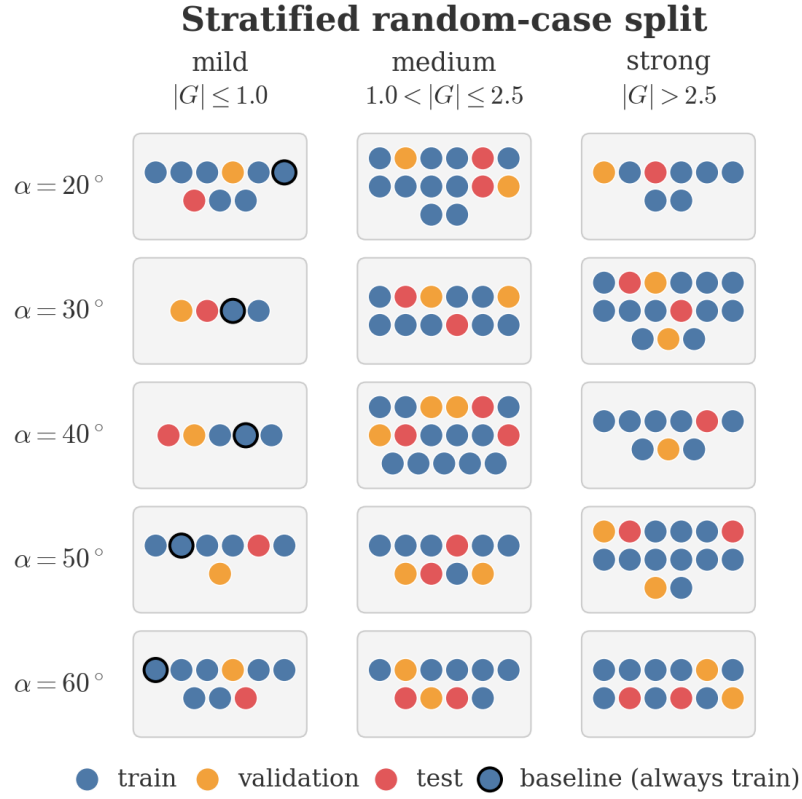


Figure 6.1. Stratified random-case split used during model development. The markers correspond to the actual simulation cases available in the dataset, grouped by angle of attack and gust-intensity band. Blue, orange, and red markers denote the 70/15/15 train, validation, and test split used during training and hyperparameter selection. Ringed markers indicate the no-gust baseline cases, which are always assigned to training.

During model development, the dataset was first divided into training, validation, and test subsets using a 70/15/15 split. After the architecture and main hyperparameters had been fixed, the final reference model was trained with an 85/15 train-test split. In this final configuration, the original test set was kept unchanged, while the validation cases were absorbed into the training set. This increased the amount of data available for model fitting without altering the held-out test cases used for final evaluation. Therefore, for the final configuration, all the validation cases shown in Fig. 6.1 would become train cases.

The final train-test partition can also be visualized directly in the physical parameter space. Figure 6.2 shows the distribution of all disturbed cases in terms of gust ratio G , normalized vortex radius R , and normalized vertical position y , with the marker shape indicating the angle of attack. The figure confirms that the test cases are scattered throughout the sampled domain rather than concentrated in a single region.

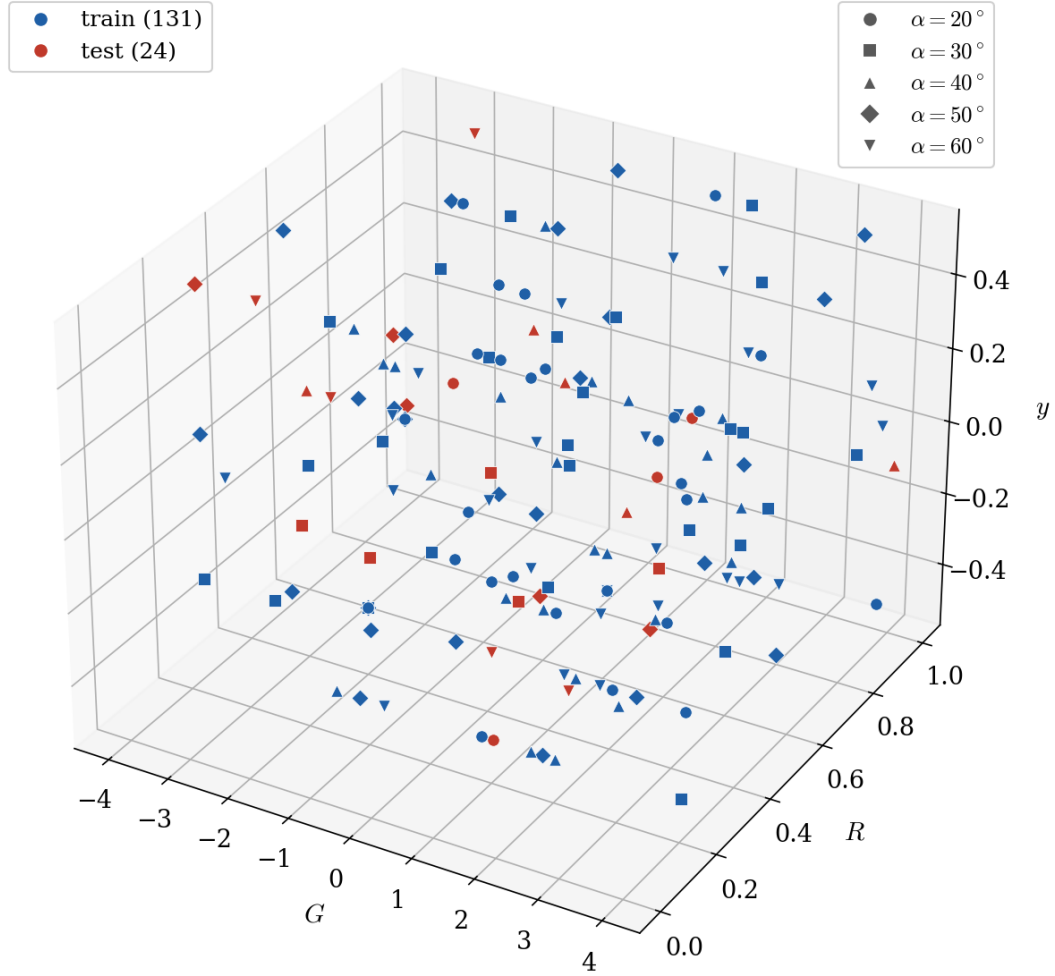


Figure 6.2. Distribution of the simulation cases in the sampled parametric space (G, R, y) for the final 85/15 train-test split.

6.2. Reference SNN Architecture

In this section, the reference SNN-based architecture will be explained in detail. A schematic of the reference network can be seen in Fig. 6.3, and a complete summary of the hyperparameters and technical details chosen for the reference model are shown in Tab. 6.1.

Parameter	Reference value
<i>Input representation and pre-processing</i>	
Input encoding	Rate Encoding
Number of input channels	2
Input channels	$[S_t^{r,+}, S_t^{r,-}]$
Normalization and clipping	P_{99}
Rate exponent	$\gamma = 0.6$
Rate gain	$g = 1.0$
Temporal interpolation factor	$N = 4$
<i>Model and temporal learning</i>	
Latent dimension	$N_z = 16$
Number of trainable parameters	58 251
Training strategy	Stateful TBPTT
Temporal window length	$T_w = 100$
Batch size	$B = 10$
<i>Optimization</i>	
Optimizer	AdamW
Initial learning rate	$\eta_0 = 10^{-4}$
Minimum learning rate	$\eta_{\min} = 10^{-6}$
Learning-rate schedule	Cosine annealing
Weight-decay coefficient	$\lambda_{\text{wd}} = 10^{-4}$
Maximum gradient norm	$g_{\max} = 1.0$
Loss function	$\mathcal{L}_{\text{MSE}}$
Number of epochs	100
<i>Final training and testing</i>	
Dataset split	85/15 train-test
Number of training encounters	131
Number of held-out test encounters	24
Test-time warm-up interval	First $N_w = 30$ time steps
Metric evaluation interval	From $t = N_w + 1$ to the end of the encounter

Table 6.1. Reference configuration used for the final model evaluation.

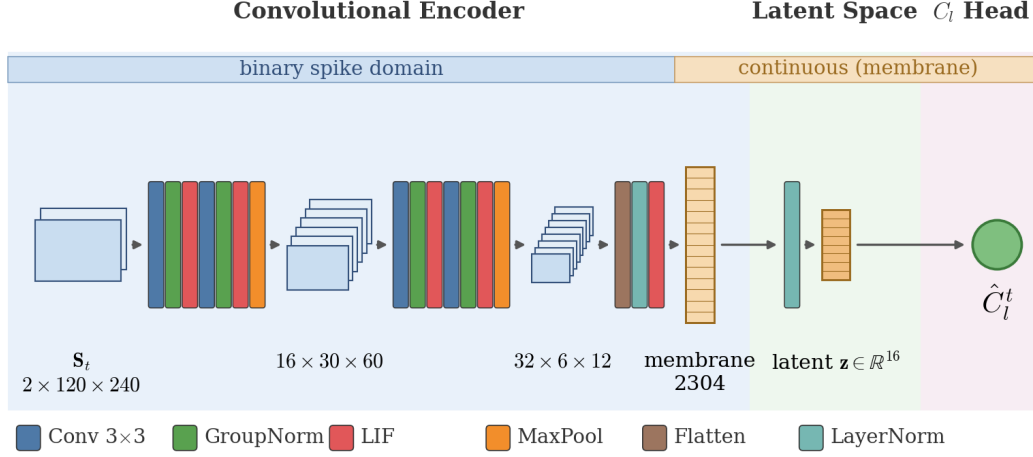


Figure 6.3. Schematic representation of the proposed SNN architecture.

6.2.1. Input Tensor to the Network

At each temporal index t , the network receives a binary spike tensor

$$S_t \in \{0, 1\}^{C_{in} \times N_y \times N_x}, \quad (6.1)$$

where C_{in} is the number of spike channels, and N_y and N_x are the spatial dimensions of the vorticity field. In this work, the spatial dimensions are $N_y \times N_x = 120 \times 240$.

The value of C_{in} depends on the spike-encoding configuration provided to the model. Different channel configurations can be considered, including rate-only channels, delta-only channels, or a combined rate-plus-delta representation. The comparison between these alternative channel configurations is treated as an experimental ablation in Ch. 7.

The temporal organisation of the input differs between training and testing. During training, the model does not receive the full spike sequence of a simulation case in a single forward pass. Instead, the sequence is divided into temporal windows. A training window is arranged as

$$S \in \{0, 1\}^{T_w \times B \times C_{in} \times N_y \times N_x}, \quad (6.2)$$

where T_w is the number of time steps in the temporal window and B is the batch size.

The complete spike sequence of each simulation case could, in principle, be processed within a single computational graph. In practice, BPTT requires the intermediate activations, membrane states, spikes, and surrogate gradient information from every included time step to be stored until the backward pass is completed. For the sequence lengths considered here, propagating gradients through an entire case would exceed the available GPU memory. For this reason, the simulations were divided into shorter temporal windows, allowing the network to process the data in their original temporal order while keeping the memory requirements manageable.

The window length T_w determines the balance between temporal context and computational cost. Increasing T_w allows gradients to propagate across a longer interval, but also increases the computational graph and its associated memory usage. Conversely, a shorter window reduces the training cost while limiting the temporal range over which parameters can be updated within one backward pass. All models were trained on a single NVIDIA GeForce RTX 3090 GPU with 24 GB of VRAM, and $T_w = 100$ was selected as a practical compromise between these two requirements.

This choice is also consistent with the temporal dynamics of the LIF neurons. The membrane contribution of an earlier input decays approximately as β^k after k time steps. For the initial decay factor $\beta_{\text{init}} = 0.9$, this contribution is reduced to approximately $0.9^{100} \approx 2.7 \times 10^{-5}$ after one window. Although β is learnable and the exact temporal memory also depends on spike activity, threshold crossings, and reset dynamics, the influence of distant inputs remains finite and progressively vanishes. Therefore, extending backpropagation across the complete simulation would significantly increase memory usage while providing little additional useful temporal context. For this reason, the selected value $T_w = 100$ is long enough to capture the effective LIF memory while remaining computationally feasible.

6.2.2. Convolutional Spiking Encoder

The purpose of the convolutional spiking encoder is to transform the high-dimensional spatial spike representation into a compact set of features that retains the information required for lift coefficient reconstruction. At every time step, the encoder receives the binary spike tensor S_t and processes it through two convolutional spiking blocks. The convolutional layers extract spatial patterns from the vorticity field, while the LIF neurons introduce temporal dynamics by integrating the incoming activity into their membrane potentials.

The encoder follows a hierarchical structure in which the spatial resolution is progressively reduced while the number of feature channels is increased. This allows the first layers to identify local flow structures at relatively high spatial resolution, while the deeper layers combine these local responses into more abstract representations over larger regions of the flow field. The architecture contains two convolutional blocks, providing a compromise between representational capacity, computational cost, and the memory requirements associated with backpropagation through time.

Each convolution uses a 3×3 kernel with unit stride and padding that preserves the spatial dimensions. Small convolutional kernels are well suited to the extraction of local spatial structures, such as shear layers, vortex cores, and wake features. Stacking two 3×3 convolutions within each block also increases the effective receptive field while requiring fewer parameters than a single convolution with a substantially larger kernel.

The first convolutional block is defined as

$$\begin{aligned} & \text{Conv}(C_{in} \rightarrow 16, 3 \times 3) \rightarrow \text{GroupNorm} \rightarrow \text{LIF} \\ & \rightarrow \text{Conv}(16 \rightarrow 16, 3 \times 3) \rightarrow \text{GroupNorm} \rightarrow \text{LIF} \rightarrow \text{MaxPool}(4 \times 4). \end{aligned} \quad (6.3)$$

The first convolution maps the input spike channels to 16 feature maps, while the second convolution preserves this number of channels and further combines the spatial information extracted by the preceding layer. Each normalized convolution is followed by a LIF layer. Within the convolutional encoder, the quantity transmitted from one processing stage to the next is the binary spike output generated by the LIF neurons, rather than their continuous membrane potential. The membrane potential is retained internally by each neuron and evolves over time, providing temporal memory, but it is not forwarded as the activation of the following convolutional layer.

This distinction preserves the spike-based nature of the encoder. Since communication between successive encoder layers occurs through binary events, inactive neurons do not transmit an activation, resulting in sparse information propagation. Such sparse, event-based communication is one of the main mechanisms through which SNNs can reduce computational activity and energy consumption when deployed on compatible neuromorphic hardware. Therefore, the convolutional encoder retains the principal neuromorphic advantage of spike-based processing while still exploiting the temporal integration provided by the internal LIF membrane states.

The first pooling operation reduces the spatial dimensions by a factor of four in each direction. For an input tensor with dimensions

$$C_{in} \times 120 \times 240, \quad (6.4)$$

the output of the first block is

$$16 \times 30 \times 60. \quad (6.5)$$

This initial reduction substantially decreases the computational and memory requirements of the following layers while retaining the dominant spatial activity. Because the pooling operation is applied to binary spike maps, max pooling preserves whether at least one neuron fired within each local pooling region.

The second convolutional block is defined as

$$\begin{aligned} & \text{Conv}(16 \rightarrow 32, 3 \times 3) \rightarrow \text{GroupNorm} \rightarrow \text{LIF} \\ & \rightarrow \text{Conv}(32 \rightarrow 32, 3 \times 3) \rightarrow \text{GroupNorm} \rightarrow \text{LIF} \rightarrow \text{MaxPool}(5 \times 5). \end{aligned} \quad (6.6)$$

At this stage, the number of feature channels is increased from 16 to 32. As the spatial resolution becomes smaller, increasing the channel dimension allows the network to retain a richer set of learned features and to represent more complex combinations of the spatial structures identified by the preceding block. The second convolution then refines these features while preserving the 32-channel representation.

The second pooling operation reduces the spatial dimensions from 30×60 to 6×12 . The resulting encoder output therefore has dimensions

$$32 \times 6 \times 12. \quad (6.7)$$

Group Normalization is applied before each LIF layer to regulate the distribution of the input currents received by the spiking neurons. Unlike Batch Normalization, Group Normalization does not depend on statistics computed across different samples in the batch. This is advantageous because SNN training is performed with relatively small batches and the level of spike activity may vary substantially between temporal windows and simulation cases. Normalizing the convolutional responses also limits excessively large or small input currents, helping to maintain stable membrane dynamics and avoiding firing regimes in which neurons become either almost inactive or excessively active.

The encoder was intentionally kept shallow. Increasing the number of spiking layers can improve hierarchical feature extraction, but it also makes SNN optimization more difficult. At every LIF layer, continuous membrane activity is converted into sparse binary events through thresholding. Subthreshold information that does not generate a spike is not propagated to the following layer, and repeated spike generation across a deeper architecture can progressively increase the loss of information. Greater depth also lengthens the surrogate-gradient propagation path and can make training less stable.

6.2.3. Latent Representation

After the second convolutional block, the binary spiking feature tensor is flattened into a 2304-dimensional vector ($32 \times 6 \times 12 = 2304$). This vector is normalized and passed through a flat LIF layer. The latent representation is not read from the output spikes of this flat LIF layer. Instead, it is read from its membrane potential. This provides a continuous, graded representation of the spiking feature state.

Let $U_t^{flat} \in \mathbb{R}^{2304}$ denote the membrane potential of the flat LIF layer at time t . The latent vector is obtained through a linear projection followed by Layer Normalization,

$$z_t = \text{LayerNorm}(W_z U_t^{flat} + b_z), \quad (6.8)$$

where

$$z_t \in \mathbb{R}^{N_z}. \quad (6.9)$$

The selected model uses

$$N_z = 16. \quad (6.10)$$

The choice $N_z = 16$ defines the selected reference model. Other latent dimensions are explored in the parametric study reported in Ch. 7, but they are not part of the final selected network.

6.2.4. Lift Coefficient Readout Head

The lift coefficient readout is a single linear layer applied to the latent vector,

$$\hat{C}_l(t) = W_{C_l} z_t + b_{C_l}. \quad (6.11)$$

The readout therefore maps

$$\mathbb{R}^{16} \longrightarrow \mathbb{R}. \quad (6.12)$$

No hidden layer is used in the final readout. This choice keeps the mapping from latent state to scalar lift coefficient simple and reduces the risk that the final regression head, rather than the spiking encoder, performs most of the modeling work. It also improves interpretability of the bottleneck, such that the latent state is forced to carry the information needed by a linear scalar readout.

The latent projection and the final lift coefficient readout are continuous operations. Consequently, the final selected model should be described as a spiking convolutional feature extractor with a continuous bottleneck and continuous scalar regression head. It is not a fully spiking end-to-end architecture.

6.2.5. Temporal State Propagation

Temporal information is propagated through the membrane potentials of the LIF neurons. At each time step, the model updates its internal state according to

$$h_t = \Phi_\theta(h_{t-1}, S_t), \quad (6.13)$$

where h_t denotes the collection of membrane states after processing frame t . In the selected architecture, h_t includes the membrane potentials of all LIF layers: the LIF neurons in both convolutional blocks and the LIF layer applied after flattening.

The LIF dynamics used in the model follow the discrete Leaky Integrate and Fire structure introduced in Sec. 2.5.1. In the selected implementation, the LIF decay factor and threshold are learnable parameters. The decay is initialized at

$$\beta_{init} = 0.9, \quad (6.14)$$

and the threshold is initialized at

$$U_{thr,init} = 0.5. \quad (6.15)$$

The temporal memory of the model is finite. It is governed by the learned leak factor, reset dynamics, threshold, spike activity, and recurrent use of membrane state across time. The model should therefore not be interpreted as having access to the full past history of the simulation. It carries a filtered memory of previous inputs through the LIF state.

6.3. Training

This section describes the procedure used to train the SNN, including the training strategy and optimization details.

6.3.1. Training Strategies

As discussed in Subsec. 6.2.1, processing an entire simulation with full backpropagation through time would require retaining the intermediate activations, membrane states, spikes, and surrogate-gradient information associated with every time step until the backward pass is completed. Although full-sequence BPTT is theoretically possible, its memory requirements increase with the sequence length and become impractical for the long, high-dimensional spike sequences considered in this work. The simulations are therefore divided into fixed-length temporal windows, limiting the portion of the computational graph retained in memory at any given time.

Backpropagation is performed only within each temporal window, giving rise to TBPTT. The window length T_w determines the temporal interval over which gradients can propagate and therefore represents a compromise between computational memory requirements and the temporal context available for learning. The two training strategies considered in this work differ primarily in how these windows are sampled and in whether the LIF membrane states are reset or preserved between consecutive windows.

Two mutually exclusive training strategies were implemented: random-window training and stateful truncated backpropagation through time. These strategies are not consecutive stages of the same training process. Instead, they define two alternative ways of presenting the temporal windows to the network and managing the internal membrane states of the spiking neurons.

Random-window training. In random-window training, fixed-length temporal windows are extracted from the available simulation cases and presented to the network in a shuffled order. Let a window begin at time index t_0 . The corresponding training sample is

$$S_{t_0:t_0+T_w-1} \longrightarrow C_I(t_0 : t_0 + T_w - 1), \quad (6.16)$$

where T_w is the temporal window length. At the beginning of every window, the membrane states of all LIF neurons are reset to zero. The model then performs full backpropagation through time within that window.

The main advantage of this strategy is the potential help in generalization. Since the windows are independent, they can be shuffled and processed in arbitrary order, increasing the diversity of local temporal contexts presented during training.

However, resetting the membrane states at the beginning of every window interrupts the temporal continuity of the simulation. A window that begins in the middle of a gust

encounter is processed without access to the internal state that would have been generated by the preceding flow evolution. Therefore, the recurrent state available to the model represents only the history contained within the current window.

This limitation is particularly relevant for the present problem because the vortex-gust interaction is a temporally coherent process. The flow state developed before the gust reaches the airfoil may influence the subsequent interaction, the resulting lift peak, and the recovery phase. By resetting the membrane potentials at arbitrary window boundaries, random-window training prevents the network from preserving this longer-term temporal context. The model must repeatedly reconstruct an internal dynamical state from an artificial zero-state initialization, even when the corresponding physical encounter is already in progress.

Stateful Truncated Backpropagation Through Time. Stateful TBPTT was used to preserve the temporal continuity lost in random-window training while retaining the computational advantages of window-based gradient truncation. This is the training strategy selected for the present thesis.

In this mode, each simulation case is processed chronologically as a sequence of consecutive, non-overlapping temporal windows. The selected window length is

$$T_w = 100. \quad (6.17)$$

The membrane states are initialized to zero once at the beginning of each simulation case. The trajectory is then divided into consecutive, non-overlapping windows of exactly T_w time steps. Only complete windows are retained, and any samples remaining after the last complete window are discarded.

For a case divided into K temporal windows, the network processes

$$S_{1:100}, S_{101:200}, \dots, S_{(K-1) \cdot 100 + 1 : K \cdot 100} \quad (6.18)$$

in chronological order.

After each window, one optimizer step is performed. The resulting membrane states are carried forward as the initial states of the following window, but are detached from the computational graph at each window boundary. This preserves temporal continuity during the forward pass while restricting gradient propagation to the current window. After the final retained window, the membrane states are reset to zero before the next batch of cases is processed.

The selected strategy therefore provides two complementary advantages. First, it preserves causal temporal continuity during the forward pass, allowing the membrane states to retain information accumulated from earlier stages of the encounter. Second, it avoids the memory requirements of applying full BPTT to the complete simulation sequence, because intermediate activations and gradients only need to be stored for one temporal window at a time.

6.3.2. Optimization Details

The network is trained using the AdamW optimizer [53]. Unlike Adam with L_2 regularization, AdamW applies weight decay directly to the model parameters and decouples it from the gradient-based optimization update. The main optimization parameters are

$$\eta_0 = 10^{-4}, \quad \eta_{\min} = 10^{-6}, \quad \lambda_{wd} = 10^{-4}, \quad g_{\max} = 1.0, \quad B = 10, \quad (6.19)$$

where η_0 is the initial learning rate, $\eta_{\min}$ is the minimum learning rate, λ_{wd} is the decoupled weight-decay coefficient, $g_{\max}$ is the global gradient-norm clipping threshold, and B is the batch size. In the selected stateful training configuration, each batch contains 10 simulation cases.

The learning rate is not held constant during training. Instead, a cosine-annealing schedule is applied over the complete training process. For epoch e , the learning rate is defined as

$$\eta_e = \eta_{\min} + \frac{1}{2} (\eta_0 - \eta_{\min}) \left[1 + \cos \left(\frac{\pi e}{E} \right) \right], \quad (6.20)$$

where $E = 100$ is the total number of training epochs. The learning rate therefore decreases smoothly from $\eta_0 = 10^{-4}$ to $\eta_{\min} = 10^{-6}$ as training progresses. This provides relatively large parameter updates during the initial learning stages and progressively smaller updates as the optimization approaches convergence.

Gradient-norm clipping is used to improve training stability. Backpropagation through the recurrent membrane dynamics of the LIF neurons can produce large gradients, particularly when temporal dependencies extend over many time steps. After the gradients have been computed, their global L_2 norm is evaluated. Whenever this norm exceeds the prescribed threshold $g_{\max} = 1.0$, all gradients are rescaled by a common factor such that

$$\|\nabla_{\theta} \mathcal{L}\|_2 \leq g_{\max}. \quad (6.21)$$

This limits the magnitude of the optimization update and reduces the risk of exploding-gradient instabilities during truncated backpropagation through time.

Mixed-precision training combines numerical formats with different levels of precision during the forward and backward passes. Under CUDA automatic mixed precision, operations that can be evaluated safely in reduced precision are executed using bfloat16, while operations that require a larger numerical precision remain in float32. This reduces the memory required to store activations and gradients and can improve computational throughput without converting the entire training process to a lower-precision format.

In this work, bfloat16 is used instead of float16. Although bfloat16 has fewer mantissa bits and therefore lower numerical precision, it preserves the same 8-bit exponent range as float32. This wider dynamic range is useful for the membrane dynamics and surrogate gradients of the SNN because small gradient values are less likely to underflow to zero. By contrast, float16 training commonly uses loss scaling, in which the loss is

multiplied before backpropagation to keep small gradients within the representable range and the gradients are subsequently unscaled before the optimizer update. Since `bfloat16` retains the exponent range of `float32`, this additional scaling procedure is not used in the present implementation and the gradient scaler remains disabled.

6.4. Loss Function

The model is trained using the Mean Squared Error (MSE) of the reconstructed lift coefficient as the objective function. For a batch containing B temporal windows, each of length T_w , the loss is defined as

$$\mathcal{L}_{\text{MSE}} = \frac{1}{BT_w} \sum_{b=1}^B \sum_{t=1}^{T_w} \left(\hat{C}_{l,b}(t) - C_{l,b}(t) \right)^2. \quad (6.22)$$

This loss directly optimizes the scalar quantity of interest. No loss is applied to the vorticity field, because the model has no decoder. No reconstruction loss is applied to the latent vector, because the latent state is not constrained to match a prescribed manifold. The bottleneck is shaped only by its usefulness for reconstructing $C_l(t)$.

Additional temporal regularization terms were not included in the final training objective. The interpolation introduced during pre-processing already provides a smoother representation of the C_l evolution, distributing magnitude changes over several intermediate time steps and reducing temporal discretization effects. Therefore, explicit temporal regularization terms are less necessary than they would be on the original, coarser temporal grid.

Furthermore, a pure MSE objective avoids introducing additional weighting coefficients and competing optimization targets. Combining multiple loss terms can push the model parameters in conflicting directions, making optimization harder and increasing the risk of convergence to a poorer local minimum. The additional terms could therefore reduce, rather than improve, the final performance unless their relative weights are carefully tuned.

6.5. Evaluation Metrics

Model performance is assessed using two complementary groups of metrics. Accuracy metrics quantify the reconstruction quality of the lift coefficient signal, whereas energy-efficiency metrics estimate the computational cost associated with inference.

6.5.1. Accuracy Metrics

The reconstruction performance is evaluated by comparing the predicted and reference lift-coefficient signals independently for each test encounter. The resulting encounter-level metrics are subsequently aggregated across the complete test set or across selected aerodynamic groups. This procedure ensures that every test case contributes equally to the reported results and prevents encounters with a larger number of temporal samples from dominating the evaluation.

At the beginning of each test encounter, the LIF membrane states are initialized at zero. To prevent this artificial cold start from influencing the reported performance, the first $N_w = 30$ time steps are used as a warm-up interval. The model processes these frames normally using the spike encoded vorticity fields, allowing the membrane states to evolve before the quantitative evaluation begins. Reconstructions generated during the warm-up interval are excluded from all reported metrics, which are computed from time step $t = N_w + 1$ until the end of the encounter. Then, for test encounter i , the evaluation interval begins after the warm-up period and contains

$$T_i^{\text{eval}} = T_i - N_w, \quad (6.23)$$

time steps, corresponding to $t = N_w + 1, \dots, T_i$.

Four complementary evaluation metrics are considered: Mean Absolute Error, Root Mean Squared Error, Relative L_2 Error, and Cosine Similarity.

Mean Absolute Error

The Mean Absolute Error (MAE) for encounter i is defined as

$$\text{MAE}^{(i)} = \frac{1}{T_i^{\text{eval}}} \sum_{t=N_w+1}^{T_i} |\hat{C}_{l,i}(t) - C_{l,i}(t)|. \quad (6.24)$$

The MAE measures the average magnitude of the reconstruction error over the complete encounter. Since it is expressed in the same units as C_l , it provides a direct and physically interpretable measure of the typical error between the predicted and reference lift coefficients. All temporal errors contribute linearly to the metric, making the MAE comparatively less sensitive to isolated large deviations.

Root Mean Squared Error

The Root Mean Squared Error (RMSE) for encounter i is defined as

$$\text{RMSE}^{(i)} = \sqrt{\frac{1}{T_i^{\text{eval}}} \sum_{t=N_w+1}^{T_i} (\hat{C}_{l,i}(t) - C_{l,i}(t))^2}. \quad (6.25)$$

The RMSE also retains the units of C_l , but penalizes large reconstruction errors more strongly because the pointwise errors are squared before averaging. This makes the metric particularly relevant for extreme gust encounters, where inaccurate reconstruction of a sharp transient response may be more significant than several smaller errors distributed throughout the signal.

Relative L_2 Error

The Relative L_2 Error for encounter i is defined as

$$\varepsilon_{L_2}^{(i)} = \frac{\|\hat{\mathbf{C}}_{l,i} - \mathbf{C}_{l,i}\|_2}{\|\mathbf{C}_{l,i}\|_2} = \frac{\sqrt{\sum_{t=N_w+1}^{T_i} (\hat{C}_{l,i}(t) - C_{l,i}(t))^2}}{\sqrt{\sum_{t=N_w+1}^{T_i} C_{l,i}(t)^2}}. \quad (6.26)$$

The Relative L_2 Error measures the magnitude of the reconstruction error relative to the magnitude of the reference lift signal. A value of zero represents an exact reconstruction, while increasing values indicate a larger error relative to the reference signal.

Unlike MAE and RMSE, the Relative L_2 Error is dimensionless and therefore facilitates comparisons between encounters with different C_l magnitudes. However, the normalization also implies that encounters with a smaller ground truth norm may exhibit a larger relative error for a similar absolute reconstruction error. The metric is therefore interpreted together with MAE and RMSE rather than in isolation.

Cosine Similarity

The Cosine Similarity for encounter i is defined as

$$\text{CosSim}^{(i)} = \frac{\hat{\mathbf{C}}_{l,i} \cdot \mathbf{C}_{l,i}}{\|\hat{\mathbf{C}}_{l,i}\|_2 \|\mathbf{C}_{l,i}\|_2} = \frac{\sum_{t=N_w+1}^{T_i} \hat{C}_{l,i}(t) C_{l,i}(t)}{\sqrt{\sum_{t=N_w+1}^{T_i} \hat{C}_{l,i}(t)^2} \sqrt{\sum_{t=N_w+1}^{T_i} C_{l,i}(t)^2}}. \quad (6.27)$$

Cosine Similarity evaluates the alignment between the reconstructed and ground truth C_l histories when both signals are interpreted as vectors. Its value lies between -1 and 1 . A value close to 1 indicates strong agreement in the overall temporal evolution, a value close to zero indicates weak alignment, and a negative value indicates an opposing signal direction.

Because the metric is normalized by the magnitudes of both signals, it is less sensitive to uniform amplitude scaling than the error-based metrics. Consequently, a prediction may obtain a high Cosine Similarity while still underestimating or overestimating the

magnitude of the C_l response. For this reason, Cosine Similarity is interpreted together with the Relative L_2 Error, MAE, and RMSE.

Encounter-Level Aggregation

Each metric is first evaluated independently for every test encounter i , obtaining $M^{(i)}$. Then, for a group containing N_g encounters, the arithmetic mean is computed as

$$\overline{M}_g = \frac{1}{N_g} \sum_{i=1}^{N_g} M^{(i)}. \quad (6.28)$$

The corresponding median is

$$\widetilde{M}_g = \text{median} \left(M^{(1)}, M^{(2)}, \dots, M^{(N_g)} \right). \quad (6.29)$$

The mean summarizes the overall performance across the evaluated encounters and remains sensitive to a limited number of difficult cases with large reconstruction errors. The median provides a complementary measure of the typical encounter and is less affected by extreme values. A substantial difference between the mean and median may therefore indicate an asymmetric metric distribution or the presence of a small number of particularly challenging-to-reconstruct encounters.

The standard deviation is not used as a primary reported statistic. Since the test set contains aerodynamically heterogeneous encounters with different lift magnitudes and reconstruction difficulties, the metric distributions may be asymmetric and influenced by outliers. Instead, the distribution of encounter-level values is represented using box plots.

6.5.2. Energy-Efficiency Metrics

The evaluation metrics introduced above determine how accurately the model reconstructs the lift coefficient, but they do not account for the computational resources required to obtain that result. Since one of the main engineering motivations of SNNs is the potential reduction of inference cost through sparse computation, the architecture is also evaluated on the test set in terms of trainable parameters, spike activity, arithmetic operations, estimated inference energy, and memory requirements.

Number of Parameters

The number of parameters, denoted by N_θ , is the total number of trainable scalar values contained in the model. It is defined as

$$N_\theta = \sum_{l=1}^L N_\theta^{(l)}, \quad (6.30)$$

where $N_\theta^{(l)}$ is the number of trainable parameters in layer l and L is the total number of layers. This count includes weights, biases, and learnable neuron parameters where applicable.

The parameter count describes the capacity of an architecture. It is also needed to establish a fair comparison between spiking and non-spiking models. If one architecture contains significantly fewer parameters, a reduction in computational cost may be due simply to the use of a smaller model rather than from spike-based processing.

Spike Activity

The potential computational advantage of an SNN depends on the sparsity of its spike trains. To quantify this property, the activity of each spiking layer is measured through its firing density. For a layer l , the firing density is defined as

$$\rho_l = \frac{\sum_{i=1}^{N_{\text{enc}}} \sum_{t=1}^{T_i} \sum_{n=1}^{N_l} S_{l,n}^{(i)}(t)}{\sum_{i=1}^{N_{\text{enc}}} T_i N_l}, \quad (6.31)$$

where N_{enc} is the number of evaluated encounters, T_i is the number of time steps in encounter i , N_l is the number of neurons in layer l , and $S_{l,n}^{(i)}(t) \in \{0, 1\}$ is the spike emitted by neuron n at time t .

A firing density of $\rho_l = 0.1$ indicates that spikes are generated in 10% of the possible neuron-time activations, whereas the remaining activations are silent. For example, if the layer contains 100 neurons and is evaluated over 100 time steps, there are 10000 possible spike emissions, of which approximately 1000 occur.

Spike activity is measured at the encoded input and at the output of each spiking layer. This metric is needed because the presence of spiking neurons does not automatically make a model computationally efficient. If most neurons fire at every time step, the network behaves as a nearly dense system and much of the event-driven advantage is lost. Conversely, low firing densities indicate that only a limited fraction of the possible synaptic connections must be activated.

Multiply-Accumulate Operations and Accumulate Operations

The arithmetic cost of the models is evaluated by *multiply-accumulate operations* and *accumulate operations*.

A *multiply-accumulate operation* (MAC) has the form

$$y \leftarrow y + wx, \quad (6.32)$$

where a continuous activation x is multiplied by a weight w and the result is added to the output y . MACs represent the main weighted operation in conventional neural networks.

For a convolutional layer l , the number of dense MAC operations is

$$N_{\text{MAC}}^{(l)} = H_{\text{out}}^{(l)} W_{\text{out}}^{(l)} C_{\text{out}}^{(l)} C_{\text{in}}^{(l)} K_h^{(l)} K_w^{(l)}, \quad (6.33)$$

where $H_{\text{out}}^{(l)}$ and $W_{\text{out}}^{(l)}$ are the spatial dimensions of the output, $C_{\text{in}}^{(l)}$ and $C_{\text{out}}^{(l)}$ are the input and output channel counts, and $K_h^{(l)} \times K_w^{(l)}$ is the convolutional-kernel size.

For a fully connected layer, the number of MAC operations is

$$N_{\text{MAC}}^{(l)} = N_{\text{in}}^{(l)} N_{\text{out}}^{(l)}, \quad (6.34)$$

where $N_{\text{in}}^{(l)}$ and $N_{\text{out}}^{(l)}$ are the input and output dimensions of the layer.

In a LIF layer, the presynaptic activation is binary. Thus, when a spike occurs, x becomes 1, and the weighted operation turns into

$$y \leftarrow y + w. \quad (6.35)$$

This operation requires an accumulation (AC) but no multiplication. When $x = 0$, no synaptic update is required.

The number of AC operations in a spiking layer is estimated from its presynaptic firing density as

$$N_{\text{AC}}^{(l)} \approx \rho_{\text{pre}}^{(l)} N_{\text{MAC,dense}}^{(l)}, \quad (6.36)$$

where $\rho_{\text{pre}}^{(l)}$ is the firing density of the spike train entering layer l and $N_{\text{MAC,dense}}^{(l)}$ is the number of MACs that the same layer would require with dense continuous activations.

The total number of operations over a complete encounter is obtained by adding the contributions of all layers and time steps. Convolutional and feedforward layers are associated with MAC operations and spiking layers are represented through AC operations. Therefore, these two metrics describe the main arithmetic workload of a model.

Estimated Inference Energy

The number of MACs and ACs is converted into an estimated inference energy by assigning a representative energy cost to each operation. The values adopted in this work are based on the approximate arithmetic-energy costs reported by Horowitz for a 45nm complementary metal-oxide-semiconductor (CMOS) process operating at 0.9V [54]. A 32-bit floating-point multiplication is assigned a cost of 3.7pJ, while a 32-bit floating-point addition is assigned a cost of 0.9pJ. Since a MAC combines one multiplication and one addition, its estimated energy cost is

$$\varepsilon_{\text{MAC}} = 3.7 + 0.9 = 4.6 \text{ pJ}. \quad (6.37)$$

An AC requires only the accumulation part, thus its energy cost is approximated as

$$\varepsilon_{AC} = 0.9 \text{ pJ}. \quad (6.38)$$

Although CUDA mixed precision is used in this thesis, reproducing the exact distribution of reduced- and full-precision operations would depend on hardware-specific kernel execution, making the comparison unnecessarily complex. The energy analysis therefore applies the same 32-bit operation costs to all evaluated configurations.

The estimated arithmetic energy of model m for encounter i is calculated as

$$\widehat{E}^{(i,m)} = N_{MAC}^{(i,m)} \varepsilon_{MAC} + N_{AC}^{(i,m)} \varepsilon_{AC}, \quad (6.39)$$

where $N_{MAC}^{(i,m)}$ and $N_{AC}^{(i,m)}$ are the total numbers of continuous and spike-driven operations performed by model m during encounter i .

To enable comparisons independently of the encounter length, the model-level estimated arithmetic energy per time step is defined as

$$\widehat{E}_{\text{step}}^{(m)} = \frac{\sum_{i=1}^{N_{\text{enc}}} \widehat{E}^{(i,m)}}{\sum_{i=1}^{N_{\text{enc}}} T_i}, \quad (6.40)$$

where N_{enc} is the number of evaluated encounters and T_i is the number of time steps processed for encounter i .

It should be clear that $\widehat{E}_{\text{step}}^{(m)}$ is an analytical approximation rather than a direct measurement of the electrical energy consumed by the GPU or by a neuromorphic processor. The actual hardware consumption also depends on memory access, spike routing, numerical precision, normalization operations, neuron-state updates, and the target hardware architecture.

6.6. CNN-Based Architectures

Two conventional CNN architectures were developed to compare the reference SNN, explained in Sec. 6.2, with non-spiking models under complementary fairness criteria: a parameter-matched CNN and an energy-matched CNN. The parameter-matched CNN model preserves approximately the same number of trainable parameters as the SNN-based architecture, allowing the two approaches to be compared at similar model capacity. The energy-matched CNN instead operates within approximately the same estimated inference energy budget, allowing their reconstruction accuracy to be compared at similar computational cost. Both CNNs receive the complete continuous vorticity field and reconstruct the instantaneous lift coefficient using independent frame-level processing.

6.6.1. Parameter-Matched CNN

The parameter-matched CNN was designed to compare the reference SNN with a conventional model of nearly identical trainable capacity. Therefore, its spatial architecture follows the reference network closely, while the spike-based components are replaced by their continuous counterparts. Because the two models have almost the same number of trainable parameters, differences in reconstruction performance cannot be attributed to one model having greater capacity than the other. Instead, they reflect the different ways in which the CNN and SNN represent and process the input data.

As shown in Fig. 6.4, the model receives the interpolated signed vorticity field Ω_t as a single continuous input channel of size $1 \times 120 \times 240$. Thus, no spike encoding or polarity separation is applied. The convolutional encoder contains two blocks with the same channel widths, kernel sizes, normalization layers, and pooling operations as the reference SNN. Each block contains two 3×3 convolutions followed by Group Normalization and ReLU activations. Max-pooling factors of 4 and 5 reduce the spatial representation to $32 \times 6 \times 12$, which is flattened into 2304 features.

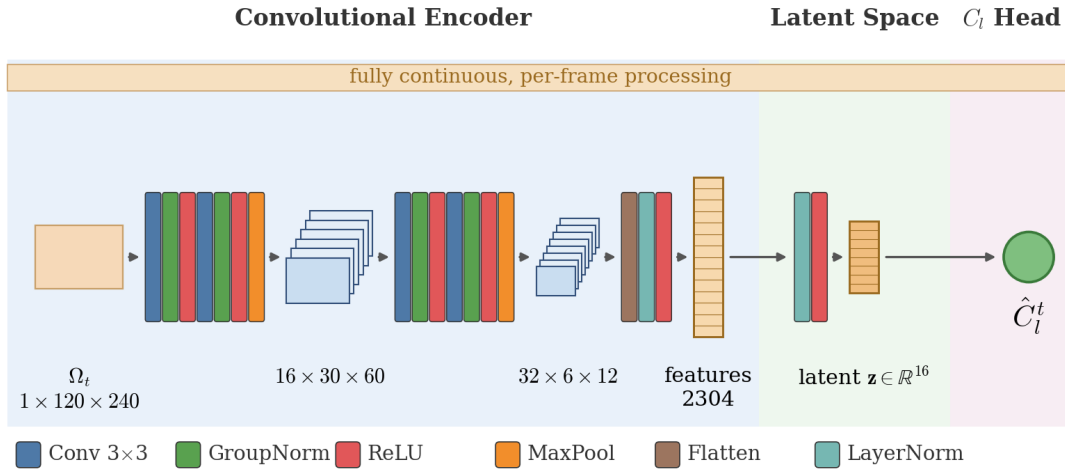


Figure 6.4. Architecture of the parameter-matched CNN.

The flattened features are normalized and projected onto a latent vector $\mathbf{z} \in \mathbb{R}^{16}$, after which a single linear layer produces the instantaneous estimate $\hat{C}_l^t$. The latent dimension and lift readout are unchanged from the reference model. However, the LIF neurons are replaced by ReLU activations, and no membrane state is preserved between consecutive frames. Therefore, the CNN is fully feedforward, meaning that each vorticity snapshot is processed independently, mapping its corresponding lift coefficient without any type of recurrence or TBPTT.

The reference SNN contains 58 251 trainable parameters, while the parameter-matched CNN contains 58 097. The difference of 154 parameters corresponds to only 0.26% of the SNN parameter count, resulting in practically the same model capacity.

The same train-test split, loss function, optimizer settings, learning-rate schedule, number

of epochs, and evaluation procedure were used for both architectures. CNN training was performed at frame level using shuffled mini-batches, since no temporal state or sequence unrolling was required. During evaluation, the same encounter-level metrics and the same exclusion of the first 30 frames were applied. Finally, note that inference energy was not matched between networks.

6.6.2. Energy-Matched CNN

The energy-matched CNN was designed to compare the reference SNN with a conventional model operating under approximately the same estimated inference-energy budget. Thus, it allows one to study whether sparse spiking computation or dense continuous computation provides greater reconstruction accuracy when both models are assigned a similar energy cost per time step.

The model follows the same general template as the parameter-matched CNN. It receives the interpolated continuous vorticity field as a single input channel, uses ReLU activations instead of LIF neurons, and processes each frame independently. The convolutional structure, 3×3 kernels, Group Normalization, pooling factors, Layer Normalization, 16-dimensional latent space representation, and linear lift coefficient readout are also retained. The only architectural change is the reduction of the convolutional block widths from 16 and 32 channels to 4 channels in both blocks.

As shown in Fig. 6.5, the input field of size $1 \times 120 \times 240$ is first processed by two convolutions with 4 output channels, followed by MaxPool(4). The second block also contains two 4-channel convolutions and MaxPool(5), producing a representation of size $4 \times 6 \times 12$. After flattening, the resulting 288 features are normalized and projected onto a latent vector $\mathbf{z} \in \mathbb{R}^{16}$, from which the instantaneous lift estimate $\hat{C}_l^t$ is obtained.

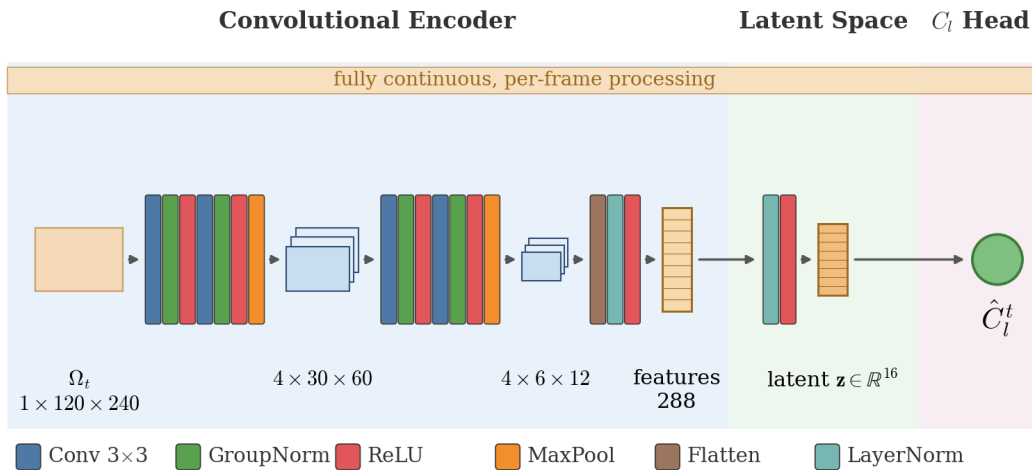


Figure 6.5. Architecture of the energy-matched CNN.

The energy-matched CNN performs 5 707 024 MACs and no ACs per time step, corresponding to an estimated energy of $26.2523 \mu\text{J}$ per step. By comparison, the reference

SNN performs 36.880 k MACs and 28.153 M spike-driven ACs per time step, resulting in an estimated energy of 25.5069 μ J. Thus, the estimated energy values differ by only 2.9%, and the two architectures can be considered practically energy-equivalent.

Matching the SNN energy budget requires a substantial reduction in CNN capacity. The energy-matched model contains 5 765 trainable parameters, compared with 58 251 in the reference SNN. This difference is expected, as the CNN performs dense MAC operations across all connections, whereas the SNN benefits from sparse spike activity and replaces many multiplications with lower-cost accumulation operations.

The model was trained and evaluated using the same data split, loss function, optimizer, learning-rate schedule, number of epochs, mixed-precision configuration, and encounter-level evaluation metrics as the parameter-matched CNN. Training was again performed using shuffled individual frames, since the model contains no temporal state.

7. RESULTS AND DISCUSSION

The following sections examine the experimental evidence supporting the principal design choices included in the reference model configuration of Sec. 6.2. Then, a reconstruction performance analysis is carried out by studying its C_I accuracy metrics across the test set. Finally, a comparison of performance and energy efficiency will be carried out between reference SNN-based model and two different CNN architectures.

7.1. Ablation Study

The following ablation studies evaluate the contribution of individual design choices relative to the reference configuration explained in Sec. 6.2. In each experiment, only the parameter or technique choice under investigation is modified, while the remaining design choices are kept fixed according to Tab. 6.1.

7.1.1. Influence of Temporal Resolution

The influence of temporal resolution was evaluated by comparing the model trained on the original dataset and the model trained on a cubic-splines interpolated dataset with $N = 4$. In the latter case, four intermediate (vorticity and C_I) snapshots are introduced between each pair of original flow fields.

Table 7.1 shows the mean and median metrics over the test set, with the corresponding distributions presented in Fig. 7.1. Each point in the box plots represents a single test case.

Grid	MAE	RMSE	Relative L_2 error	Cosine similarity
Original	0.2299 (0.1031)	0.3478 (0.1533)	0.1713 (0.1135)	0.9836 (0.9945)
$N = 4$	0.1395 (0.0627)	0.2342 (0.1015)	0.1133 (0.0801)	0.9919 (0.9971)

Table 7.1. Test case accuracy metrics obtained using the original temporal grid and cubic-spline interpolation with $N = 4$. Each entry reports the mean followed by the median in parentheses.

Increasing the temporal resolution produces a consistent improvement across all four metrics. Relative to the original dataset, interpolation reduces the mean MAE, RMSE, and relative L_2 error by approximately 39%, 33%, and 34%, respectively. The median errors are reduced by approximately 29%-39%, showing that the improvement is not limited to a small number of encounters but extends to the central part of the test distribution as

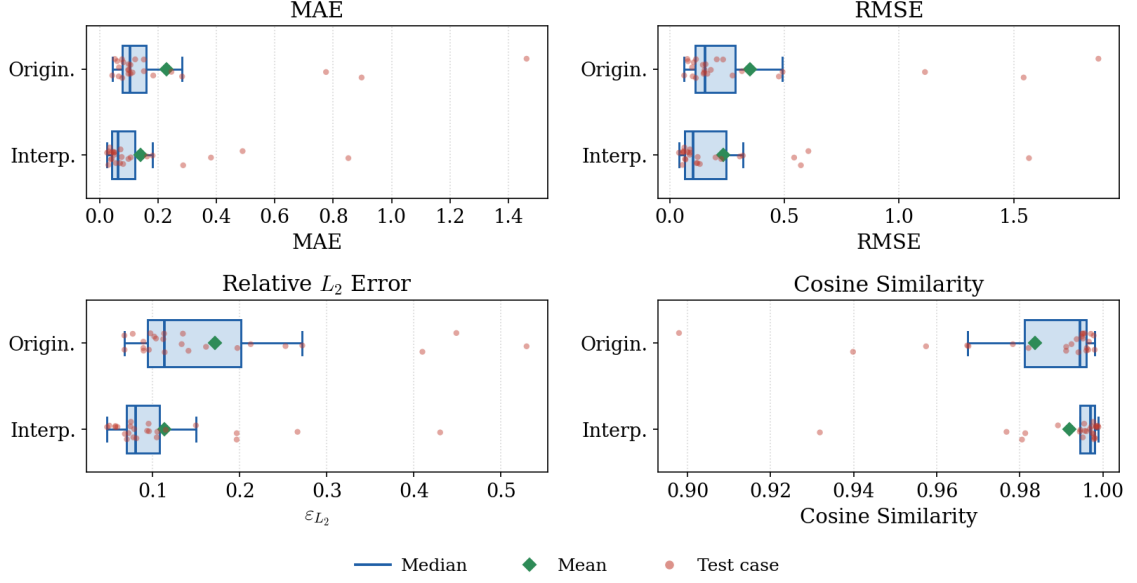


Figure 7.1. Distribution of the test set accuracy metrics for the original and temporally interpolated ($N = 4$) datasets.

well. In fact, the box plots show narrower central distributions and less severe worst-case deviations for the interpolated configuration.

The improvement is mainly explained by the reduction in temporal discretization error during spike generation. In the original dataset, only the vorticity magnitudes available at the sampled time steps can be encoded, while the intermediate flow evolution remains unresolved. Consequently, the spike probabilities may change abruptly between consecutive inputs, and the corresponding flow variation can only be assigned to the available discrete time steps. With temporal interpolation, the evolution between two original snapshots is distributed across several smaller steps. The rate encoder can therefore represent intermediate vorticity levels, producing a more gradual spike sequence and providing the LIF neurons with more frequent state updates. This allows the membrane states to follow the temporal evolution of the flow more closely.

The temporal resolution is expressed using the non-dimensional convective time $t^* = U_\infty t / c$, where $\Delta t^* = 1$ represents the time required for the freestream to travel one airfoil chord. The original interval, $\Delta t^* = 8.5 \times 10^{-3}$, corresponds to approximately 118 snapshots per convective time (see Eq. 4.8). With $N = 4$ intermediate samples inserted between each pair of original snapshots, the interval decreases to $\Delta t^* = 1.7 \times 10^{-3}$, giving approximately 588 steps over the same physical duration.

Interpolation does not extend the simulation or add new physical information, instead it only provides a finer temporal grid, allowing the vorticity evolution to be represented more gradually, providing better spike timing, and updating the LIF membrane states more frequently. The resulting sequence is approximately five times longer, increasing training and inference cost. Further interpolation would add more estimated fields between the same original snapshots without recovering unresolved physical variations.

Therefore, $N = 4$ was selected as a compromise between temporal resolution and computational cost.

7.1.2. Influence of Spike Encoding

The influence of the input encoding was evaluated using three input configurations: rate-only encoding, delta-only encoding, and a hybrid combination of rate and delta channels³. The rate-only and delta-only inputs contain two polarity channels, while the hybrid representation combines the four channels described in Ch. 5.

Table 7.2 shows the mean and median metrics over the test set, and the corresponding distributions are presented in Fig. 7.2.

Encoding	MAE	RMSE	Relative L_2 error	Cosine similarity
Rate	0.1395 (0.0627)	0.2342 (0.1015)	0.1133 (0.0801)	0.9919 (0.9971)
Delta	0.2143 (0.1028)	0.3313 (0.1657)	0.1809 (0.1366)	0.9808 (0.9919)
Hybrid	0.1642 (0.0638)	0.2491 (0.1144)	0.1242 (0.0832)	0.9915 (0.9967)

Table 7.2. Test case accuracy metrics obtained using rate-only, delta-only, and hybrid spike encoding. Each entry reports the mean followed by the median in parentheses.

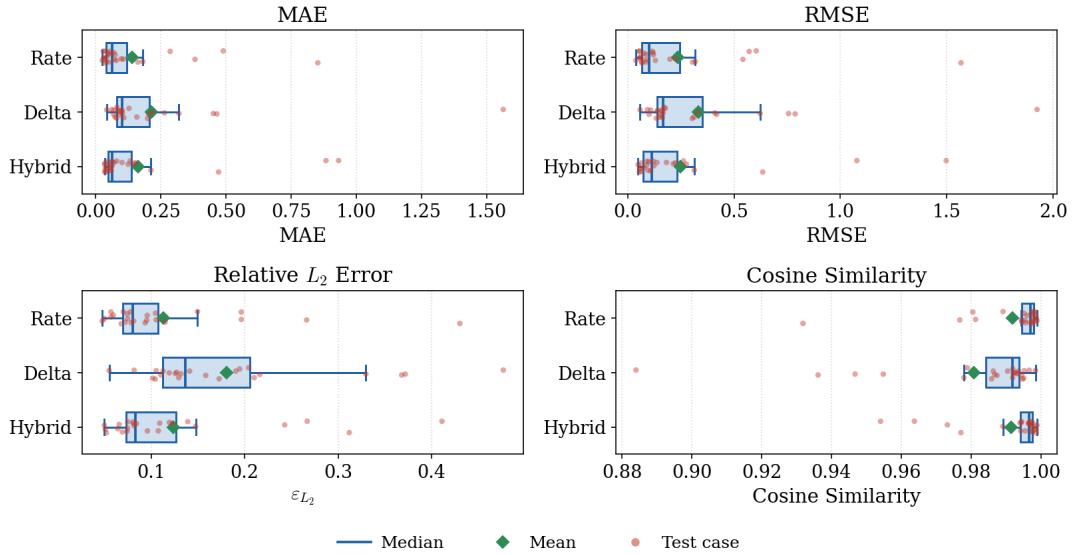


Figure 7.2. Distribution of the test set accuracy metrics for rate-only, delta-only, and hybrid spike encoding.

The delta-only configuration performs worse than the other two input-channel configurations across all accuracy metrics. Its median relative L_2 error increases to 0.1366,

³The use of a hybrid representation combining rate and delta spike channels was inspired by the NeuSpike architecture proposed by Zhu et al. [39].

compared with 0.0801 for rate-only encoding and 0.0832 for the hybrid representation. A similar deterioration is observed in the median RMSE and MAE, while the median cosine similarity decreases to 0.9919. The wider distributions shown in Fig. 7.2 also indicate that delta-only encoding produces less consistent reconstruction performance across the test encounters.

Delta encoding identifies temporal changes in the vorticity field but does not preserve its absolute magnitude. Therefore, slowly varying vortical structures may produce little spike activity despite remaining relevant to the aerodynamic load. Furthermore, temporal variations below the selected delta threshold are discarded, limiting the information available for reconstructing C_l . Although the recurrent membrane states can accumulate information from previous events, they cannot fully recover the absolute flow state when it is absent from the input representation.

The hybrid input-channel representation was expected to improve reconstruction by combining the complementary properties of both encoding methods. However, the results do not show an improvement over rate encoding alone. The rate-only configuration achieves the lowest mean and median MAE, RMSE, and relative L_2 error, while also obtaining the highest mean and median cosine similarity. The differences in median MAE and relative L_2 error between the rate-only and hybrid configurations are small, but the aggregate metrics consistently favour rate encoding. The overlapping distributions shown in Fig. 7.2 indicate that both configurations perform similarly for many encounters, although the hybrid representation does not provide a systematic improvement.

This result suggests that the rate-encoded spike representation already contains sufficient temporal information for the present reconstruction task. Although each rate-encoded frame represents the instantaneous sign and magnitude of the vorticity field, the evolution of the spike activity across consecutive time steps allows the stateful LIF neurons to detect changes in the flow. The additional delta channels are therefore partly redundant and do not provide sufficient complementary information to compensate for the increase in input complexity.

In addition, rate-only encoding is computationally cheaper. It reduces the input representation from four channels to two, decreasing the input data volume and the processing required by the first network layer during inference. Therefore, rate-only encoding was selected for the reference model.

7.1.3. Influence of Training Strategy

In this section, the random-window training and the stateful truncated backpropagation through time strategies are compared. Table 7.3 shows the mean and median metrics over the test set, and the corresponding distributions are presented in Fig. 7.3.

It is clear that stateful training produces a consistent improvement across all four accuracy metrics. Relative to random-window training, the mean MAE, RMSE, and relative

Strategy	MAE	RMSE	Relative L_2 error	Cosine similarity
Random	0.1604 (0.0680)	0.2453 (0.1042)	0.1244 (0.0945)	0.9908 (0.9959)
Stateful	0.1395 (0.0627)	0.2342 (0.1015)	0.1133 (0.0801)	0.9919 (0.9971)

Table 7.3. Test case accuracy metrics obtained using random-window and stateful TBPTT training. Each entry reports the mean followed by the median in parentheses.

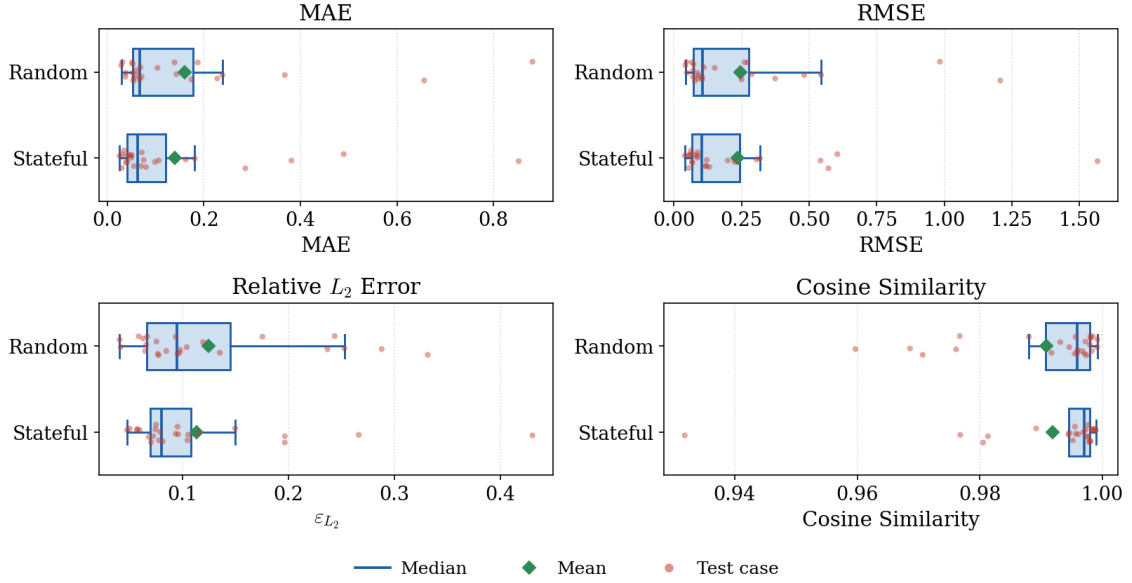


Figure 7.3. Distribution of the test set accuracy metrics for random-window and stateful TBPTT training.

L_2 error decrease by approximately 13%, 5%, and 9%, respectively. Their median values are also reduced, with the largest improvement observed in the median relative L_2 error, which decreases by approximately 15%. The mean cosine similarity increases slightly from 0.9908 to 0.9919. The box plots show a moderate displacement of the central distributions towards lower errors for stateful training, although difficult outlier encounters remain present in both configurations.

This difference is explained by the temporal nature of both the aerodynamic response and the SNN. The instantaneous lift coefficient depends not only on the current vorticity field, but also on the preceding evolution of the vortical structures around the airfoil. Random-window training removes this context and repeatedly forces the network to begin from zero membrane states. Consequently, the model must reconstruct C_l from an incomplete temporal history and from internal states that are not representative of the actual evolution of the encounter.

Stateful TBPTT avoids this repeated cold start by transferring the membrane states between consecutive windows. Although the gradient is truncated at each window boundary, the membrane temporal state is preserved, allowing each window to be processed using

information accumulated during the preceding parts of the encounter. Therefore, this method provides a temporal context longer than the training window T_w without storing the computational graph of the complete sequence. This addresses the limitation of GPU memory while retaining the temporal context necessary to reconstruct the lift response.

7.1.4. Influence of the Latent Space Dimension

The latent space dimension determines the amount of information that can be retained by the encoder before the lift coefficient is reconstructed. A latent space that is too small may not be able to capture the essential features of the flow, whereas a latent space that is too big increases the model capacity without necessarily improving generalization. To evaluate this trade-off, three models were trained using distinct latent dimensions: $N_z = 8$, 16, and 32.

Table 7.4 reports the mean and median values obtained over the test cases with the corresponding distributions shown in Fig. 7.4.

N_z	MAE	RMSE	Relative L_2 error	Cosine similarity
8	0.1739 (0.0823)	0.2826 (0.1330)	0.1403 (0.1056)	0.9889 (0.9957)
16	0.1395 (0.0627)	0.2342 (0.1015)	0.1133 (0.0801)	0.9919 (0.9971)
32	0.1652 (0.0714)	0.2536 (0.1141)	0.1267 (0.0990)	0.9918 (0.9964)

Table 7.4. Test case reconstruction metrics for different latent space dimensions. Each entry reports the mean followed by the median in parentheses.

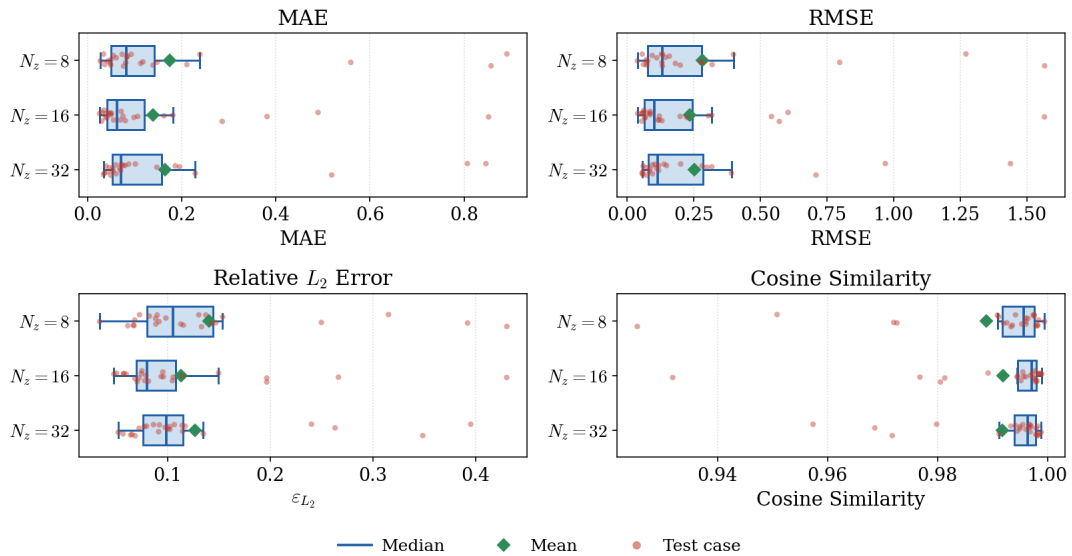


Figure 7.4. Distribution of the test set accuracy metrics for different latent space dimensions.

Increasing the latent dimension from $N_z = 8$ to $N_z = 16$ yields a clear improvement across the accuracy metrics, with mean reductions of approximately 17-20% in MAE,

RMSE, and relative L_2 error. The corresponding median errors decrease by approximately 24%. This indicates that $N_z = 8$ imposes an overly restrictive bottleneck and does not consistently preserve all the relevant information contained in the encoded flow fields.

Increasing the dimension further to $N_z = 32$ does not provide an additional benefit. Instead, its mean and median errors are consistently worse than those obtained with $N_z = 16$. The box plots show that the distributions remain partly overlapping, but their central values generally favour $N_z = 16$. Therefore, doubling the latent dimension increases the model capacity without improving reconstruction performance.

For all configurations, the mean errors exceed their corresponding medians because a small number of particularly difficult encounters generate large outliers. Since these cases remain challenging even for $N_z = 32$, their errors cannot be attributed solely to insufficient latent capacity. Therefore, $N_z = 16$ offers the most favourable balance between reconstruction accuracy and bottleneck efficiency.

The comparison with Fukami and Taira helps interpret these results [2]. Their lift augmented autoencoder organised the continuous vorticity fields within a three-dimensional latent space, whereas the present SNN achieved its best performance with $N_z = 16$. This does not imply that the underlying flow has a higher intrinsic dimension. Rather, the SNN latent space must compensate for the information loss introduced by binary spike encoding and sparse propagation through the network. The additional coordinates may therefore provide the redundancy needed to preserve the load-relevant features of the original vorticity field.

7.2. Reference Model Reconstruction Performance

This section evaluates the reconstruction performance of the reference model with parameters shown in Tab. 6.1. The analysis is organised into three parts. First, the overall reconstruction performance is studied using the accuracy metrics and representative C_l histories. Then, the effect of the angle of attack on the reconstruction error is investigated. Finally, the results are analyzed as a function of gust intensity and gust sign.

7.2.1. Overall Reconstruction Performance

Table 7.5 summarizes the mean and median values of the four principal reconstruction metrics across the 24 test encounters. Figure 7.5 shows the corresponding distributions.

The median values indicate that the reference model reconstructs the majority of the test encounters with good accuracy. A median relative L_2 error of around 8.0% and a median cosine similarity of 0.9971 indicate that the reconstructed and reference lift coefficients are quite close both in magnitude and in temporal evolution.

The mean errors are substantially larger than their corresponding medians. For example,

Metric	Mean	Median
MAE	0.1395	0.0627
RMSE	0.2342	0.1015
Relative L_2 error	0.1133	0.0801
Cosine similarity	0.9919	0.9971

Table 7.5. Reconstruction metrics of the reference model over the test set.

the mean RMSE is 0.2342, more than twice the median value. This right-skewed distribution is expected given the broad variety of aerodynamic conditions represented in the test set. The encounters combine different angles of attack, gust intensities and signs, vortex sizes, and vertical positions, resulting in very different lift responses and reconstruction difficulties. Most cases are reconstructed accurately, whereas a small number of particularly challenging parameter combinations produce much larger errors and increase the mean. The median therefore represents the typical model performance, while the mean accounts for the influence of the most difficult encounters.

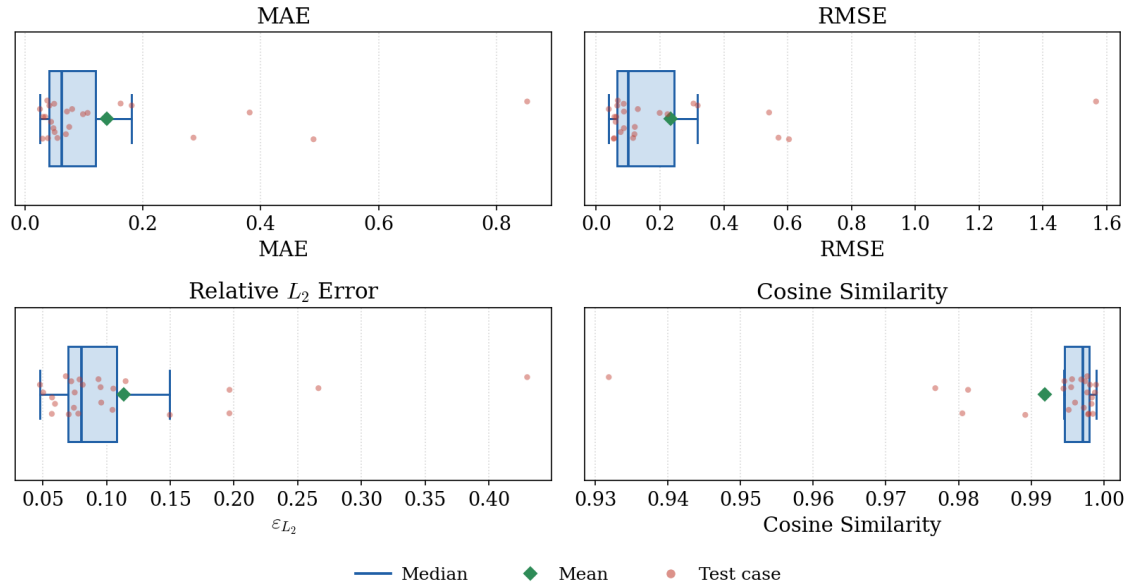


Figure 7.5. Distribution of the reference model reconstruction metrics across the test set.

Figure 7.6 shows a representative test case with $\alpha = 50^\circ$ and $G = 2.2$. Its RMSE of 0.1306 lies reasonably close to the median of the test set, making it representative of the normal model behavior.

The reconstructed signal captures the main positive increase in C_l , its timing, the subsequent reduction, and the slower wake evolution. The most visible discrepancies occur around the regions where C_l changes rapidly, particularly near the main peak, the descending branch and the following local minimum. On the other hand, the model seems to perform best once the strongest transient has passed (i.e. the vortex has passed the

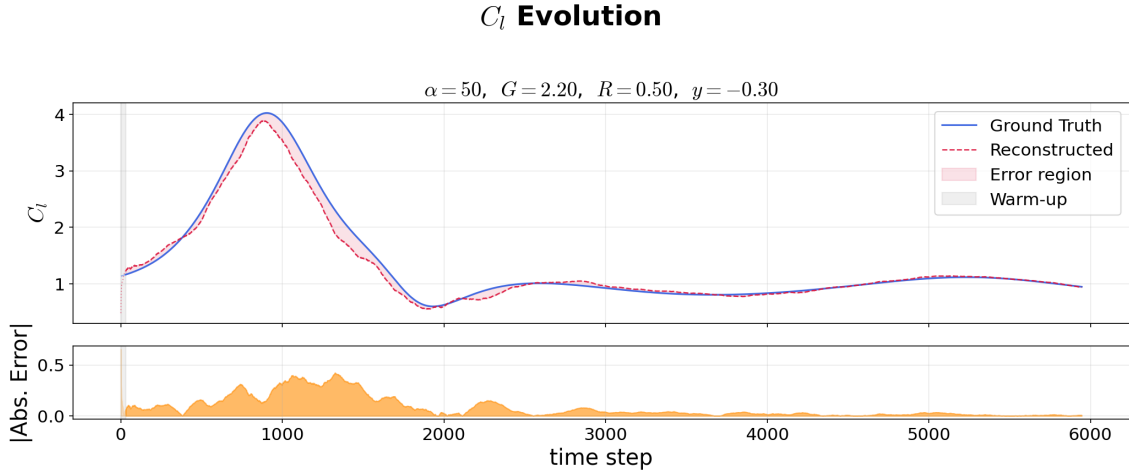


Figure 7.6. Reference and reconstructed C_l histories for a representative test case. The lower panel shows the instantaneous absolute reconstruction error. This case has an RMSE of 0.1306, an MAE of 0.0804, a relative L_2 error of 0.0814, and a cosine similarity of 0.9981.

airfoil) and the response enters in the recovery stage.

Looking at the lower panel of Fig. 7.6, one can see that the instantaneous absolute error is not distributed uniformly over the full encounter. Instead, it is primarily generated during the gust-induced transient and the early recovery phase, thus when the C_l undergoes its fastest and largest changes. Once the vortex has passed the airfoil, the instantaneous absolute error remains small.

This behavior is most evident during rapid transients, when the flow changes substantially over a small number of time steps and the information density is highest. One possible explanation is that, in this regime, the input sparsity that makes SNN processing efficient also becomes an information bottleneck. Continuous vorticity variations are reduced to binary spike events, so information loss is inevitable. Therefore, the network receives a sparser and less precisely timed representation of the flow exactly when it matters the most, that is when the aerodynamic state is evolving fastest. This makes the reconstruction job more difficult, thus increasing the likelihood of generating errors. These errors can also perturb the membrane potentials and the subsequent spike trains, causing part of the discrepancy to persist for several time steps. Once the flow enters the recovery phase and evolves more slowly, the input becomes easier to represent and the membrane states can gradually realign with the underlying dynamics. As a result, the reconstruction error decreases and the model produces a more accurate estimate of C_l .

7.2.2. Performance by Angle of Attack

Table 7.6 summarizes the mean and median reconstruction metrics for each angle of attack. Figure 7.7 shows the distribution of the relative L_2 error, selected as a representative measure of reconstruction accuracy.

α	n	MAE	RMSE	Relative L_2 error	Cosine similarity
20°	4	0.1179 (0.0777)	0.2280 (0.1430)	0.1079 (0.0879)	0.9928 (0.9961)
30°	5	0.2106 (0.0418)	0.3918 (0.0662)	0.1644 (0.0750)	0.9814 (0.9974)
40°	5	0.1630 (0.0555)	0.2384 (0.1158)	0.0979 (0.0788)	0.9953 (0.9969)
50°	5	0.0801 (0.0755)	0.1382 (0.1211)	0.0791 (0.0814)	0.9971 (0.9972)
60°	5	0.1214 (0.0699)	0.1733 (0.0871)	0.1161 (0.0954)	0.9929 (0.9955)

Table 7.6. Mean reconstruction metrics grouped by angle of attack, with median values shown in parentheses.

The results do not show a consistent deterioration as the angle of attack increases. The lowest median ε_{L_2} is obtained at $\alpha = 30^\circ$, followed closely by $\alpha = 40^\circ$ and $\alpha = 50^\circ$, whereas the highest occurs at $\alpha = 60^\circ$. However, the large differences between the means and medians, particularly at $\alpha = 30^\circ$, show that the averages are strongly affected by outliers. Similar, although smaller, differences are observed at $\alpha = 20^\circ$, 40° , and 60° . The contrary can be seen for $\alpha = 50^\circ$, where the results are compact, with similar mean and median values across all metrics.

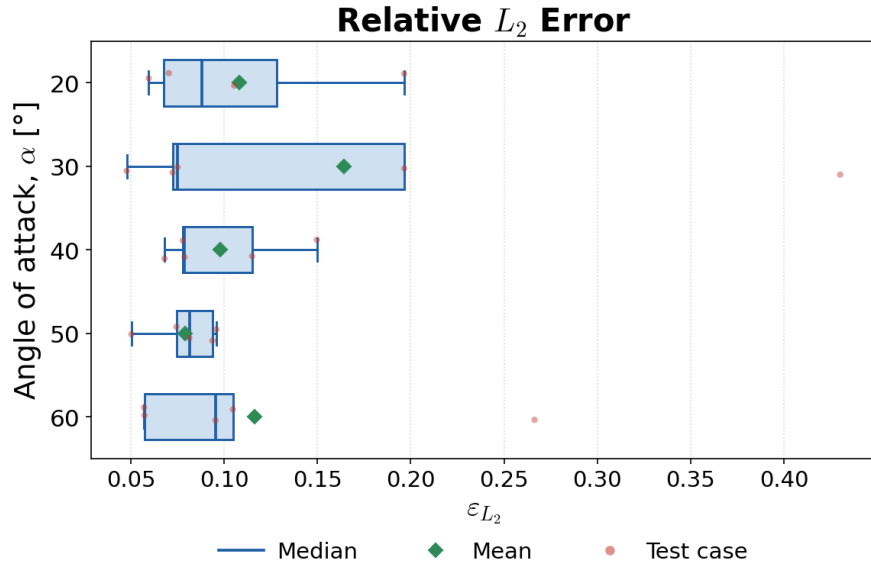


Figure 7.7. Distribution of the relative L_2 reconstruction error as a function of angle of attack. Each point represents one test encounter. The boxplots show the median and interquartile range, while the green diamonds denote the group means.

The largest individual relative L_2 error is 0.4301, occurring at $\alpha = 30^\circ$, because this group includes the most difficult-to-reconstruct test case, with $G = -3.6$. This isolated result raises the mean relative error at $\alpha = 30^\circ$ to 0.1644, despite its comparatively low median of 0.0750. Other difficult cases are present at $\alpha = 20^\circ$, 40° , and 60° , including encounters with $G = -2.2$, $G = 4.0$, and $G = -3.0$, respectively. These cases explain the differences between the mean and median values at the corresponding angles.

From a physical perspective, higher angles of attack could be expected to make reconstruction more difficult because the corresponding undisturbed (no gust) baseline flows present more unsteady dynamics. However, this trend is not observed in the present test set, as cases at high α are often reconstructed accurately. This suggests that reconstruction difficulty depends on the combined effect of angle of attack, gust intensity, vortex size, and vertical position rather than on α alone.

This conclusion must be interpreted with caution as the dataset encapsulates a large parametric space relative to its size. Therefore, the design decision of choosing a small test set (15%) and a larger training set (85%) in order to obtain sufficient generalization and reconstruction accuracy came with the price of a less robust assessment of the influence of α due to the small number of test cases, just four or five per angle of attack.

7.2.3. Performance by Gust Intensity

The dependence on gust intensity, G , is more significant than on angle of attack. As explained in Sec. 6.1, test encounters are divided into the gust categories mild, medium, and strong:

$$\begin{aligned} \text{mild: } & |G| \leq 1, \\ \text{medium: } & 1 < |G| \leq 2.5, \\ \text{strong: } & |G| > 2.5. \end{aligned} \tag{7.1}$$

Table 7.7 reports the mean reconstruction metrics for each category, with the medians shown in parentheses, as well as the amount of test cases found in each gust category.

Category	n	MAE	RMSE	Relative L_2 error	Cosine similarity
Mild	5	0.0364 (0.0312)	0.0598 (0.0572)	0.0609 (0.0573)	0.9982 (0.9984)
Medium	11	0.0865 (0.0555)	0.1561 (0.1158)	0.0912 (0.0778)	0.9956 (0.9977)
Strong	8	0.2767 (0.1346)	0.4505 (0.2643)	0.1764 (0.1275)	0.9827 (0.9918)

Table 7.7. Mean reconstruction metrics grouped by absolute gust intensity, with median values shown in parentheses.

The mild and medium categories both present relatively low reconstruction errors, although performance deteriorates from mild to medium gusts. Their median relative L_2 errors are 0.0573 and 0.0778, respectively. A more evident error increase appears for strong gusts, whose median relative error increases to 0.1275 and median cosine similarity decreases to 0.9918. The large differences between the mean and median RMSE and MAE in this category also show that some strong gust encounters produce significantly larger errors than the rest.

Analyzing Fig. 7.8, one can see that the mild-gust cases form a compact group with low error. Most medium-gust cases also remain below a relative error of 12%, although the encounter with $\alpha = 20^\circ$ and $G = -2.2$ reaches approximately 19.6%. For $|G| > 2.5$, both

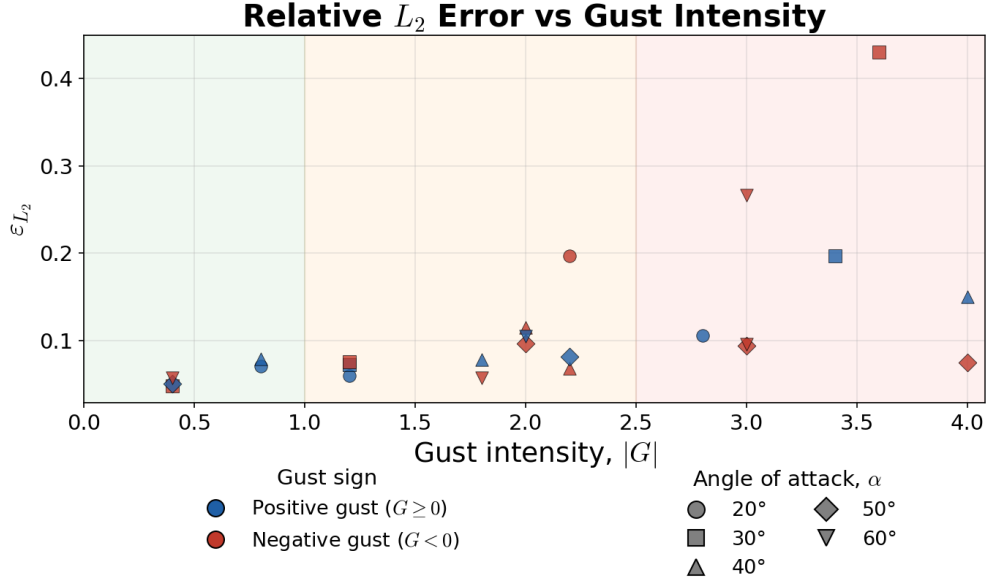


Figure 7.8. Relative L_2 reconstruction error as a function of absolute gust intensity. Colour identifies the gust sign and marker shape identifies the angle of attack. The shaded regions distinguish mild, medium, and strong gust conditions.

the average error and its dispersion increase substantially, indicating that strong gusts are generally more difficult to reconstruct. However, as for angle of attack, gust intensity alone does not determine the reconstruction accuracy. This is illustrated by the two strong negative-gust encounters presented below. The case with $G = -3.0$ achieves a relative L_2 error of 9.37%, whereas the case with the similar gust intensity $G = -3.6$ reaches 43.01%.

The successfully reconstructed $G = -3.0$ encounter is shown in Fig. 7.9. The model captures the initial negative excursion, the timing and magnitude of the main positive peak, and the subsequent decay towards the baseline response. The largest discrepancies occur during the gust-induced transient, particularly in the secondary variations following the main peak, while the slower recovery is reconstructed with high accuracy. This case demonstrates that a strong gust does not necessarily lead to poor performance, since the resulting C_l evolution also depends on the vortex size, vertical position, rotational direction, angle of attack, and their combined influence on the vortex-airfoil interaction.

The opposite behavior is observed for the $G = -3.6$ encounter shown in Fig. 7.10. Although the model captures the broad sequence of the response, it underestimates both the main negative and positive peaks and predicts a faster recovery towards the baseline state after the positive peak. This case illustrates the main limitation observed in the most difficult encounters, for which neither the load magnitude nor its complete temporal evolution is reconstructed accurately. The relative L_2 error increases to 0.4301, while the cosine similarity decreases to 0.9319, confirming that both the magnitude and temporal shape of the signal are affected.

Regarding gust sign, negative gusts generally show worse reconstruction performance than positive gusts, although the trend is not consistent across every median metric, as

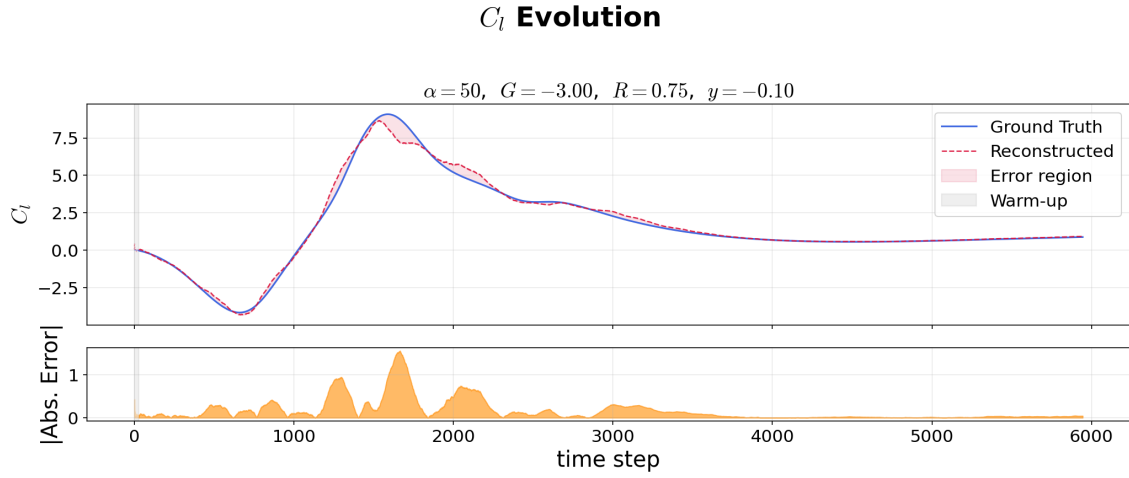


Figure 7.9. Reference and reconstructed C_l histories for a successfully reconstructed strong-gust encounter with $\alpha = 50^\circ$, $G = -3.0$, $R = 0.75$, and $y = -0.10$. The case has an RMSE of 0.3050, an MAE of 0.1626, a relative L_2 error of 0.0937, and a cosine similarity of 0.9956.

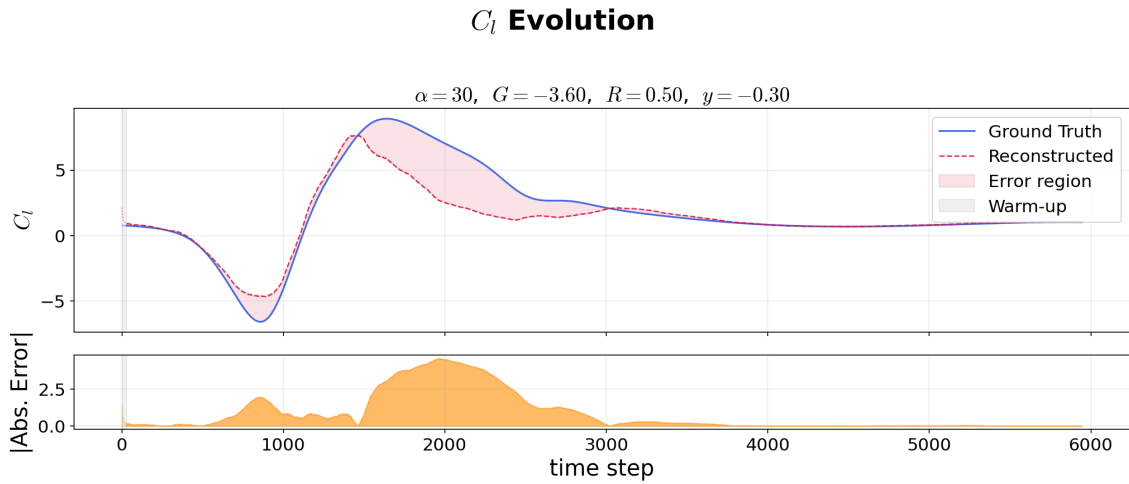


Figure 7.10. Reference and reconstructed C_l histories for the most difficult test encounter, with $\alpha = 30^\circ$, $G = -3.6$, $R = 0.50$, and $y = -0.30$. The case has an RMSE of 1.5666, an MAE of 0.8520, a relative L_2 error of 0.4301, and a cosine similarity of 0.9319.

shown in Fig. 7.11. Their median relative L_2 error increases from 0.0788 to 0.0937, their median MAE increases from 0.0555 to 0.0716, and their median cosine similarity decreases from 0.9974 to 0.9960. However, the median RMSE is lower for negative gusts, decreasing from 0.1158 to 0.0871. The group means consistently favour positive gusts, but they are strongly influenced by a small number of difficult negative-gust encounters.

One possible explanation is the moderate imbalance in the training data, which contains more positive-gust encounters (71 cases) than negative ones (55 cases), potentially biasing the model slightly towards positive-gust dynamics⁴. In addition, the vortex–airfoil interaction changes significantly with the gust rotational direction, so the observed differences may also reflect variations in the physical complexity of the positive- and negative-gust subsets.

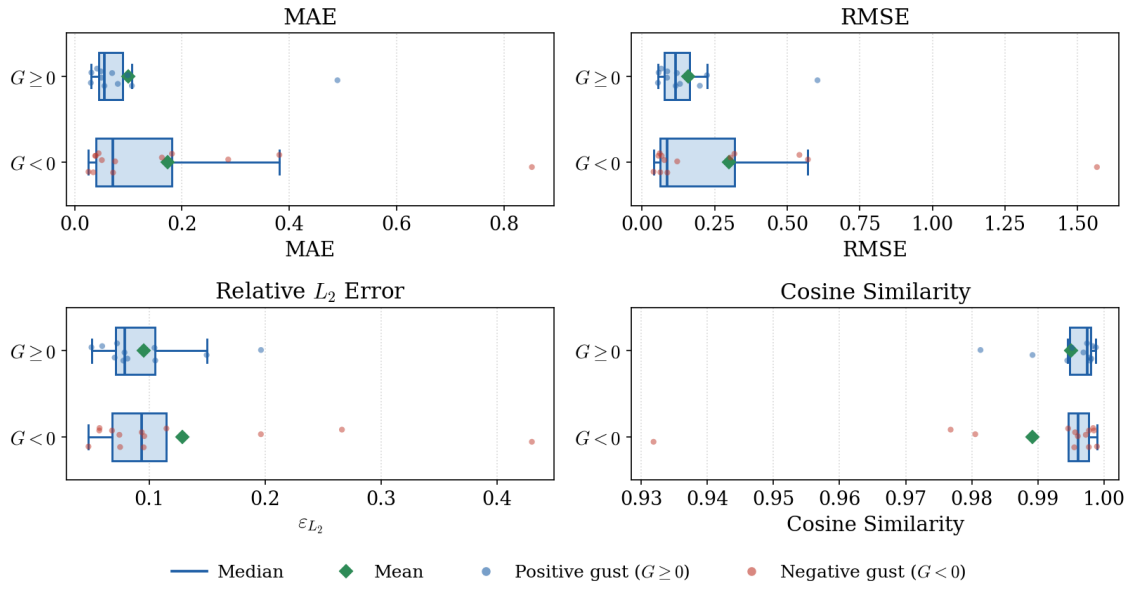


Figure 7.11. Reconstruction accuracy metrics by gust sign.

Overall, gust intensity appears to be the aerodynamic parameter with the clearest influence on reconstruction difficulty within the present test set. No monotonic relationship is observed with angle of attack, while the differences between positive and negative gusts are smaller and not consistent across all metrics. In contrast, the median relative L_2 error increases from 0.0573 for mild gusts to 0.1275 for strong gusts. Physically, larger values of $|G|$ produce stronger disturbances of the separated flow and wake, leading to larger and more rapid changes in the vorticity field and sharper transients in the lift response. These conditions are more difficult to represent through binary spikes and to track through the temporal dynamics of the SNN. Nevertheless, gust intensity is not the only determining factor, since the vortex size, vertical position, rotational direction, and angle of attack control how the gust interacts with the airfoil and explain the considerable variation observed among cases with similar $|G|$.

⁴The remaining five training cases correspond to the undisturbed encounters.

7.3. Comparison with CNN-Based Architectures

In this section the reference SNN is compared with the two conventional CNN architectures introduced in Sec. 6.6. These comparisons examine the trade-off between reconstruction accuracy and energy efficiency under two complementary conditions. The parameter-matched comparison evaluates both models at approximately equal trainable capacity, whereas the energy-matched comparison evaluates their reconstruction performance under approximately equal estimated inference energy budgets.

7.3.1. Parameter-Matched CNN

The parameter-matched CNN contains 58 097 trainable parameters, compared with 58 251 in the reference SNN. The difference is only 0.26%, meaning that the two architectures have effectively the same trainable capacity. Their reconstruction and computational results are summarized in Tab. 7.8, while the complete metric distributions over the test set are shown in Fig. 7.12.

Quantity	Reference SNN	Parameter-Matched CNN
MAE	0.1395 (0.0627)	0.0698 (0.0214)
RMSE	0.2342 (0.1015)	0.0984 (0.0324)
Relative L_2 error	0.1133 (0.0801)	0.0428 (0.0288)
Cosine similarity	0.9919 (0.9971)	0.9986 (0.9996)
Trainable parameters	58 251	58 097
MACs per time step	36.880 k	95.422 M
ACs per time step	28.153 M	0
Estimated energy per time step	25.5069 μ J	438.9434 μ J

Table 7.8. Reconstruction and computational comparison between the reference SNN and the parameter-matched CNN. Reconstruction metrics are reported as mean values, with medians in parentheses. The reported energy values are analytical operation-level estimates rather than direct hardware measurements. Therefore, they should be interpreted as indicators of the relative arithmetic efficiency of the architectures.

The parameter-matched CNN provides better reconstruction performance across all four metrics. Its mean MAE and RMSE are reduced by approximately 50.0% and 58.0%, respectively, relative to the SNN, while the mean relative L_2 error decreases by 62.2%. The differences are even more pronounced for the medians, with reductions of 65.9%, 68.1%, and 64.0% in MAE, RMSE, and relative L_2 error. The CNN also increases the mean cosine similarity from 0.9919 to 0.9986.

The same conclusion can be seen in Fig. 7.12, where even though both models present isolated difficult encounters, the CNN generally produces narrower error distributions and

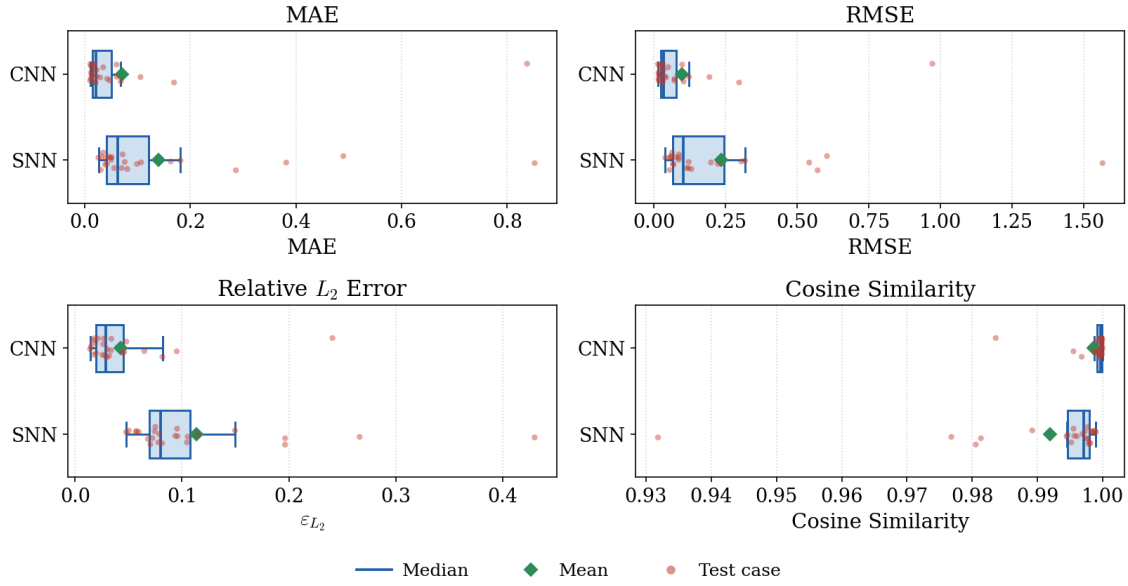


Figure 7.12. Encounter-level reconstruction metrics for the parameter-matched CNN and the reference SNN.

lower central values. This indicates that its improvement is not caused only by a small number of favourable test cases, but is present across most of the test set.

The higher accuracy of the CNN is expected because it performs a more direct supervised regression task. It receives the signed continuous vorticity field and maps it directly to the continuous lift coefficient, preserving the input magnitudes throughout the network. In contrast, the SNN must reconstruct the same continuous target from a binary spike representation. The flow information is first translated into spikes, propagated through the temporal dynamics of the LIF neurons, and finally converted back into a continuous output. These additional representation changes introduce information loss and make the reconstruction task more difficult. Therefore, the observed accuracy gap should be interpreted as a difference between the complete processing pipelines, rather than solely between the CNN and SNN architectures.

The improvement in CNN reconstruction accuracy is accompanied by a much larger estimated arithmetic cost. The CNN performs 95.422 million dense MACs per time step, producing an estimated energy of $438.9434 \mu\text{J}$ per step. The SNN instead performs only 36.880 thousand MACs in its continuous readout layers and approximately 28.153 million spike-driven ACs, resulting in $25.5069 \mu\text{J}$ per step. Under the adopted operation-level energy model, the parameter-matched CNN therefore requires approximately 17.2 times more inference energy, or equivalently, the SNN reduces the estimated energy by approximately 94.2%.

Therefore, the equal-capacity comparison reveals a clear trade-off. The direct processing of continuous vorticity fields allows the CNN to achieve higher reconstruction accuracy, but at a substantially greater estimated inference energy cost. The reference SNN does not outperform the CNN in accuracy at matched model capacity; its principal advantage

is that it maintains useful reconstruction performance while requiring far less energy. This trade-off could be exploited in a low-power onboard monitoring system for small unmanned aircraft, where the model would operate continuously to identify large gust-induced load variations or potential load-threshold exceedances. In such an application, reliable detection of significant load events may be more important than reproducing every instantaneous value of C_l with maximum accuracy, particularly when battery capacity and onboard computing resources are limited.

7.3.2. Energy-Matched CNN

The energy-matched comparison addresses the main engineering motivation for using a spiking architecture. This experiment constrains a CNN-based model to approximately the same estimated inference energy budget as the reference SNN-based architecture. Thus, examining which processing approach provides the most accurate lift reconstruction for a comparable arithmetic energy cost.

Their reconstruction and computational results are summarized in Tab. 7.9, while the encounter-level metric distributions are shown in Fig. 7.13.

The energy-matched CNN requires 5.707 M dense MACs per time step, corresponding to an estimated energy of $26.2523 \mu\text{J}$. The reference SNN instead performs 36.880 k MACs and 28.153 M spike-driven ACs per time step, resulting in an estimated energy of $25.5069 \mu\text{J}$. The difference between the two estimates is only 2.9%, and the architectures can therefore be considered practically energy-equivalent.

Quantity	Reference SNN	Energy-Matched CNN
MAE	0.1395 (0.0627)	0.1584 (0.0610)
RMSE	0.2342 (0.1015)	0.2535 (0.0970)
Relative L_2 error	0.1133 (0.0801)	0.1191 (0.0893)
Cosine similarity	0.9919 (0.9971)	0.9901 (0.9965)
Trainable parameters	58 251	5 765
MACs per time step	36.880 k	5.707 M
ACs per time step	28.153 M	0
Estimated energy per time step	$25.5069 \mu\text{J}$	$26.2523 \mu\text{J}$

Table 7.9. Reconstruction and computational comparison between the reference SNN and the energy-matched CNN. Reconstruction metrics are reported as mean values, with medians in parentheses. The reported energy values are analytical operation-level estimates rather than direct hardware measurements. Therefore, they should be interpreted as indicators of the relative arithmetic efficiency of the architectures.

At approximately equal estimated inference energy, the two architectures achieve broadly

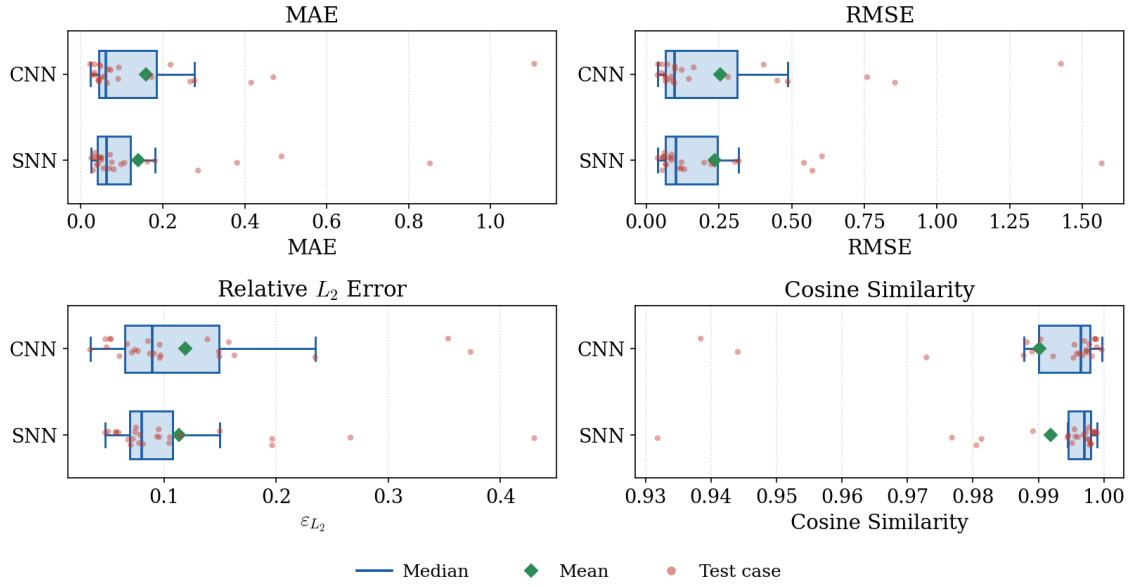


Figure 7.13. Encounter-level reconstruction metrics for the energy-matched CNN and the reference SNN.

comparable reconstruction performance. The reference SNN obtains lower mean errors across the three error metrics. Its mean MAE, RMSE, and relative L_2 error are respectively 12.0%, 7.6%, and 4.9% lower than those of the energy-matched CNN. The SNN also achieves a slightly higher mean cosine similarity, increasing from 0.9901 to 0.9919.

The median results are more evenly divided. The energy-matched CNN obtains slightly lower median MAE and RMSE, with reductions of 2.7% and 4.4%, respectively. Conversely, the SNN achieves a 10.3% lower median relative L_2 error and a slightly higher median cosine similarity. The substantial overlap between the distributions in Fig. 7.13 indicates that neither architecture is uniformly superior for every encounter. Nevertheless, the lower SNN mean errors and its narrower central distributions for several metrics suggest greater overall robustness across the test set.

The main result of this comparison is not a large difference in reconstruction accuracy, but the architectural capacity that can be retained within the same estimated energy budget. The reference SNN contains 58 251 trainable parameters, whereas the energy-matched CNN contains only 5 765. Thus, the SNN accommodates approximately 10.1 times more trainable parameters at a similar estimated arithmetic energy cost. This difference arises because the CNN evaluates all convolutional connections using dense MAC operations, while the SNN activates synaptic operations according to the measured spike activity and replaces most multiplications with lower-cost ACs.

This additional capacity may allow the SNN to compensate for some of the disadvantages introduced by spike-based processing. The SNN receives a stochastic binary representation in which part of the original vorticity magnitude information has been discarded, whereas the CNN receives the complete continuous flow field. Despite this input-level advantage for the CNN, the SNN maintains comparable and, according to the mean metrics,

moderately better reconstruction performance under the same estimated energy constraint. Its temporal membrane states may also contribute by integrating information across consecutive frames, while the CNN processes every snapshot independently.

8. CONCLUSIONS, LIMITATIONS AND FUTURE WORK

8.1. Conclusions

This thesis investigated whether a Spiking Neural Network could reconstruct the lift coefficient evolution, $C_l(t)$, of an airfoil undergoing extreme vortex-gust interactions from continuous vorticity fields converted into spike trains. The main objective was to determine whether the transformation from continuous aerodynamic fields to sparse binary events preserves sufficient load-relevant information for accurate reconstruction. The results support this hypothesis and demonstrate that a convolutional SNN-based architecture can recover the main temporal evolution of the lift coefficient across gust encounters that were not included during training. Therefore, this work establishes the feasibility of spike-based aerodynamic load reconstruction as an intermediate step between high-dimensional flow sensing and future neuromorphic prediction and control systems.

The proposed pre-processing pipeline explored two complementary spike-encoding mechanisms. Rate encoding represents the sign and magnitude of the normalized vorticity through separate positive and negative channels, while delta encoding generates spike events associated with positive and negative temporal variations. However, the ablation study showed that the two-channel rate-only representation achieved better overall reconstruction performance than the other input-channel configurations. This result indicates that the spatial distribution, sign, and magnitude of the instantaneous vorticity already provide sufficient information for the stateful SNN to learn the relevant temporal evolution. Although the selected rate encoding inevitably discards part of the continuous flow-field information, the reconstruction results show that the remaining spike activity retains the principal flow features required to estimate the aerodynamic lift coefficient.

The architecture was able to compress the encoded vorticity fields into a compact load-relevant representation before reconstructing the lift coefficient. The presented ablation studies showed that a latent space dimension of $N_z = 16$ provided the most appropriate balance between compression and reconstruction accuracy. This result indicates that the main features of the high dimensional flow can be expressed by a low dimensional vector while retaining sufficient information to reconstruct the target aerodynamic quantity.

Increasing the temporal resolution of the dataset by applying cubic-spline interpolation with four intermediate snapshots substantially improved all reconstruction metrics. The higher temporal resolution produces a smoother evolution of the vorticity fields, thus allowing transient changes to be represented through a more gradual sequence of events. This reduces temporal discretization errors and the misplacement of spike activity during rapidly changing phases of the interaction.

The stateful training strategy was beneficial for preserving the longer temporal evolution

of each encounter under the available hardware constraints. Processing complete encounters in a single backpropagation pass was not possible because of the large memory required to store the temporal computational graph. This constraint was addressed through stateful truncated backpropagation through time, that is, using windows of $T_w = 100$ snapshots and transferring the membrane potential states from one consecutive window to the next. Therefore, the computational graph was truncated at each window boundary, but the forward temporal state of the network was preserved. This strategy allowed the model to retain information from earlier portions of the encounter and to learn from temporal evolutions extending beyond an individual training window.

For the final reference configuration, the evaluation over the 24 test encounters produced mean MAE, RMSE, relative L_2 error, and cosine similarity values of 0.1395, 0.2342, 0.1133, and 0.9919, respectively. The corresponding median values were 0.0627, 0.1015, 0.0801, and 0.9971. The differences between the mean and median errors indicate the presence of a small number of substantially more difficult encounters rather than uniformly poor performance across the test set. For most cases, the reconstructed signal follows the reference C_l history closely in both magnitude and temporal evolution.

The largest reconstruction errors are generally concentrated around gust impingement and the sharp C_l peaks associated with rapid transients such as vortex deformation, flow separation, and interaction with the pre-existing baseline wake. These intervals combine large temporal gradients with strongly nonlinear aerodynamic behavior, making them the most difficult phases of the signal to reconstruct. In contrast, the later recovery phase presents lower reconstruction errors as the lift coefficient evolution becomes smoother. Furthermore, performance does not decrease monotonically with angle of attack, indicating that reconstruction difficulty cannot be explained by the incidence angle alone. Instead, it depends on the specific combination of gust intensity, rotational direction, gust position, angle of attack, and the resulting interaction with the undisturbed baseline flow. Strong-gust encounters show greater error dispersion and contain the principal outliers. Negative gusts also show a general tendency towards worse reconstruction in relative L_2 error, MAE, and cosine similarity, although this trend is not consistent across every metric within the selected test set.

The comparison with the parameter-matched CNN establishes the accuracy cost associated with the spike-based processing pipeline. At approximately equal trainable capacity, the conventional CNN achieves substantially better reconstruction performance, with reductions of approximately 50.0%, 58.0%, and 62.2% in mean MAE, RMSE, and relative L_2 error, respectively. However, this accuracy improvement is obtained at a considerably greater estimated arithmetic cost. The parameter-matched CNN requires approximately 17.2 times more estimated inference energy per time step than the reference SNN. Equivalently, the SNN reduces the estimated energy by approximately 94.2% while maintaining useful lift reconstruction performance. The reference SNN therefore does not outperform a similarly sized CNN in accuracy, but it achieves its reconstruction using substantially fewer high-cost arithmetic operations.

The energy-matched comparison provides the complementary engineering result. When the CNN is constrained to an estimated energy budget within 2.9% of that of the reference SNN, the difference in reconstruction accuracy becomes considerably smaller. The reference SNN also retains substantially greater model capacity, with 58 251 trainable parameters compared with 5 765 for the energy-matched CNN.

This additional capacity partly compensates for the more demanding representation process performed by the SNN. The CNN receives a continuous vorticity field and maps it directly to a continuous lift coefficient. The SNN must instead convert the continuous input into binary spikes, preserve the relevant information through sparse and threshold-dependent neuronal dynamics, and finally recover a continuous value of C_l from the resulting internal representation. The results therefore suggest that much of the remaining reconstruction error is associated with the information discarded during spike encoding and with the difficulty of decoding sparse spike activity back into a continuous aerodynamic quantity, rather than with insufficient model capacity alone. Despite these additional constraints, the SNN achieves competitive accuracy while retaining approximately ten times more trainable parameters within a similar estimated arithmetic energy budget.

Taken together, the two CNN comparisons reveal the principal accuracy-efficiency trade-off of the proposed approach. Direct processing of the continuous vorticity field provides the best reconstruction accuracy when computational cost is not restricted. However, once the estimated inference-energy budget is constrained, the accuracy advantage of the conventional CNN disappears because its dense multiply-accumulate operations require a substantial reduction in model capacity. Sparse spike activity and the replacement of many multiply-accumulate operations with lower-cost accumulation operations allow the reference SNN to retain a larger architecture within the same approximate energy budget.

The main conclusion of this thesis is therefore twofold. First, the transformation of continuous vorticity fields into sparse rate-encoded spike trains does not eliminate the aerodynamic information required for lift coefficient reconstruction. Despite the inevitable information loss associated with spike encoding, temporal discretization, and spiking-neuron dynamics, the proposed model retains sufficient information to reconstruct the evolution of C_l with useful accuracy. Second, the comparison with conventional CNNs indicates that this reconstruction can be achieved with a favourable architecture-level trade-off between accuracy and estimated arithmetic energy. Therefore, this work provides a proof of concept for the application of SNNs to extreme unsteady aerodynamics and, more generally, to the field of fluid mechanics.

8.2. Limitations and Future Work

The first limitation concerns the information discarded during the transformation of continuous vorticity fields into rate-encoded binary spike trains. Even though the results show that the spike-encoded fields retain sufficient aerodynamic information for the reconstruc-

tion of C_l , they do not reveal how much of the remaining error originates in the encoding stage and how much is introduced by the architecture and training process. This distinction is difficult to establish because there are currently no directly comparable studies that quantify the separate contributions of spike encoding and SNN processing. Future research should therefore focus on developing suitable comparison frameworks to distinguish the information loss introduced by rate encoding from the errors associated with the network used to process the resulting spike trains.

A further limitation is the relatively small number of available simulations compared to the parametric space. The dataset contains variations in angle of attack, gust intensity, rotational direction, vortex radius and relative vertical position, but only a limited number of combinations of these variables are present. As a consequence, the model must generalize across relatively large gaps in the parametric space. Furthermore, the small test set also limits the statistical strength of the drawn conclusions, especially when individual groups only contain a few encounters and their means are strongly affected by outliers. A larger dataset would provide a denser coverage of the existing parametric space, allow the use of a larger test set and make it possible to construct groups where one parameter is varied while the remaining ones are kept fixed. This would improve both model training and the physical interpretation of the results.

The available GPU memory also constrained the temporal training procedure. Each encounter was divided into windows of $T_w = 100$ time steps and processed using stateful truncated backpropagation through time. Preserving the membrane states between consecutive windows is a way to transfer memory during the forward pass of the network. However, these states must be detached from the computational graph after each backward pass, which means that an error observed in a later window cannot propagate its gradient through the complete preceding history of the encounter. If enough memory was available, each case could be processed as a single temporal sequence and allow every weight update to account for the complete evolution of the C_l . Therefore, future studies could analyze the use of longer temporal windows, full-sequence training or distributed training to determine whether extending the gradient history improves the reconstruction of long-sequence C_l dynamics.

The present work is limited to two-dimensional vortex-airfoil interactions, so it does not simulate the full complexity of a real wing encountering an atmospheric disturbance. Three-dimensional effects may alter both the aerodynamic response and the spike activity generated from the flow field. Extending the approach to 3D simulations would be an important step to assess whether the load-relevant information remains sufficiently low dimensional for a SNN to reconstruct. Such a study would require significantly greater computational resources due to the increase in the input dimensionality, dataset size, and needed training memory.

The task considered here is reconstruction rather than forecasting. Thus, for a vorticity field observed at time t , the model estimates the lift coefficient at that same instant. Nev-

ertheless, one could imagine a future model using an early sequence of flow observations to predict the subsequent evolution of the C_l over a prescribed forecast horizon. Such predictive capability would represent a critical step towards gust-load alleviation, as it would provide the signal required for a control system to make the necessary adjustments before the most severe peaks are reached.

The use of complete vorticity fields around the airfoil as inputs limits the direct applicability of the current system. Measuring the full flow field around an airfoil is feasible in numerical simulations, but it is certainly not realistic for an onboard sensing system. A more deployable version would rely on a limited number of measurements obtained from sensors located upstream of the airfoil or embedded in its surface. Future work could investigate how reconstruction and prediction performance vary with different sensor configurations. Therefore, combining sparse upstream flow sensing with load forecasting would push the proposed method towards a system capable of supporting real-time aerodynamic control.

The computational-efficiency comparison is based on an analytical energy model rather than direct measurements of physical power consumption. The adopted estimate assigns fixed energy costs to 32-bit MAC and AC operations, but it does not fully account for things like memory access, data movement, spike routing and membrane-state updates. This is particularly relevant because the SNN must preserve a large set of membrane states between time steps, whereas the feedforward CNN has no temporal-state memory. For this reason, the reported 17.2x difference for the parameter-matched CNN and the approximate equivalence of the energy-matched architectures should be interpreted as indicators of relative arithmetic efficiency rather than definitive energy measurements. Future work should evaluate the models on specific hardware platforms and measure energy consumption, memory usage, throughput, and inference latency directly.

Finally, the proposed model is a convolutional SNN-based architecture rather than a fully neuromorphic implementation. Even though it incorporates LIF neurons and rate-encoded spike-train inputs, some operations are not directly compatible with current neuromorphic chips. In particular, the membrane potential of the flattened LIF layer is read by a continuous latent projection, and components such as Group Normalization are not naturally supported by current neuromorphic hardware. For this reason, the present work should be interpreted as a proof of concept showing that SNN-based architectures can reconstruct aerodynamic loads from spike-encoded flow fields, rather than as a complete neuromorphic approach. Future work should focus on adapting the architecture to the constraints of specific neuromorphic chips. Only after this reformulation the potential benefits of SNNs, including reduced energy consumption and low inference latency, can be properly assessed and exploited.

BIBLIOGRAPHY

- [1] A. R. Jones, O. Cetiner, and M. J. Smith, “Physics and modeling of large flow disturbances: Discrete gust encounters for modern air vehicles,” *Annual Review of Fluid Mechanics*, vol. 54, pp. 469–493, 2022. doi: [10.1146/annurev-fluid-031621-085520](https://doi.org/10.1146/annurev-fluid-031621-085520) [Online]. Available: <https://doi.org/10.1146/annurev-fluid-031621-085520>
- [2] K. Fukami and K. Taira, “Grasping extreme aerodynamics on a low-dimensional manifold,” *Nature Communications*, vol. 14, p. 6480, 2023. doi: [10.1038/s41467-023-42213-6](https://doi.org/10.1038/s41467-023-42213-6) [Online]. Available: <https://doi.org/10.1038/s41467-023-42213-6>
- [3] S. L. Brunton, B. R. Noack, and P. Koumoutsakos, “Machine learning for fluid mechanics,” *Annual Review of Fluid Mechanics*, vol. 52, pp. 477–508, 2020. doi: [10.1146/annurev-fluid-010719-060214](https://doi.org/10.1146/annurev-fluid-010719-060214) [Online]. Available: <https://doi.org/10.1146/annurev-fluid-010719-060214>
- [4] K. Boahen, “Neuromorphic microchips,” *Scientific American*, vol. 292, no. 5, pp. 56–63, 2005. doi: [10.1038/scientificamerican0505-56](https://doi.org/10.1038/scientificamerican0505-56) [Online]. Available: <https://doi.org/10.1038/scientificamerican0505-56>
- [5] W. Maass, “Networks of spiking neurons: The third generation of neural network models,” *Neural Networks*, vol. 10, no. 9, pp. 1659–1671, 1997. doi: [10.1016/S0893-6080\(97\)00011-7](https://doi.org/10.1016/S0893-6080(97)00011-7) [Online]. Available: [https://doi.org/10.1016/S0893-6080\(97\)00011-7](https://doi.org/10.1016/S0893-6080(97)00011-7)
- [6] A. Tavanaei, M. Ghodrati, S. R. Kheradpisheh, T. Masquelier, and A. Maida, “Deep learning in spiking neural networks,” *Neural Networks*, vol. 111, pp. 47–63, 2019. doi: [10.1016/j.neunet.2018.12.002](https://doi.org/10.1016/j.neunet.2018.12.002) [Online]. Available: <https://doi.org/10.1016/j.neunet.2018.12.002>
- [7] J. D. Nunes, M. Carvalho, D. Carneiro, and J. S. Cardoso, “Spiking neural networks: A survey,” *IEEE Access*, vol. 10, pp. 60 738–60 764, 2022. doi: [10.1109/ACCESS.2022.3179968](https://doi.org/10.1109/ACCESS.2022.3179968) [Online]. Available: <https://doi.org/10.1109/ACCESS.2022.3179968>
- [8] K. Roy, A. Jaiswal, and P. Panda, “Towards spike-based machine intelligence with neuromorphic computing,” *Nature*, vol. 575, no. 7784, pp. 607–617, 2019. doi: [10.1038/s41586-019-1677-2](https://doi.org/10.1038/s41586-019-1677-2) [Online]. Available: <https://doi.org/10.1038/s41586-019-1677-2>
- [9] L. Paulun, A. Wendt, and N. Kasabov, “A retinotopic spiking neural network system for accurate recognition of moving objects using neucube and dynamic vision sensors,” *Frontiers in Computational Neuroscience*, vol. 12, p. 42, 2018. doi:

- 10.3389/fncom.2018.00042 [Online]. Available: <https://doi.org/10.3389/fncom.2018.00042>
- [10] M. Davies et al., “Advancing neuromorphic computing with loihi: A survey of results and outlook,” *Proceedings of the IEEE*, vol. 109, no. 5, pp. 911–934, 2021. doi: 10.1109/JPROC.2021.3067593 [Online]. Available: <https://doi.org/10.1109/JPROC.2021.3067593>
 - [11] K. Fukami, L. Smith, and K. Taira, “Extreme vortex-gust airfoil interactions at Reynolds number 5000,” *Physical Review Fluids*, vol. 10, no. 8, p. 084 703, 2025. doi: 10.1103/vcbd-tvz1 [Online]. Available: <https://doi.org/10.1103/vcbd-tvz1>
 - [12] J. Zhao, M. Monforte, G. Indiveri, C. Bartolozzi, and E. Donati, “Learning inverse kinematics using neural computational primitives on neuromorphic hardware,” *npj Robotics*, vol. 1, p. 1, 2023. doi: 10.1038/s44182-023-00001-w [Online]. Available: <https://doi.org/10.1038/s44182-023-00001-w>
 - [13] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998. doi: 10.1109/5.726791 [Online]. Available: <https://doi.org/10.1109/5.726791>
 - [14] K. Fukami, H. Nakao, and K. Taira, “Data-driven transient lift attenuation for extreme vortex gust-airfoil interactions,” *Journal of Fluid Mechanics*, vol. 992, A17, 2024. doi: 10.1017/jfm.2024.592 [Online]. Available: <https://doi.org/10.1017/jfm.2024.592>
 - [15] H. Biler, G. Sedky, A. R. Jones, M. Saritas, and O. Cetiner, “Experimental investigation of transverse and vortex gust encounters at low reynolds numbers,” *AIAA Journal*, vol. 59, no. 3, pp. 786–799, 2021. doi: 10.2514/1.J059658 [Online]. Available: <https://doi.org/10.2514/1.J059658>
 - [16] Y. Qian, Z. Wang, and I. Gursul, “Interaction of quasi-two-dimensional vortical gusts with airfoils, unswept and swept wings,” *Experiments in Fluids*, vol. 63, no. 8, p. 124, 2022. doi: 10.1007/s00348-022-03477-8 [Online]. Available: <https://doi.org/10.1007/s00348-022-03477-8>
 - [17] H. G. Küssner, “Zusammenfassender Bericht über den instationären Auftrieb von Flügeln,” *Luftfahrtforschung*, vol. 13, no. 12, pp. 410–424, 1936.
 - [18] W. R. Sears, “Some aspects of non-stationary airfoil theory and its practical application,” *Journal of the Aeronautical Sciences*, vol. 8, no. 3, pp. 104–108, 1941. doi: 10.2514/8.10655 [Online]. Available: <https://doi.org/10.2514/8.10655>
 - [19] A. R. Jones and O. Cetiner, “Overview of unsteady aerodynamic response of rigid wings in gust encounters,” *AIAA Journal*, vol. 59, no. 2, pp. 731–736, 2021. doi: 10.2514/1.J059602 [Online]. Available: <https://doi.org/10.2514/1.J059602>

- [20] J. G. Leishman and T. S. Beddoes, “A semi-empirical model for dynamic stall,” *Journal of the American Helicopter Society*, vol. 34, no. 3, pp. 3–17, 1989. doi: [10.4050/JAHS.34.3.3](https://doi.org/10.4050/JAHS.34.3.3) [Online]. Available: <https://doi.org/10.4050/JAHS.34.3.3>
- [21] K. Hornik, M. Stinchcombe, and H. White, “Multilayer feedforward networks are universal approximators,” *Neural Networks*, vol. 2, no. 5, pp. 359–366, 1989. doi: [10.1016/0893-6080\(89\)90020-8](https://doi.org/10.1016/0893-6080(89)90020-8) [Online]. Available: [https://doi.org/10.1016/0893-6080\(89\)90020-8](https://doi.org/10.1016/0893-6080(89)90020-8)
- [22] J. L. Elman, “Finding structure in time,” *Cognitive Science*, vol. 14, no. 2, pp. 179–211, 1990. doi: [10.1207/s15516709cog1402_1](https://doi.org/10.1207/s15516709cog1402_1) [Online]. Available: https://doi.org/10.1207/s15516709cog1402_1
- [23] I. J. Goodfellow et al., “Generative adversarial nets,” in *Advances in Neural Information Processing Systems*, vol. 27, Curran Associates, Inc., 2014, pp. 2672–2680. [Online]. Available: <https://papers.nips.cc/paper/5423-generative-adversarial-nets>
- [24] M. Raissi, P. Perdikaris, and G. E. Karniadakis, “Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations,” *Journal of Computational Physics*, vol. 378, pp. 686–707, 2019. doi: [10.1016/j.jcp.2018.10.045](https://doi.org/10.1016/j.jcp.2018.10.045) [Online]. Available: <https://doi.org/10.1016/j.jcp.2018.10.045>
- [25] L. Guastoni et al., “Convolutional-network models to predict wall-bounded turbulence from wall quantities,” *Journal of Fluid Mechanics*, vol. 928, A27, 2021. doi: [10.1017/jfm.2021.812](https://doi.org/10.1017/jfm.2021.812) [Online]. Available: <https://doi.org/10.1017/jfm.2021.812>
- [26] J. Borée, “Extended proper orthogonal decomposition: A tool to analyse correlated events in turbulent flows,” *Experiments in Fluids*, vol. 35, no. 2, pp. 188–192, 2003. doi: [10.1007/s00348-003-0656-3](https://doi.org/10.1007/s00348-003-0656-3) [Online]. Available: <https://doi.org/10.1007/s00348-003-0656-3>
- [27] A. Güemes, S. Discetti, A. Ianiro, B. Sirmacek, H. Azizpour, and R. Vinuesa, “From coarse wall measurements to turbulent velocity fields through deep learning,” *Physics of Fluids*, vol. 33, no. 7, p. 075 121, 2021. doi: [10.1063/5.0058346](https://doi.org/10.1063/5.0058346) [Online]. Available: <https://doi.org/10.1063/5.0058346>
- [28] A. Cuéllar, A. Güemes, A. Ianiro, Ó. Flores, R. Vinuesa, and S. Discetti, “Three-dimensional generative adversarial networks for turbulent flow estimation from wall measurements,” *Journal of Fluid Mechanics*, vol. 991, A1, 2024. doi: [10.1017/jfm.2024.432](https://doi.org/10.1017/jfm.2024.432) [Online]. Available: <https://doi.org/10.1017/jfm.2024.432>

- [29] J. L. Lumley, “The structure of inhomogeneous turbulent flows,” in *Atmospheric Turbulence and Radio Wave Propagation*, A. M. Yaglom and V. I. Tatarsky, Eds., Moscow: Nauka, 1967, pp. 166–178. [Online]. Available: <https://cir.nii.ac.jp/crid/1571980075051475712?lang=en>
- [30] G. Berkooz, P. Holmes, and J. L. Lumley, “The proper orthogonal decomposition in the analysis of turbulent flows,” *Annual Review of Fluid Mechanics*, vol. 25, pp. 539–575, 1993. doi: [10.1146/annurev.fl.25.010193.002543](https://doi.org/10.1146/annurev.fl.25.010193.002543) [Online]. Available: <https://doi.org/10.1146/annurev.fl.25.010193.002543>
- [31] P. J. Schmid, “Dynamic mode decomposition of numerical and experimental data,” *Journal of Fluid Mechanics*, vol. 656, pp. 5–28, 2010. doi: [10.1017/S0022112010001217](https://doi.org/10.1017/S0022112010001217) [Online]. Available: <https://doi.org/10.1017/S0022112010001217>
- [32] C. W. Rowley, I. Mezić, S. Bagheri, P. Schlatter, and D. S. Henningson, “Spectral analysis of nonlinear flows,” *Journal of Fluid Mechanics*, vol. 641, pp. 115–127, 2009. doi: [10.1017/S0022112009992059](https://doi.org/10.1017/S0022112009992059) [Online]. Available: <https://doi.org/10.1017/S0022112009992059>
- [33] K. Taira et al., “Modal analysis of fluid flows: An overview,” *AIAA Journal*, vol. 55, no. 12, pp. 4013–4041, 2017. doi: [10.2514/1.J056060](https://doi.org/10.2514/1.J056060) [Online]. Available: <https://doi.org/10.2514/1.J056060>
- [34] G. E. Hinton and R. R. Salakhutdinov, “Reducing the dimensionality of data with neural networks,” *Science*, vol. 313, no. 5786, pp. 504–507, 2006. doi: [10.1126/science.1127647](https://doi.org/10.1126/science.1127647) [Online]. Available: <https://doi.org/10.1126/science.1127647>
- [35] T. Murata, K. Fukami, and K. Fukagata, “Nonlinear mode decomposition with convolutional neural networks for fluid dynamics,” *Journal of Fluid Mechanics*, vol. 882, A13, 2020. doi: [10.1017/jfm.2019.822](https://doi.org/10.1017/jfm.2019.822) [Online]. Available: <https://doi.org/10.1017/jfm.2019.822>
- [36] P. Baldi and K. Hornik, “Neural networks and principal component analysis: Learning from examples without local minima,” *Neural Networks*, vol. 2, no. 1, pp. 53–58, 1989. doi: [10.1016/0893-6080\(89\)90014-2](https://doi.org/10.1016/0893-6080(89)90014-2) [Online]. Available: [https://doi.org/10.1016/0893-6080\(89\)90014-2](https://doi.org/10.1016/0893-6080(89)90014-2)
- [37] K. Fukami, T. Nakamura, and K. Fukagata, “Convolutional neural network based hierarchical autoencoder for nonlinear mode decomposition of fluid field data,” *Physics of Fluids*, vol. 32, no. 9, p. 095110, 2020. doi: [10.1063/5.0020721](https://doi.org/10.1063/5.0020721) [Online]. Available: <https://doi.org/10.1063/5.0020721>
- [38] M. Raffel, C. E. Willert, F. Scarano, C. J. Kähler, S. T. Wereley, and J. Kompenhans, *Particle Image Velocimetry: A Practical Guide*, 3rd ed. Cham: Springer International Publishing, 2018. doi: [10.1007/978-3-319-68852-7](https://doi.org/10.1007/978-3-319-68852-7) [Online]. Available: <https://doi.org/10.1007/978-3-319-68852-7>

- [39] L. Zhu, J. Li, X. Wang, T. Huang, and Y. Tian, “NeuSpike-Net: High speed video reconstruction via bio-inspired neuromorphic cameras,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2021, pp. 2400–2409. [Online]. Available: https://openaccess.thecvf.com/content/ICCV2021/html/Zhu_NeuSpike-Net_High-Speed_Video_Reconstruction_via_Bio-Inspired_Neuromorphic_Cameras_ICCV_2021_paper.html
- [40] A. Viale, A. Marchisio, M. Martina, G. Masera, and M. Shafique, “CarSNN: An efficient spiking neural network for event-based autonomous cars on the Loihi neuromorphic research processor,” in *2021 International Joint Conference on Neural Networks (IJCNN)*, 2021, pp. 1–8. doi: [10.1109/IJCNN52387.2021.9533738](https://doi.org/10.1109/IJCNN52387.2021.9533738) [Online]. Available: <https://doi.org/10.1109/IJCNN52387.2021.9533738>
- [41] M. Pfeiffer and T. Pfeil, “Deep learning with spiking neurons: Opportunities and challenges,” *Frontiers in Neuroscience*, vol. 12, p. 774, 2018. doi: [10.3389/fnins.2018.00774](https://doi.org/10.3389/fnins.2018.00774) [Online]. Available: <https://doi.org/10.3389/fnins.2018.00774>
- [42] W. Gerstner, W. M. Kistler, R. Naud, and L. Paninski, *Neuronal Dynamics: From Single Neurons to Networks and Models of Cognition*. Cambridge: Cambridge University Press, 2014. [Online]. Available: <https://neurondynamics.epfl.ch/>
- [43] A. L. Hodgkin and A. F. Huxley, “A quantitative description of membrane current and its application to conduction and excitation in nerve,” *The Journal of Physiology*, vol. 117, no. 4, pp. 500–544, 1952. doi: [10.1113/jphysiol.1952.sp004764](https://doi.org/10.1113/jphysiol.1952.sp004764) [Online]. Available: <https://doi.org/10.1113/jphysiol.1952.sp004764>
- [44] E. M. Izhikevich, “Simple model of spiking neurons,” *IEEE Transactions on Neural Networks*, vol. 14, no. 6, pp. 1569–1572, 2003. doi: [10.1109/TNN.2003.820440](https://doi.org/10.1109/TNN.2003.820440) [Online]. Available: <https://doi.org/10.1109/TNN.2003.820440>
- [45] R. Brette and W. Gerstner, “Adaptive exponential integrate-and-fire model as an effective description of neuronal activity,” *Journal of Neurophysiology*, vol. 94, no. 5, pp. 3637–3642, 2005. doi: [10.1152/jn.00686.2005](https://doi.org/10.1152/jn.00686.2005) [Online]. Available: <https://doi.org/10.1152/jn.00686.2005>
- [46] J. K. Eshraghian et al., “Training spiking neural networks using lessons from deep learning,” *Proceedings of the IEEE*, vol. 111, no. 9, pp. 1016–1054, 2023. doi: [10.1109/JPROC.2023.3308088](https://doi.org/10.1109/JPROC.2023.3308088) [Online]. Available: <https://doi.org/10.1109/JPROC.2023.3308088>
- [47] E. O. Neftci, H. Mostafa, and F. Zenke, “Surrogate gradient learning in spiking neural networks: Bringing the power of gradient-based optimization to spiking neural networks,” *IEEE Signal Processing Magazine*, vol. 36, no. 6, pp. 51–63, 2019. doi: [10.1109/MSP.2019.2931595](https://doi.org/10.1109/MSP.2019.2931595) [Online]. Available: <https://doi.org/10.1109/MSP.2019.2931595>

- [48] R. J. Williams and J. Peng, “An efficient gradient-based algorithm for on-line training of recurrent network trajectories,” *Neural Computation*, vol. 2, no. 4, pp. 490–501, 1990. doi: [10.1162/neco.1990.2.4.490](https://doi.org/10.1162/neco.1990.2.4.490) [Online]. Available: <https://doi.org/10.1162/neco.1990.2.4.490>
- [49] M. Davies et al., “Loihi: A neuromorphic manycore processor with on-chip learning,” *IEEE Micro*, vol. 38, no. 1, pp. 82–99, 2018. doi: [10.1109/MM.2018.112130359](https://doi.org/10.1109/MM.2018.112130359) [Online]. Available: <https://doi.org/10.1109/MM.2018.112130359>
- [50] P. U. Diehl and M. Cook, “Unsupervised learning of digit recognition using spike-timing-dependent plasticity,” *Frontiers in Computational Neuroscience*, vol. 9, p. 99, 2015. doi: [10.3389/fncom.2015.00099](https://doi.org/10.3389/fncom.2015.00099) [Online]. Available: <https://doi.org/10.3389/fncom.2015.00099>
- [51] B. Rueckauer, I.-A. Lungu, Y. Hu, M. Pfeiffer, and S.-C. Liu, “Conversion of continuous-valued deep networks to efficient event-driven networks for image classification,” *Frontiers in Neuroscience*, vol. 11, p. 682, 2017. doi: [10.3389/fnins.2017.00682](https://doi.org/10.3389/fnins.2017.00682) [Online]. Available: <https://doi.org/10.3389/fnins.2017.00682>
- [52] A. Ianiro, I. Tirelli, and S. Discetti, “Event-based sensing for turbulent flows: A measurement science perspective on information-efficient estimation and control,” Manuscript submitted to Measurement Science and Technology, 2026.
- [53] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in *International Conference on Learning Representations*, 2019. [Online]. Available: <https://openreview.net/forum?id=Bkg6RiCqY7>
- [54] M. Horowitz, “1.1 computing’s energy problem (and what we can do about it),” in *2014 IEEE International Solid-State Circuits Conference Digest of Technical Papers*, 2014, pp. 10–14. doi: [10.1109/ISSCC.2014.6757323](https://doi.org/10.1109/ISSCC.2014.6757323)
- [55] European Parliament and Council of the European Union, *Regulation (eu) 2024/1689 laying down harmonised rules on artificial intelligence*, Official Journal of the European Union, 2024.
- [56] European Parliament and Council of the European Union, *Regulation (eu) 2018/1139 on common rules in the field of civil aviation and establishing a european union aviation safety agency*, Official Journal of the European Union, 2018.
- [57] European Union Aviation Safety Agency, *Artificial intelligence roadmap 2.0: A human-centric approach to ai in aviation*, 2023.
- [58] European Union Aviation Safety Agency, *Artificial intelligence concept paper issue 2: Guidance for level 1 and level 2 machine learning applications*, 2024.
- [59] European Union Aviation Safety Agency, *Amc 20-115d: Airborne software development assurance using eurocae ed-12 and rtca do-178*, 2017.

- [60] European Commission, *Commission implementing regulation (eu) 2023/203 on the management of information security risks with a potential impact on aviation safety*, Official Journal of the European Union, 2023.
- [61] Jefatura del Estado, *Real decreto legislativo 1/1996 por el que se aprueba el texto refundido de la ley de propiedad intelectual*, Boletín Oficial del Estado, 1996.
- [62] NVIDIA Corporation, *Nvidia ampere ga102 gpu architecture*, GeForce RTX 3090 total graphics power: 350 W, 2021.

A. REGULATORY FRAMEWORK

This thesis presents an academic prototype for reconstructing the lift coefficient from spike-encoded numerical vorticity fields. The model is evaluated offline using simulation data and is not connected to an aircraft, flight-control system, or certified airborne product. The current implementation is therefore not an operational aviation system. Nevertheless, its possible future integration into an aircraft would require compliance with artificial-intelligence regulation, aviation safety requirements, software-assurance standards, and information-security rules.

The main general regulation relevant to this thesis is Regulation (EU) 2024/1689, known as the Artificial Intelligence Act [55]. The present work is limited to research and does not place an artificial intelligence system on the market or put one into operational service. Therefore, its relevance to the Regulation is mainly prospective. A future version used as part of an aircraft safety function could fall within the high-risk framework if it were considered a safety component subject to third-party conformity assessment. Such a system would require documented risk management, controlled training and validation data, technical documentation, traceability, human oversight, and evidence of accuracy, robustness, and cybersecurity.

An airborne implementation would also fall under Regulation (EU) 2018/1139, which establishes the common European framework for civil-aviation safety and the responsibilities of the European Union Aviation Safety Agency (EASA) [56]. EASA's Artificial Intelligence Roadmap 2.0 and its Concept Paper for Level 1 and Level 2 machine learning applications provide guidance for the development of learning-based aviation systems [57, 58].

Established airborne-software assurance practices would also need to be considered. EUROCAE ED-12C and RTCA DO-178C are recognised by EASA through AMC 20-115D as an acceptable means of compliance for the software aspects of airborne certification [59]. Under this framework, the preprocessing pipeline, network implementation, trained parameters, and execution environment would require controlled development and verification. Furthermore, the reconstruction metrics reported in this thesis would provide useful performance evidence, but they would not by themselves demonstrate compliance with aviation safety requirements.

These requirements are directly related to the limitations of the proposed system. The model has been trained using a finite numerical dataset and has not been evaluated with flight-test measurements, realistic sensor errors, hardware failures, or aircraft configurations outside the available data. Its validity is limited to the aerodynamic conditions represented during development and testing. The current model must therefore not be regarded as flight-ready or used as the sole source of information for a safety-critical decision.

A deployable system would require a clearly defined operational domain stating the conditions under which its output is considered valid. It should also detect inputs outside the validated range, monitor failures or unreliable outputs, and provide a safe fallback when the model cannot be trusted.

Cybersecurity would be relevant in a future application. An operational implementation would need to protect sensor inputs, trained parameters, software updates, configuration files, and output signals against accidental corruption and deliberate manipulation. Commission Implementing Regulation (EU) 2023/203 and the associated EASA Part-IS framework address the management of information-security risks that may affect aviation safety [60]. For the proposed system, this would require control of the integrity and authenticity of both the model and the information exchanged with other aircraft systems.

Finally, the development and publication of the thesis must respect intellectual-property and licensing conditions. The original text and software are protected under the Spanish Intellectual Property Act, while external software, datasets, and reproduced figures remain subject to their corresponding licences [61]. Any public repository should identify third-party dependencies, state the licence applied to the original code, cite the source dataset, and preserve the attribution conditions of reused material.

B. SOCIO-ECONOMIC IMPACT

This annex estimates the resources used to complete the thesis and discusses the possible economic, social, environmental, and ethical effects of its results. The current work is an academic proof of concept.

B.1. Project Budget

The budget estimates the direct financial resources required to replicate the computational experiments presented in this thesis. It includes the acquisition of the GPU used for training and the electricity consumed during the reported model-training runs. Personnel costs are not included because the purpose of the estimate is to quantify the technical resources needed to reproduce the work.

All models were trained using an NVIDIA GeForce RTX 3090 GPU. The reference model and five additional configurations each required approximately 24 hours of GPU-intensive training. These configurations correspond to the delta-only encoding, the hybrid input representation, the random-window training strategy, and latent-space dimensions $N_z = 8$ and $N_z = 32$. The reference model trained at the original temporal resolution required approximately 6 hours. Finally, the parameter-matched and energy-matched CNN models required approximately 4 hours and 3.5 hours, respectively.

Training task	Number of runs	GPU time [h]
Reference and main ablation models	6	144.0
Reference model using original temporal grid	1	6.0
Parameter-matched CNN model	1	4.0
Energy-matched CNN model	1	3.5
Total	9	157.5

Table B.1. Estimated GPU training time for the experiments conducted in this thesis.

The RTX 3090 has a reference total graphics power of 350W [62]. Assuming that the GPU operated close to this value throughout training, the corresponding electricity consumption is estimated as

$$E_{\text{GPU}} = 157.5 \text{ h} \times 0.350 \text{ kW} = 55.125 \text{ kWh.} \quad (\text{B.1})$$

Using an assumed electricity price of 0.25€/kWh, the estimated electricity cost is 13.78€. This calculation includes only the GPU power consumption. The additional consumption of the CPU, memory, storage devices, power-supply losses, and cooling system was not

measured. The result should therefore be interpreted as a lower-bound estimate of the total electricity cost.

Inference over the complete test set of 24 encounters requires approximately two minutes per model. Assuming one complete inference evaluation for each of the nine trained models, the total inference time amounts to 0.3 h. This computational time is negligible compared with the 157.5 hours required for model training, thus it has been discarded from the total costs.

No acquisition cost is assigned to the aerodynamic dataset because it is publicly available. The numerical simulations used to create the dataset were not repeated as part of this thesis. The software environment was also based on freely available tools and open-source machine learning libraries, so no licensing costs for specific software are included.

Table B.2 presents the resulting replication budget. The GPU is included at an estimated acquisition cost of 1,549€. This estimate assumes that a compatible workstation, including the CPU, memory, storage, motherboard, cooling system, and power supply, is already available. If a complete workstation had to be acquired specifically for replication, the cost of these additional components would need to be included.

Resource	Quantity	Cost [€]
NVIDIA GeForce RTX 3090	1 unit	1,549.00
GPU electricity consumption	55.125 kWh	13.78
Total estimated replication cost		1,562.78

Table B.2. Estimated direct cost of replicating the computational work.

The hardware acquisition represents almost all of the estimated budget. If an RTX 3090 or an equivalent GPU is already available, the additional direct cost of reproducing the complete training procedure is approximately 13.78€.

B.2. Expected Socio-Economic Impact

The main potential application of this work is the development of compact aerodynamic-load estimation systems for small air vehicles, particularly unmanned aerial vehicles. These platforms are especially sensitive to strong gusts because of their low flight velocities and reduced inertia, while their limited payload capacity places strict constraints on the mass, size, and power consumption of onboard computers. Installing larger GPUs would impose considerable weight and power consumption penalties. In contrast, a compact neuromorphic processor designed for event-driven computation could provide a more suitable platform for an SNN-based estimator. A future system could combine upstream or surface sensors with such a processor to estimate or predict gust-induced loads in real time.

The potential economic and operational value of the proposed approach lies partly in reducing the computational resources required for continuous onboard estimation. Neuromorphic hardware is specifically designed to exploit the sparse and event-driven operation of SNNs, so the energy advantage over conventional neural networks could be greater on this type of processor. A lower power demand could reduce the required battery capacity, cooling and onboard computing mass, which is particularly relevant for small unmanned vehicles.

The energy-efficiency benefit is not limited to future neuromorphic hardware. The analysis presented in this thesis indicates that the reference SNN has a lower estimated inference energy than the evaluated CNN alternatives, even before considering deployment on dedicated neuromorphic hardware. This difference is mainly associated with sparse spike activity and the replacement of many multiply-accumulate operations with lower-cost accumulation operations. The result therefore suggests that part of the computational advantage comes from the spiking architecture itself, although specialised neuromorphic hardware could increase this advantage further.

At system level, lower inference power could enable smaller and lighter onboard computers with reduced cooling requirements, which would be particularly valuable for unmanned aerial vehicles. These effects remain prospective because the present model reconstructs the instantaneous lift coefficient from full-field numerical data. It has not yet been connected to physical sensors or extended to predict the load before the gust reaches the airfoil.

The main social benefit would be improved safety and operational reliability. Small aircraft and unmanned aerial vehicles are particularly vulnerable to strong disturbances because of their low flight speed and limited inertia. A reliable onboard load-estimation system could support inspection, environmental monitoring, emergency response, or other missions carried out in difficult atmospheric conditions.

This potential benefit also introduces a risk. An inaccurate estimate may create false confidence in the vehicle's ability to withstand a gust or continue a mission safely. The system should therefore be used only within the conditions for which it has been validated, and its limitations should remain clear to operators and system designers.

The environmental impact of the research is modest, although it is not zero. The estimated GPU energy consumption associated with model training is 55.125 kWh. This figure excludes the remaining workstation components and does not account for the environmental cost of manufacturing the hardware. However, the use of an existing public dataset avoided the much larger computational expense of generating a new set of numerical simulations.

The principal ethical issue concerns how the model's performance is interpreted and communicated. The available dataset covers a limited range of angles of attack and gust parameters, and the results show that reconstruction accuracy is not uniform across all encounters. In particular, differences between positive and negative gusts, together with

the presence of difficult outlier cases, show that overall averages may conceal poor performance under specific conditions.

Future work should therefore examine whether the training data are sufficiently balanced and representative of the intended application. Failure cases should be reported explicitly and claims about generalisation should remain consistent with the available evidence.

C. CODE AVAILABILITY

The source code developed for this thesis is publicly available in the following GitHub repository:

<https://github.com/jmanresarigo/SNN-vortex-gust-lift-reconstruction>

The repository contains the code used for data pre-processing, temporal interpolation, spike encoding, model training, and the accuracy and energy-efficiency evaluation of the reference SNN and the two CNN-based baselines.

The original aerodynamic dataset is not redistributed in the repository and must be obtained from the source indicated in Annex D. The interpolated dataset used to train and evaluate the reference model can be reproduced from the original data using the script named `dataset_interpolation.py` provided in the repository.

D. DATA AVAILABILITY

The original dataset used in this thesis was produced by Fukami and Taira as part of their study on the low-dimensional representation of extreme vortex-gust airfoil interactions [2]. The dataset contains the time-resolved vorticity fields and corresponding lift coefficient histories employed throughout the present work. It is publicly available through the Open Science Framework repository at:

<https://doi.org/10.17605/OSF.IO/7VSH8>

The temporally interpolated dataset used to train and evaluate the reference SNN model is a derived version of the original data and is not distributed separately. It can be reproduced using the `dataset_interpolation.py` script found in the GitHub repository of this thesis. After downloading the original dataset from the repository above, this script applies the cubic-spline temporal interpolation described in Ch. 5, thus generating the dataset used in the reference model experiments.

E. AI DECLARATION OF USE

In accordance with the guidelines of Universidad Carlos III de Madrid, the completed AI Declaration of Use form for this Master Thesis is included in the following pages.

DECLARATION OF USE OF GENERATIVE ARTIFICIAL INTELLIGENCE (AI) IN MASTER FINAL PROJECT (TFM)

I have used Generative AI in my TFM

Check all that apply:

YES	NO
-----	----

If you have ticked YES, please complete the following 3 parts of this document:

Part 1: Reflection on legal, ethical and responsible behavior

Please be aware that the use of AI carries some risks and may lead to a series of serious academic consequences: the TFM will not be evaluated by the University if the use of AI involves the use of confidential data, copyrighted materials, or personal data, used without complying with the conditions required in each case (authorization of the interested parties, authorization of the owners, following the instructions of the University).

Question	
1. In my interactions with AI tools, I have provided confidential data, always with the appropriate authorization of the data subjects. Confidentiality encompasses any information that a person or organization wishes to protect for legal, commercial, privacy, or strategic reasons (such as patents or trade secrets).	
<u>YES, I have used this data with permission</u>	NO, I have not used confidential data
2. In my interaction with Generative AI tools, I have submitted copyrighted materials with the permission of those concerned.	
<u>YES, I have used these materials with permission from the copyright holders, or without permission because they fall within one of the exceptions or limitations permitted by law:</u> • <u>Public domain work</u> • <u>Licensed work (Creative Commons licenses)</u>	NO, I have not used copyrighted materials

• <u>Use of excerpts for research purposes (fair use)</u>	
3. In my interaction with Generative AI tools, I have submitted personal data with the consent of the data subjects.	
<u>YES, I have used this data with the authorization of the interested parties and in accordance with the instructions contained in the guide approved by the University.</u>	NO, I have not used personal data
4. My use of the Generative AI tool has respected its terms of use, as well as the essential ethical principles, not being maliciously oriented to obtain an inappropriate result for the work presented, that is to say, one that produces an impression or knowledge contrary to the reality of the results obtained, that supplants my own work or that could harm people.	
<u>YES</u>	NO

Part 2: Declaration of technical use

Use the following model statement as many times as necessary, in order to reflect all types of iteration you have had with Generative AI tools. Include one example for each type of use where indicated: *[Add an example]*.

I declare that I have made use of the Generative AI system (Name of AI system/tool and version: ChatGPT, Gemini, Copilot...) **for:**

Documentation and drafting:

- *Supporting reflection in relation to the development of the work: iterative process of analysing alternatives and approaches using AI*

I have used different LLM models for brainstorming and exploring different ideas to address the problem stated at the beginning of the paper.

- *Revision or rewriting of previously drafted paragraphs*

I have asked for grammatical correction and proofreading the paragraphs to ensure the text was free of errors. Furthermore, to ensure text cohesion in some parts of the thesis, I explicitly asked to change the wording of some of my paragraphs. That being said, the same original ideas were always kept.

- *Search for information or answers to specific questions*

The questions asked were aimed at trying to further understand a technical concept by splitting it into simpler parts, understanding some sections of scientific papers, asking for examples of use of a certain technical solution.

- *Summary of bibliography consulted*

With the aim to speed up the bibliography research process, I asked the LLMs to summarize papers and their key findings to determine whether they were relevant or not for my thesis objective.

Develop specific content

Generative AI has been used as a support tool for the development of the specific content of the TFM, including:

- *Assistance in the development of lines of code (programming)*

In certain sections of my codebase, vibe coding was used in the following manner: I would explain extensively the task to be done with requirements, limitations, objectives, background, desired inputs and outputs, output organization within my directory and edge cases considerations. Then the LLM would complete the task and I would revise the code generated by it, as it is the source of truth. This was an iterative process until the output obtained was something I considered.

- *Generation of diagrams, images, audios or videos*

No diagrams, images, audios or videos in the present thesis were generated by AI.

- *Optimisation processes*

I have asked for help to correct and explain errors in a code that was not working properly in order to speed up the progress.

- *Data processing: collection, analysis, cross-checking of data...*

Brainstorming using AI was performed regarding processes needed to tackle some data processing problems that would be crucial for the subsequent analysis.

- *Inspiration of ideas in the creative process*

Due to the lack of published work regarding my thesis topic, I sometimes found myself having the objective very clear but with lack of creative ideas to tackle a certain problem. In these cases, AI has been very helpful to hint me toward a creative idea that would help me get closer to my objective.

- *Other uses linked to the generation of specific points of the specific development of the paper*

If you think it is necessary to include it, add it in each case:

using the prompt ...

(Write the request made to the AI)

having as interaction ...

(Describe what interaction you made with the AI after the prompt's response)

Example: *I declare that I have made use of the Generative AI system Chat GPT X.x to search for information using the prompt: "Tell me a concrete example that illustrates the application of the urban planning proposal of the 15-minute City in Spain", having as interaction the proposition of the concrete example of the Chamartín district that has been reflected in the dissertation.*

Part 3: Reflection on utility

Please provide a personal assessment (free format) of the strengths and weaknesses you have identified in the use of Generative AI tools in the development of your work. Mention if it has helped you in the learning process, or in the development or drawing conclusions from your work.

When working in the forefront of a specific research topic, creativity is very important yet difficult to find. Having a conversation with an AI can help one break this creativity barrier and explore new trains of thought that would have been otherwise harder to find. Furthermore, code generation in moments of less inspiration has proven to help me make great progress that would have been very hard if I had no access to these tools. The downsides were typical LLM problems such as hallucinations and fixing one error but creating another one. These are problems that needed to be considered when using these tools. That is why I never trusted an LLM response without putting proper thought on it.